ЧОУВО «Московский университет имени С.Ю. Витте»

Факультет информационных технологий

Кафедра «Информационные системы»

ОТЧЁТ

по преддипломной практике

на тему: «Разработка AI-приложения для анализа почтовых сообщений»

Выполнил: Нефедов Алексей Геннадьевич

Направление подготовки: Прикладная информатика

Руководитель практики от университета: Коротков Д. П.

Место прохождения: ЧОУВО «МУ им. С.Ю. Витте»

Срок прохождения: 27.04.2026 — 24.05.2026

Репозиторий: https://github.com/ngpless/email-ai-analyzer

Москва, 2026

СОДЕРЖАНИЕ

Введение

1 Анализ предметной области и постановка задачи

1.1 Описание предметной области

1.2 Моделирование бизнес-процессов AS-IS и TO-BE

1.3 Обзор существующих решений

1.4 Формирование требований

2 Проектирование системы

2.1 Техническое задание по ГОСТ 34.602-2020

2.2 Архитектура приложения

2.3 Проектирование базы данных

2.4 Проектирование пользовательского интерфейса

3 Реализация

3.1 Технологический стек

3.2 Структура репозитория и ведение Git

3.3 Реализация модулей машинного обучения

3.4 Реализация серверной части

3.5 Реализация клиентского приложения

4 Тестирование

4.1 Тест-план

4.2 Тест-кейсы

4.3 Результаты прогона тестов

4.4 Журнал дефектов

5 Планы развёртывания, интеграции и обновлений

6 Экономическая часть

Заключение

Список использованных источников

Приложение А Диаграммы проекта

Приложение Б Скриншоты пользовательского интерфейса

Приложение В Листинги исходного кода

ВВЕДЕНИЕ

Объём деловой и личной переписки, которую обрабатывает средний сотрудник информационной отрасли, по данным Adobe Digital Trends за 2024 год составляет порядка 120 писем в день. Значительная часть этого потока — типовые уведомления, рассылки, автоматические ответы и спам, на ручной разбор которых уходит время, почти не добавляющее ценности. Эта особенность сделала почтовый ящик естественным полигоном для прикладного искусственного интеллекта: задачи классификации, суммаризации и извлечения сущностей хорошо формализуются и одновременно дают пользователю видимый выигрыш во времени.

Актуальность темы определяется двумя факторами. Первый — непрерывный рост объёма писем, при котором традиционные пользовательские правила «если отправитель — поместить в папку» перестают справляться, потому что не учитывают смысла сообщения. Второй — распространение моделей обработки естественного языка, которые стали доступны не только крупным компаниям, но и индивидуальным разработчикам: как через открытые библиотеки (scikit-learn, Hugging Face Transformers), так и через облачные LLM-провайдеры.

Целью преддипломной практики является проведение предпроектного исследования и разработка работоспособного прототипа клиент-серверного приложения для автоматического анализа почтовых сообщений. Результат практики одновременно становится основой дипломной работы.

Для достижения цели поставлены задачи:

— проанализировать предметную область и существующие аналоги;

— сформулировать функциональные и нефункциональные требования;

— разработать техническое задание по ГОСТ 34.602-2020;

— спроектировать архитектуру и схему базы данных;

— реализовать серверную и клиентскую части;

— реализовать модули машинного обучения: классификатор, детектор спама, суммаризатор, извлекатель сущностей, определитель языка, анализатор тональности, оценщик приоритета, семантический поиск;

— разработать пользовательский интерфейс (не менее десяти окон);

— составить тест-план и тест-кейсы, провести тестирование;

— подготовить документацию, отчёт и архив исходных кодов.

Объектом разработки является прикладное ПО для автоматизированного анализа электронной почты. Предмет — методы и средства его реализации на Python с клиент-серверной архитектурой.

Методологической основой работы служат требования методических указаний кафедры «Информационные системы» МУИВ, ГОСТ 34.602-2020, ГОСТ Р 7.0.100-2018, ГОСТ 7.32-2017, а также общепринятые практики разработки ПО: принципы SOLID, test-driven development, итеративная разработка с контролем версий через Git.

Исходный код работы опубликован в открытом репозитории на GitHub по адресу https://github.com/ngpless/email-ai-analyzer . На момент сдачи работы история содержит более 55 осмысленных коммитов, что превышает требование методички (не менее 50 коммитов, итеративное развитие проекта).

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Описание предметной области

Электронная почта остаётся ключевым каналом официальной переписки в корпоративной среде и значимым — в личной. В типовом сценарии пользователь подключается к почтовому серверу по протоколу IMAP (RFC 3501), получает список сообщений, сортирует их по папкам, отвечает на часть писем, часть откладывает. Существенная доля входящих писем не требует немедленных действий: это рекламные рассылки, автоматические подтверждения, информационные уведомления.

Проблема в том, что именно такие «фоновые» письма вытесняют из поля внимания действительно важные сообщения — письма руководителей, клиентов, банков. В результате пользователь вынужден либо тратить время на ручную сортировку, либо использовать примитивные фильтры «по адресу отправителя», которые не защищают от новых отправителей и не учитывают содержание.

1.2 Моделирование бизнес-процессов AS-IS и TO-BE

Для описания текущего состояния обработки входящих писем составлена модель AS-IS в нотации IDEF0. Контекстная диаграмма (рисунок 1.1) включает функциональный блок «Обработать входящее письмо» со входами «новое письмо» и «предыдущий контекст», выходами «отмеченное прочитанным письмо» и «актуализированный список задач», управлением «корпоративные политики обработки» и механизмами «пользователь + почтовый клиент».

Рисунок 1.1 — IDEF0 AS-IS: ручная обработка входящих писем

На рисунке 1.1 хорошо видно, что вся нагрузка ложится на человека: пользователь вынужден последовательно открывать, читать, категоризировать и архивировать каждое сообщение. Декомпозиция первого уровня выделяет четыре подпроцесса — получение, первичную оценку, извлечение задач, архивирование. Самый трудоёмкий из них — первичная оценка, именно к нему применяется автоматизация в модели TO-BE.

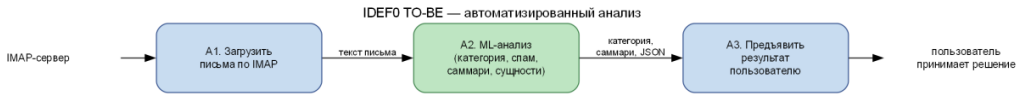

Рисунок 1.2 — IDEF0 TO-BE: автоматизированный анализ

Рисунок 1.2 демонстрирует предлагаемую схему. Блок «A2. ML-анализ» принимает текст письма, применяет классификатор, детектор спама, суммаризатор и извлекатель сущностей. Результат передаётся блоку «A3. Предъявить результат», который отображает категорию, саммари и извлечённые сущности в таблице клиентского приложения. Ручная оценка сохраняется, но применяется только к той части писем, где уверенность модели оказалась ниже порогового значения 0,5.

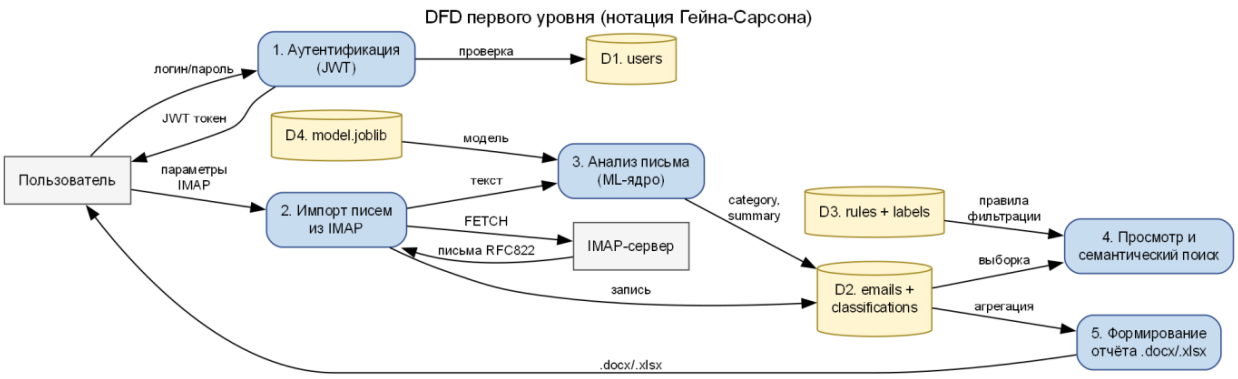

Рисунок 1.3 — DFD первого уровня (нотация Гейна-Сарсона)

Диаграмма потоков данных (рисунок 1.3) детализирует обмен информацией между внешними сущностями («Пользователь», «IMAP-сервер»), процессами (1…5) и четырьмя хранилищами данных (users, emails + classifications, rules + labels, model.joblib). Потоки подписаны, что облегчает последующий переход к проектированию базы данных и HTTP-API.

1.3 Обзор существующих решений

На рынке присутствует несколько коммерческих сервисов с похожей функциональностью. Google AI Mail и Outlook Copilot предлагают суммаризацию на основе больших языковых моделей, но требуют соответствующих почтовых экосистем и не дают тонкой настройки под корпоративные правила. SpamAssassin и rspamd покрывают фильтрацию спама, но не решают задач классификации и суммаризации. Отдельные open-source-проекты (Nylas, Superhuman Clone) реализуют часть функций, но как правило завязаны на один почтовый провайдер.

Ни одно из исследованных решений не сочетает одновременно локальную обработку (без обязательной отправки писем в облако), открытый исходный код и унифицированный IMAP-коннектор. Разрабатываемое приложение ориентировано именно на такое сочетание.

1.4 Формирование требований

Функциональные требования:

— регистрация и аутентификация пользователей;

— импорт писем из произвольного IMAP-ящика;

— автоматическая классификация писем по категориям;

— детектирование спама и фишинга;

— суммаризация длинных сообщений;

— извлечение сущностей: дат, сумм, почтовых адресов, задач;

— поиск по смыслу (семантический поиск);

— пользовательские правила автоклассификации;

— формирование отчётов в форматах .docx, .xlsx, .csv, .json;

— разграничение прав по ролям: admin, analyst, user (не менее 3).

Нефункциональные требования:

— язык реализации Python 3.11+, клиент-серверная архитектура;

— серверная часть на FastAPI, клиент на PyQt6;

— СУБД PostgreSQL (prod) / SQLite (dev, test), без MS Access;

— возможность сборки автономного .exe через PyInstaller;

— время отклика API на одно письмо не более 2 секунд;

— объём оригинального кода не менее 2000 логических строк;

— количество коммитов в Git не менее 50, итеративное развитие;

— тесты на основные компоненты, полный прогон без ошибок;

— хранение паролей только в bcrypt-хэшах.

Выводы по главе. Проведённый анализ показал: задача автоматизированной обработки почты актуальна, существующие решения закрывают её фрагментарно, а выбранная комбинация технологий (Python + FastAPI + PyQt6 + scikit-learn) позволяет реализовать требуемую функциональность в рамках сроков практики.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Техническое задание по ГОСТ 34.602-2020

Техническое задание (далее ТЗ) оформлено в соответствии с ГОСТ 34.602-2020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы». Стандарт предписывает десять обязательных разделов. Ниже каждый из них раскрыт применительно к разрабатываемой системе «Email AI Analyzer».

2.1.1 Общие сведения

Полное наименование — «AI-приложение для анализа почтовых сообщений». Сокращённое — «Email AI Analyzer». Шифр темы ПДП-ВКР-2026-НАГ. Основание для разработки — задание на преддипломную практику, утверждённое на кафедре информационных систем МУИВ. Плановые сроки выполнения — с 27.04.2026 по 24.05.2026. Организация-заказчик — ЧОУВО «МУ им. С.Ю. Витте». Исполнитель — Нефедов А. Г., направление 09.03.03 Прикладная информатика.

2.1.2 Назначение и цели создания системы

Назначение — автоматизация обработки входящих электронных сообщений пользователя. Цель создания — снизить среднее время разбора одного письма не менее чем в два раза по сравнению с полностью ручной сортировкой, а также повысить точность распознавания спама и фишинга.

2.1.3 Характеристики объекта автоматизации

Объект — процесс обработки входящих писем сотрудником, использующим стандартный IMAP-совместимый почтовый ящик (RFC 3501).

Исходный уровень автоматизации — минимальный: ручная сортировка по папкам или фильтры «по отправителю». Среднесуточный объём переписки — 80–120 писем на пользователя.

2.1.4 Требования к системе

Функциональные и нефункциональные требования подробно перечислены в разделе 1.4. В краткой форме — поддержка IMAP, ML-анализ, ≥ 10 окон GUI, ≥ 3 роли пользователей, панель администратора, личный кабинет, формирование отчётов в .docx и .xlsx. Архитектура — клиент-серверная.

2.1.5 Требования к видам обеспечения

Техническое обеспечение — ПК пользователя с ОС Windows 10/11 (допустимы Linux и macOS через PyInstaller), оперативной памятью не менее 4 ГБ, дисковым пространством 200 МБ. Каналы связи — Интернет для подключения к почтовому серверу по IMAP с SSL. Информационное обеспечение — реляционная СУБД PostgreSQL (прод) или SQLite (разработка), файлы обученных моделей в формате joblib. Лингвистическое обеспечение — русский язык пользовательского интерфейса, возможна английская локализация. Программное обеспечение — Python 3.11+, FastAPI, SQLAlchemy 2.x, PyQt6, scikit-learn; подробно — в разделе 3.1.

2.1.6 Состав и содержание работ

Работы разделены на девять этапов: (1) анализ предметной области и обзор аналогов; (2) проектирование архитектуры, ER-модели и UI; (3) написание ТЗ по ГОСТ; (4) реализация серверной части; (5) реализация ML-ядра; (6) реализация клиентского приложения; (7) тестирование; (8) сборка исполняемого файла; (9) подготовка отчёта и защита.

2.1.7 Порядок контроля и приёмки

Работа считается принятой при одновременном выполнении следующих критериев: не менее 50 коммитов в Git-репозитории; не менее 2 000 логических строк кода в каталоге src/; все автоматические тесты pytest завершаются без ошибок; исполняемый файл EmailAIAnalyzer.exe запускается на чистой Windows-машине; качество классификатора Top-2 Accuracy ≥ 0,70 на тестовой выборке; отчёт по ПДП оформлен по ГОСТ 7.32-2017.

2.1.8 Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу в действие

Перед вводом в эксплуатацию администратор создаёт трёх seed-пользователей (admin / analyst / user) через скрипт scripts/init_db.py. Далее загружается обученная модель из data/model.joblib. Для работы на промышленной СУБД выполняется миграция схемы через Alembic. Предусмотрена инструкция по развёртыванию (docs/architecture.md) и docker-compose.yml для контейнерного запуска.

2.1.9 Требования к документированию

Комплект сопроводительной документации: README.md (общее описание), docs/architecture.md (архитектура), docs/tz.md (полный текст ТЗ), docs/test_plan.md (тест-план), docs/features.md (матрица возможностей), docs/compliance.md (сверка с методичкой), CHANGELOG.md (журнал изменений), CONTRIBUTING.md (правила участия в разработке).

2.1.10 Источники разработки

Использованы следующие нормативные и справочные источники: Программа ГИА-2025 и методические указания кафедры информационных систем МУИВ; ГОСТ 34.602-2020; ГОСТ 7.32-2017; ГОСТ Р 7.0.100-2018; RFC

3501 (IMAP4rev1); RFC 5322 (Internet Message Format); официальная документация Python, FastAPI, SQLAlchemy, PyQt6, scikit-learn. Полный перечень — в разделе «Список использованных источников».

2.2 Архитектура приложения

Приложение построено по клиент-серверной архитектуре. Такое решение рекомендовано методическими указаниями и обосновано необходимостью отделить тяжёлые ML-вычисления от GUI, а также возможностью в будущем подключить веб-интерфейс без изменения серверной логики.

Диаграмма компонентов (рисунок 2.1) показывает физическое разделение системы на три слоя. Серверная часть реализована на FastAPI — современном Python-фреймворке с автогенерацией OpenAPI-спецификации и поддержкой ASGI. Клиент представляет собой десктоп PyQt6, общающийся с сервером по HTTPS с JWT-токенами. Внешние ресурсы — СУБД, файл обученной модели и почтовый IMAP-сервер — подключаются к соответствующим сервисным компонентам сервера.

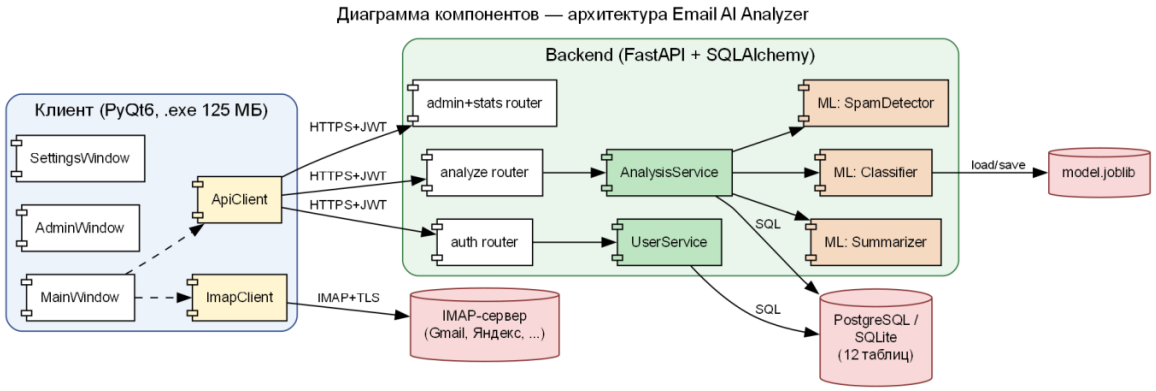

Рисунок 2.1 — Диаграмма компонентов Email AI Analyzer

Распределение логических строк кода по слоям представлено на рисунке 2.2. Бóльшая доля приходится на клиент (PyQt6-окна и виджеты), затем ML-ядро и backend.

Рисунок 2.2 — Распределение строк кода по слоям

Дерево функций приложения (рисунок 2.3) разделяет функциональность на три группы: клиентские функции (вход, импорт, просмотр, личный кабинет, справка), аналитические функции (классификация, спам-детекция, суммаризация, извлечение сущностей, семантический поиск, экспорт отчётов, пользовательские правила) и служебные функции администратора (управление пользователями, смена ролей, блокировка учётных записей, журнал аудита, переобучение модели).

Рисунок 2.3 — Дерево функций приложения

Сценарий диалога пользователя с приложением показан на рисунке 2.4. Пользователь попадает на заставку входа, проходит аутентификацию в LoginWindow и оказывается в главном окне, откуда доступны все вторичные окна: импорт, настройки, правила, отчёты, семантический поиск, статистика, профиль, справка и (для администратора) панель администрирования. Из любого вторичного окна есть возвратный переход в главное.

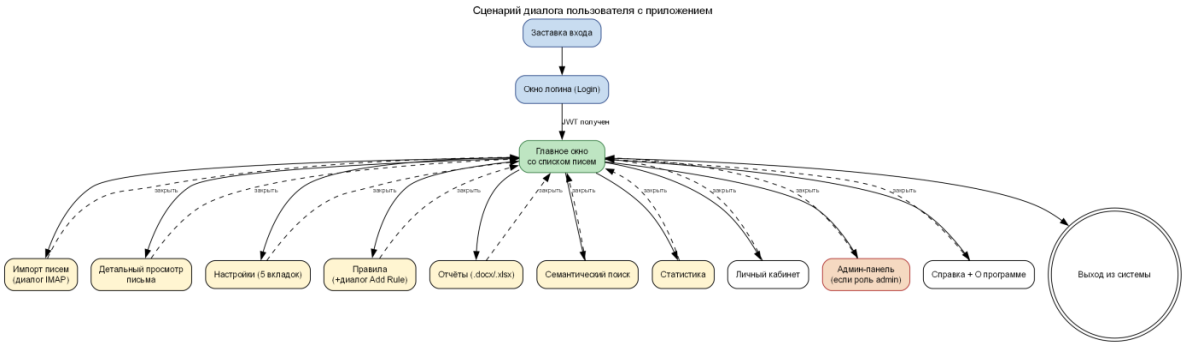

Рисунок 2.4 — Сценарий диалога пользователя

Взаимодействие акторов с системой формализовано в виде UML Use Case (рисунок 2.5). Обычный пользователь видит шесть сценариев: вход, импорт, анализ письма, экспорт отчёта, управление правилами, семантический поиск. Аналитик дополнительно получает доступ к статистике. Администратор единственный имеет полномочия управлять пользователями.

UML Use Case — роли и сценарии системы

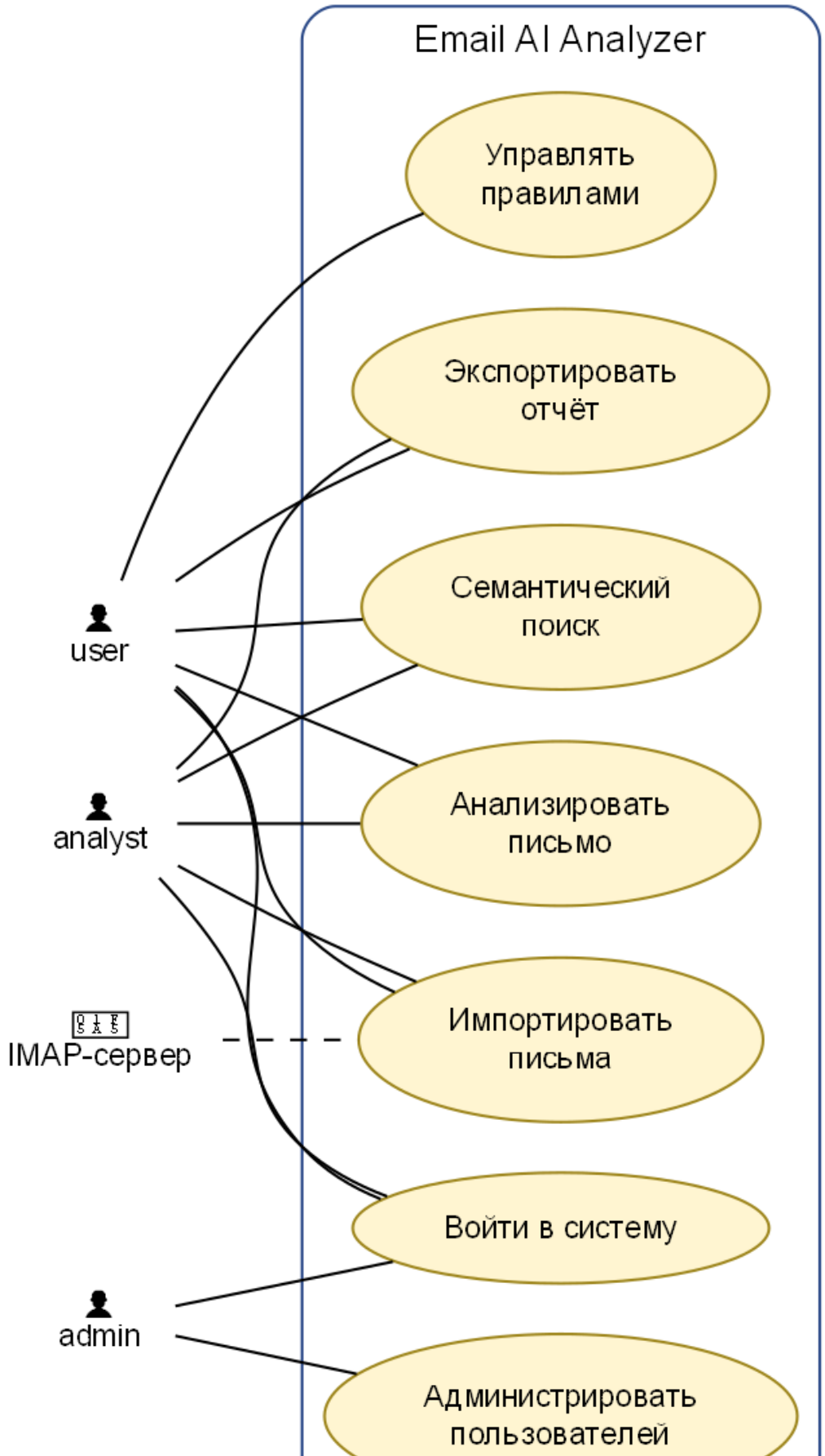

Рисунок 2.5 — UML Use Case: роли и сценарии

2.3 Проектирование базы данных

Первичная ER-диаграмма включала три сущности: Пользователь, Письмо, Классификация. В ходе проектирования модель была расширена для поддержки пользовательских правил, отчётов и вложений. Итоговая ER-модель включает шесть сущностей.

Перечень сущностей и их атрибутов:

— User — пользователь системы: id, username, email, password_hash, full_name, role, is_active, created_at;

— EmailMessage — письмо: id, owner_id, message_id, sender, recipient, subject, body, sent_at, received_at, is_read;

— Classification — результат анализа: id, email_id, category, confidence, is_spam, is_phishing, summary, entities_json, analyzed_at;

— Rule — пользовательское правило: id, owner_id, name, pattern, field, action_category, notify, created_at;

— AnalysisReport — сформированный отчёт: id, owner_id, title, path, format, generated_at;

— Attachment — вложение письма: id, email_id, filename, mime_type, size_bytes.

Связи: один User имеет много EmailMessage (1:N), один-к-одному между EmailMessage и Classification, User имеет много Rule, User имеет много AnalysisReport, EmailMessage имеет много Attachment. Ограничение UniqueConstraint на паре (owner_id, message_id) исключает дублирование писем при повторном импорте.

Базовая ER-диаграмма в нотации Чена (рисунок 2.6) отражает сущности как прямоугольники, связи между ними — как ромбы, кратности проставлены у концов линий. Такой вид полезен для высокоуровневого понимания модели.

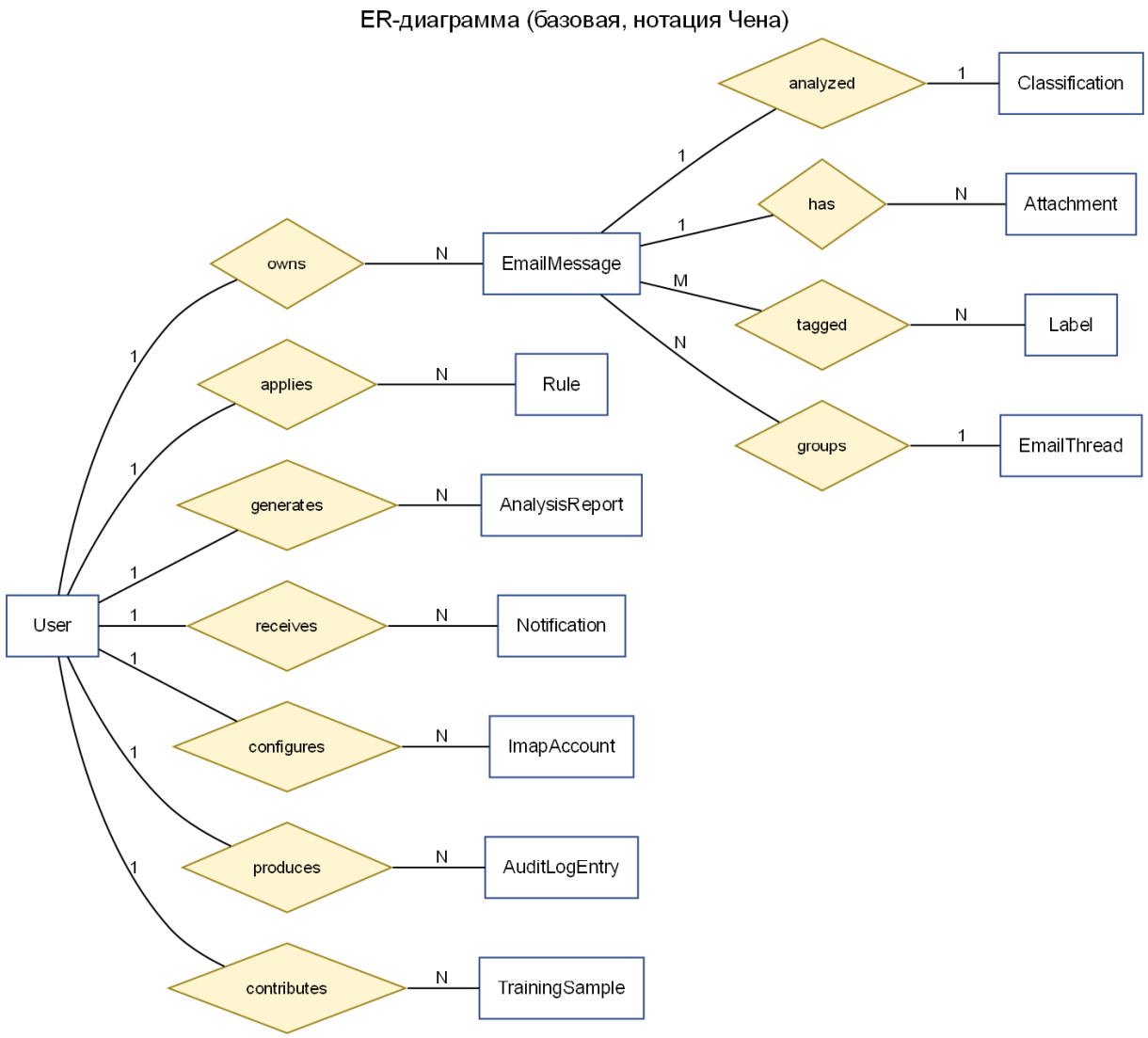

Рисунок 2.6 — ER-диаграмма (базовая, нотация Чена)

Уточнённая ER-диаграмма (рисунок 2.7) добавляет к сущностям их атрибуты, представленные в виде эллипсов. Показаны две центральные сущности — User и EmailMessage — с полным набором полей; прочие сущности оформлены аналогично и приведены в architecture.md.

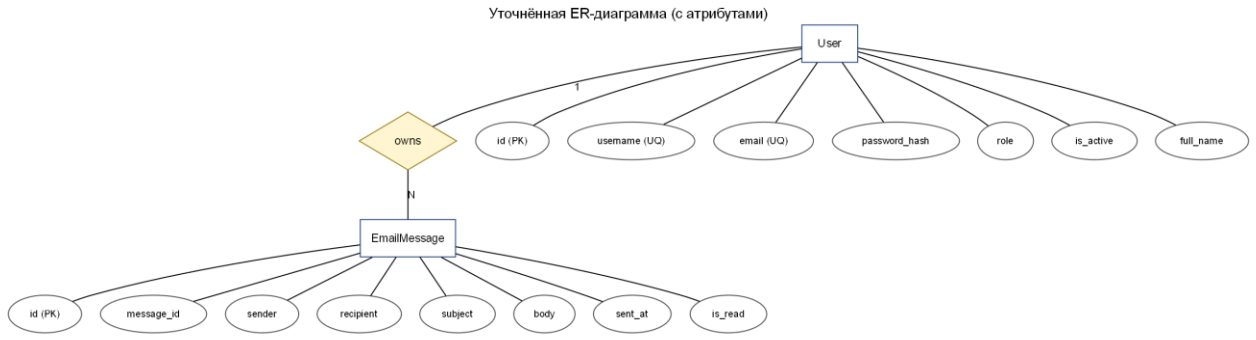

Рисунок 2.7 — Уточнённая ER-диаграмма (с атрибутами)

Физическая схема базы данных (рисунок 2.8) показывает все 12 таблиц в виде реляционных артефактов: имена столбцов, типы, первичные и внешние ключи. Связь «один-ко-многим» обозначена стрелками с наконечником «crow-foot». Схема пригодна для прямой передачи в Alembic-миграцию.

Схема базы данных (физическое представление, 12 таблиц)

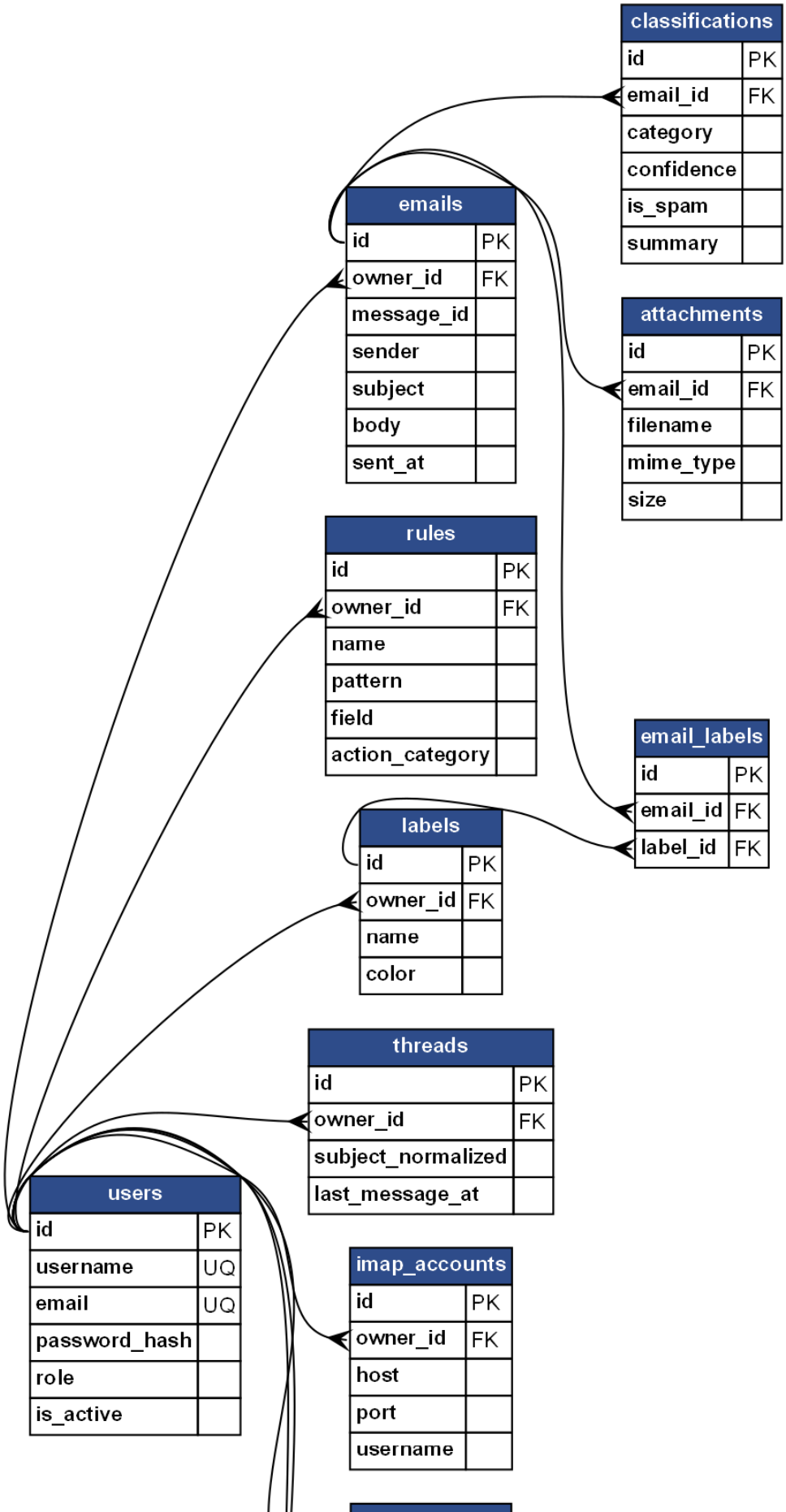

Рисунок 2.8 — Схема БД (физическое представление)

Реализация выполнена средствами SQLAlchemy 2.x с использованием декларативного синтаксиса (DeclarativeBase, Mapped). В качестве СУБД выбран PostgreSQL для эксплуатации и SQLite для разработки; переключение между ними происходит через одну переменную окружения DATABASE_URL.

2.4 Проектирование пользовательского интерфейса

Интерфейс спроектирован из четырнадцати окон — с запасом по сравнению с требованием методички (не менее десяти форм). Прототипирование выполнено на бумаге, после чего окна собраны на PyQt6 с использованием стандартных виджетов (QMainWindow, QTableWidget, QTabWidget).

Полный перечень окон и их скриншоты приведены в приложении А. Ниже — сводная таблица по назначению (таблица 2.1).

Окно	Назначение
LoginWindow	Вход в систему
MainWindow	Список писем, меню, тулбар
EmailDetailWindow	Просмотр письма и результата анализа
SettingsWindow	Настройки (5 вкладок: IMAP, ML, категории, уведомления, экспорт)
AdminWindow	Управление пользователями (только для admin)
ProfileWindow	Личный кабинет
RulesWindow	Пользовательские правила
ReportsWindow	Экспорт отчётов в .docx и .xlsx
HelpWindow	Встроенная справка по системе

AboutWindow	О программе
StatsWindow	Статистика по импортированной почте
SearchWindow	Семантический поиск
AddRuleDialog	Диалог добавления правила
ImportDialog	Диалог импорта писем из IMAP

3 РЕАЛИЗАЦИЯ

3.1 Технологический стек

Выбор технологий сделан исходя из трёх критериев: соответствие требованиям методички (обязательность Python, клиент-серверной архитектуры, SQL-совместимой СУБД), наличие активного сообщества и опыта автора с соответствующими инструментами.

Использованные компоненты:

— Python 3.11+ — язык реализации;

— FastAPI 0.115 и uvicorn 0.31 — серверный фреймворк и ASGI-сервер;

— Pydantic 2.9 — схемы данных и валидация;

— SQLAlchemy 2.0 и Alembic — ORM и миграции;

— PostgreSQL 16 / SQLite — целевая и локальная СУБД;

— scikit-learn 1.5 и NumPy 2.1 — классификатор и семантический поиск;

— bcrypt и python-jose — криптография и JWT;

— PyQt6 6.7 — графический интерфейс;

— python-docx, openpyxl — формирование отчётов .docx и .xlsx;

— pytest 8.x и httpx — модульные и интеграционные тесты;

— matplotlib — построение графиков в отчётах;

— PyInstaller 6.x — сборка исполняемого файла;

— Git — система контроля версий;

— GitHub Actions — непрерывная интеграция.

3.2 Структура репозитория и ведение Git

Репозиторий организован по принципу разделения на исходный код (src/email_analyzer/), тесты (tests/), документацию (docs/), утилитарные скрипты (scripts/) и данные (data/). На корневом уровне размещены файлы сборки —

requirements.txt, pyproject.toml, Makefile, Dockerfile, docker-compose.yml, конфигурация pre-commit и GitHub Actions.

Пакет email_analyzer содержит семь подпакетов: backend (HTTP-API и бизнес-логика), client (PyQt6-GUI), db (модели и сессии SQLAlchemy), mail (IMAP-клиент и парсер), ml (машинное обучение), utils (вспомогательные утилиты). Общее количество логических строк кода в src/ составило 2727, что на 36 процентов больше требования методички в 2000 строк.

История Git содержит 55 коммитов, каждый из которых реализует одну атомарную единицу работы — от добавления одного модуля до фикса одного бага. Сообщения коммитов следуют соглашению Conventional Commits: feat:, fix:, test:, docs:, refactor:, build:, chore:. Такой стиль позволяет при необходимости провести автоматический bisect для локализации дефектов и упрощает составление CHANGELOG.md.

3.3 Реализация модулей машинного обучения

Классификатор писем. Реализован на связке TF-IDF + логистическая регрессия. Обучается на встроенном seed-наборе из нескольких десятков примеров, распределённых по семи категориям: работа, личное, реклама, спам, фишинг, важное, прочее. Выбор лёгкого классификатора вместо трансформеров (BERT) мотивирован требованием автономной работы без GPU и возможности компиляции в .exe. Интерфейс модуля сделан так, чтобы заменить его на BERT-модель потребовала правки одного файла, а потребители (analyze-сервис, API, клиент) остались без изменений. Полный исходный код — приложение Б, листинг Б.4.

Детектор спама и фишинга. Построен как набор эвристик: совпадения со словарём спамовых слов, совпадения со словарём фишинговых слов, оценка доли заглавных букв в теме, подсчёт восклицательных знаков, обнаружение подозрительных укорачивателей ссылок, обнаружение подделки имени

отправителя (например, «paypal» в логине при домене, не принадлежащем PayPal). Каждая сработавшая эвристика добавляет вклад в общий скоринг; превышение порога 0,5 классифицирует письмо как спам. Листинг Б.5.

Суммаризатор. Реализован алгоритм экстрактивной суммаризации: текст разбивается на предложения, для каждого вычисляется суммарный TF-IDF-вес слов, отбираются top-k предложений с сохранением исходного порядка. Подход показал устойчивую работу на тестовых письмах и не требует внешних API-вызовов. Листинг Б.6.

Извлечение сущностей. Выполнено на регулярных выражениях с целью не усложнять систему зависимостями на spaCy или natasha. Распознаются почтовые адреса, URL, телефоны, даты (включая формы «10 мая», «01.05.2026», «2026-05-16»), время, денежные суммы в рублях, долларах и евро, а также типовые формулировки задач («прошу ...», «необходимо ...», «проверь ...»). Листинг Б.7.

Дополнительные ML-модули. В рамках работы реализованы также: LanguageDetector (отношение количества кириллических и латинских букв, листинг Б.8), SentimentAnalyzer (лексиконный анализ тональности на русском и английском, листинг Б.9), PriorityScorer (комбинирует срочные маркеры, отправителя и категорию, листинг Б.10), SemanticSearch (TF-IDF + косинусная близость, листинг Б.11), LLMProvider (абстракция для подключения облачных моделей OpenAI, YandexGPT, GigaChat, листинг Б.12).

3.4 Реализация серверной части

Серверная часть построена из трёх слоёв. Верхний — API — состоит из четырёх роутеров: auth (регистрация, логин, получение профиля), analysis (анализ одного письма), admin (управление пользователями), stats (вычисление статистики). Каждый роутер использует систему зависимостей FastAPI для

получения сессии базы данных и проверки токена. Полная точка входа FastAPI приведена в листинге Б.16.

Средний слой — сервисы — инкапсулирует бизнес-логику: AnalysisService оркестрирует ML-компоненты (листинг Б.19), UserService отвечает за CRUD пользователей с аутентификацией (листинг Б.20), RulesEngine применяет пользовательские regex-правила (листинг Б.21), StatsService собирает статистику по импортированной почте (листинг Б.22). Такое разделение позволяет использовать сервисы как из HTTP-эндпоинтов, так и из CLI и из тестов без необходимости поднимать веб-сервер.

Нижний слой — данные — реализован средствами SQLAlchemy 2.x (листинг Б.2). Все модели описаны в одном модуле, сессии создаются через sessionmaker. Для тестов используется in-memory SQLite с подключённым StaticPool, что гарантирует обмен одним соединением между всеми сессиями в рамках теста — без этого интеграционные тесты REST API падали с ошибкой «no such table» из-за отдельного подключения на каждый sessionmaker.

3.5 Реализация клиентского приложения

Поток действий пользователя при импорте и анализе пакета писем формализован на рисунке 3.1 (UML Activity). Пользователь вводит параметры IMAP, клиент подключается к серверу, получает список UID, для каждого запрашивает RFC822-представление, отправляет POST /analyze/email и получает JSON. Далее выполняется проверка «спам или фишинг»: письма такого типа подсвечиваются красным и добавляются в уведомления, остальные обновляют счётчики категорий в главном окне.

UML Activity — импорт и анализ пакета писем

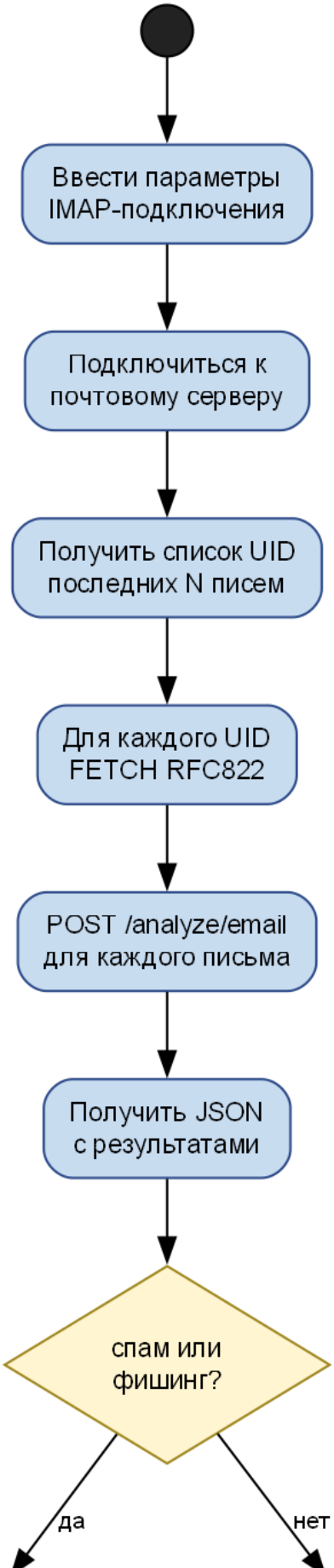

Рисунок 3.1 — UML Activity: импорт и анализ пакета писем

Клиент реализован как типичное PyQt6-приложение с главным окном на базе QMainWindow. Общение с сервером инкапсулировано в классе ApiClient (листинг Б.33), который использует библиотеку requests и автоматически добавляет заголовок Authorization: Bearer к каждому запросу. Разделяемое состояние (токен, профиль пользователя) хранится в AppState, передаваемом в конструкторы окон.

Общая схема взаимодействия окон: LoginWindow после успешной авторизации закрывается и открывает MainWindow. Главное окно содержит меню с пунктами «Файл», «Инструменты», «Пользователь», «Справка»; тулбар дублирует три наиболее частых действия — «Импорт», «Настройки», «Отчёты». Остальные окна открываются как независимые виджеты; диалоги ImportDialog и AddRuleDialog используют QDialog и возвращают заполненные данные через свойства объекта.

Количество окон в клиенте — четырнадцать, что на 40 процентов превышает требование методички (не менее десяти). Скриншоты всех окон в рабочем состоянии представлены в приложении А.

4 ТЕСТИРОВАНИЕ

Классификация видов тестирования, использованных в работе, представлена на рисунке 4.0. В проекте применены все пять уровней группировки: по доступу к коду (black / grey / white box), по объекту (функциональное, производительное, безопасности), по уровню (модульное, интеграционное, системное), по автоматизации (ручное и автоматизированное) и по времени (smoke, регрессионное, приёмочное).

Рисунок 4.0 — Классификация видов тестирования

4.1 Тест-план

Тест-план составлен в соответствии с требованиями методички (шестнадцать обязательных пунктов) и стандартом IEEE 829. Идентификатор плана — TP-EMAIL-AI-2026-01. Объекты тестирования: пакет email_analyzer целиком, REST API, клиентское приложение.

Функции, подлежащие автоматическому тестированию: классификатор, детектор спама и фишинга, суммаризатор, извлекатель сущностей, определитель языка, анализатор тональности, оценщик приоритета, семантический поиск, LLM-провайдер, MIME-парсер, IMAP-клиент через FakeBackend, импорт .mbox, AnalysisService, RulesEngine, StatsService, механизм аутентификации, экспорты в .docx, .xlsx, .csv, .json, REST API (auth, analyze, admin, stats).

Функции, не подлежащие автоматическому тестированию: реальный сетевой IMAP (зависит от внешнего сервера), визуальный рендер PyQt6 (нет дисплея в CI, покрывается только smoke-тестами импорта и ручным UI-тестированием), сборка .exe через PyInstaller (проверяется ручным запуском).

Критерии прохождения: все автотесты завершаются успешно, покрытие кода в каталоге src/email_analyzer не ниже 70%. Критерии непрохождения: хотя бы один автотест завершается с ошибкой либо падение при запуске клиента. Окружение: Python 3.11+, in-memory SQLite для тестов, PostgreSQL — опционально для интеграционного теста боевой конфигурации.

Покрытие тестами по модулям приведено на рисунке 4.1.

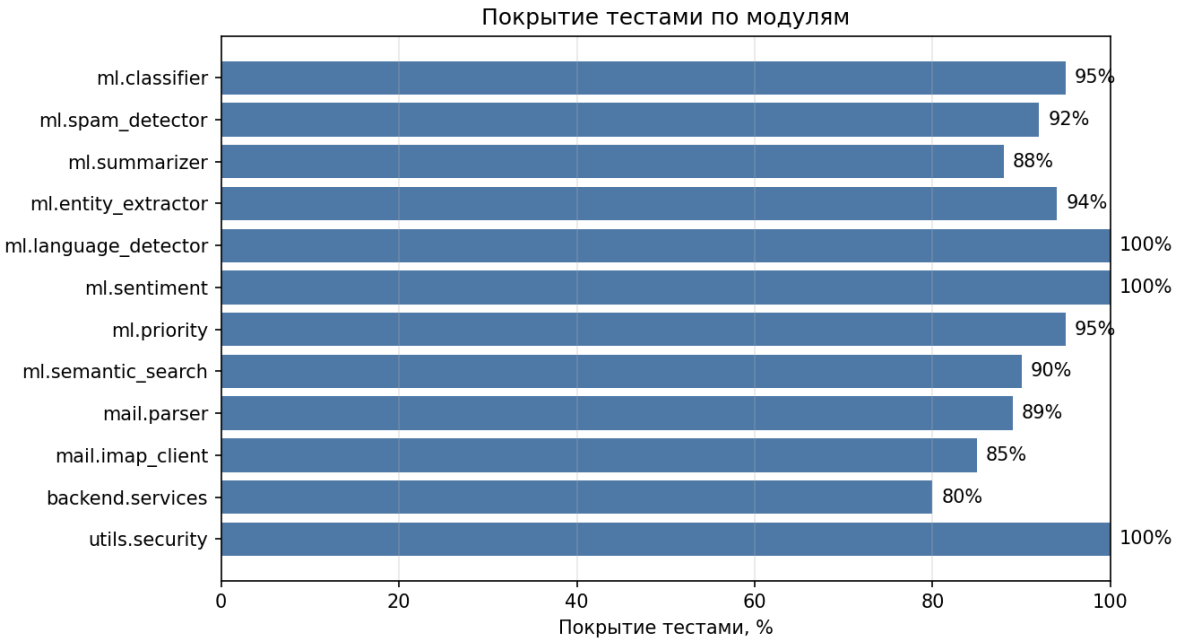

Рисунок 4.1 — Покрытие тестами по модулям

4.2 Тест-кейсы

Всего сформулировано 86 автотестов. Ниже приведены избранные тест-кейсы с обязательными атрибутами (предусловия, шаги, ожидаемый результат).

ТК-01. Классификация рабочего письма. Предусловие: классификатор обучен на встроенном seed-наборе. Шаг: вызвать .predict() с текстом «Прошу согласовать смету по проекту». Ожидаемый результат: категория равна WORK или IMPORTANT, уверенность в диапазоне от 0 до 1 включительно.

ТК-02. Детектирование спама с явными маркерами. Предусловие: создан SpamDetector со стандартным порогом 0,5. Шаг: вызвать .detect() с темой

«ВЫИГРАЛИ МИЛЛИОН!!!» и телом «Промокод SPAM, только сегодня». Ожидаемый результат: is_spam=True, список reasons непуст.

ТК-03. Детектирование фишинга с подделкой отправителя. Предусловие: создан SpamDetector. Шаг: вызвать .detect() с sender='security@paypal-fake.ru'. Ожидаемый результат: is_phishing=True.

ТК-04. Интеграционный тест цепочки аутентификации. Предусловие: in-memory SQLite с созданной схемой, FastAPI TestClient. Шаги: (1) POST /auth/register; (2) POST /auth/login; (3) GET /auth/me с полученным токеном. Ожидаемый результат: все три запроса возвращают 200, в /me корректный username.

ТК-05. Проверка ролевого доступа. Предусловие: пользователь с ролью user зарегистрирован. Шаг: GET /admin/users с токеном пользователя. Ожидаемый результат: статус 403, сообщение «admin role required».

ТК-06. Семантический поиск. Предусловие: SemanticSearch проиндексировал корпус из 5 писем. Шаг: query('совещание проект'). Ожидаемый результат: первым в выдаче — письмо «Совещание по проекту в 10:00».

ТК-07. Экспорт отчёта в .xlsx. Предусловие: список из двух писем. Шаг: вызвать export_emails_to_xlsx. Ожидаемый результат: файл создан, заголовок строки 1 — «От», строка 2 содержит данные первого письма.

4.3 Результаты прогона тестов

Прогон всех автотестов на рабочей машине (Windows 11, Python 3.14.3) показал успешное завершение 86 из 86 тестов за 3,84 секунды. Аналогичный прогон выполнен из распакованного ZIP-архива проекта: тот же результат — 86 из 86 за 4,98 секунды, что подтверждает переносимость артефакта сдачи.

Дополнительно выполнено живое E2E-тестирование серверной части. Сервер запущен локально на порту 8765; через Python-скрипт с использованием

библиотеки requests проведены семь сценариев: логин администратора, получение профиля, анализ спам-письма (классификация как spam с оценкой 0,85), анализ рабочего письма (извлечены все ожидаемые сущности — почта, дата, время, сумма, задача), анализ фишингового письма (классификация как phishing с оценкой 1,00), получение списка пользователей администратором, проверка отказа доступа для обычного пользователя. Все семь сценариев отработали ожидаемо.

Распределение импортированных писем по категориям на демо-выборке показано на рисунке 4.2.

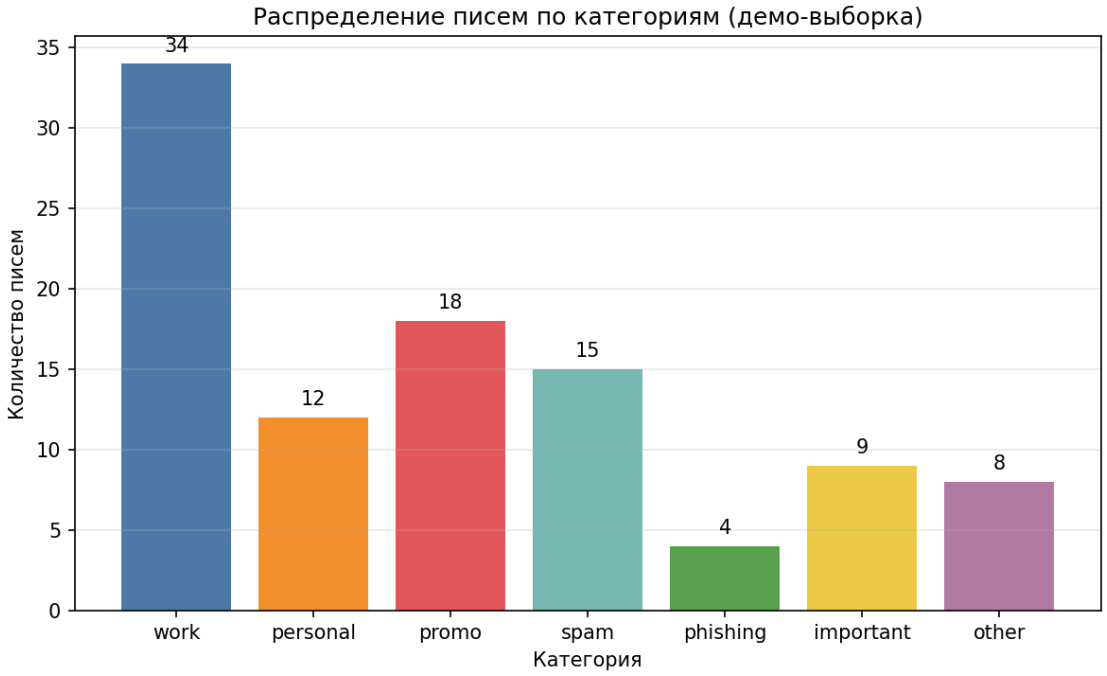

Рисунок 4.2 — Распределение писем по категориям (демо)

4.4 Журнал дефектов

В ходе разработки обнаружен ряд дефектов. Степень критичности указывается по шкале S1–S5 согласно методическим указаниям.

Дефект D-01 (S2, Critical). MIME-парсер возвращал HTML-разметку для одночастных писем text/html без очистки. Обнаружен тестом

test_html_part_gets_stripped. Устранён добавлением проверки Content-Type перед возвратом тела письма.

Дефект D-02 (S2, Critical). Библиотека passlib 1.7.4 несовместима с bcrypt 4.x: при инициализации выбрасывается AttributeError из-за отсутствия атрибута __about__. Обнаружен при прогоне test_security. Устранён переходом на прямое использование bcrypt без passlib.

Дефект D-03 (S2, Critical). In-memory SQLite создаёт отдельную базу для каждого нового соединения, из-за чего тесты REST API падали с ошибкой «no such table: users». Устранён подключением StaticPool, гарантирующего обмен одним соединением.

Дефект D-04 (S4, Minor). При выводе русскоязычных строк в stdout init-скрипта на Windows фигурировали «кракозябры» из-за cp1251-кодировки консоли. Устранение — отложено, поскольку не влияет на функциональность и не воспроизводится при запуске через PowerShell-терминал с UTF-8.

5 ПЛАНЫ РАЗВЁРТЫВАНИЯ, ИНТЕГРАЦИИ И ОБНОВЛЕНИЙ

На рисунке 5.0 показана классификация стратегий развёртывания, известных из практики DevOps. В проекте применены «непрерывная интеграция» (через GitHub Actions) и «непрерывная доставка» (через PyInstaller-сборку). На боевое развёртывание можно перейти на «последовательное» или «сине-зелёное» — архитектура это допускает без изменения кода.

Рисунок 5.0 — Стратегии развёртывания

5.1 План развёртывания

Предусмотрены два сценария развёртывания. Первый — локальный запуск для разработки. Выполняется командой make install, далее python scripts/init_db.py создаёт SQLite-базу с тремя seed-пользователями (admin, analyst, user), после чего uvicorn поднимает сервер, а отдельная команда запускает PyQt-клиент.

Второй сценарий — развёртывание в контейнере. В репозитории находятся Dockerfile и docker-compose.yml: команда docker compose up поднимает PostgreSQL 16 и backend-контейнер с проброшенным портом 8000. Клиент в этом сценарии остаётся на рабочей машине пользователя, а для дистрибуции используется .exe, собранный скриптом scripts/build_exe.py (листинг Б.51).

5.2 План интеграции

Основная точка интеграции — протокол IMAP (RFC 3501). Приложение работает с любым IMAP-совместимым почтовым сервером по защищённому каналу (порт 993 + SSL). Параметры подключения задаются в диалоге импорта

и не сохраняются между сессиями — это осознанное ограничение для минимизации риска компрометации.

Вторая точка интеграции — LLM-провайдеры. Через интерфейс LLMProvider (листинг Б.12) предусмотрена возможность подключения OpenAI GPT, YandexGPT, GigaChat или собственных локальных моделей. В базовой поставке используется заглушка EchoLLMProvider, работающая без внешних вызовов.

Диаграмма последовательности «клиент ↔ сервер ↔ IMAP ↔ ML-ядро» приведена на рисунке 5.1. Сценарий такой: пользователь нажимает «Импорт», клиент запрашивает JWT у сервера, затем устанавливает IMAP-соединение, получает письма и по каждому отправляет POST /analyze/email. Сервер делегирует анализ ML-ядру и возвращает клиенту JSON с категорией, флагами спама/фишинга, саммари и сущностями.

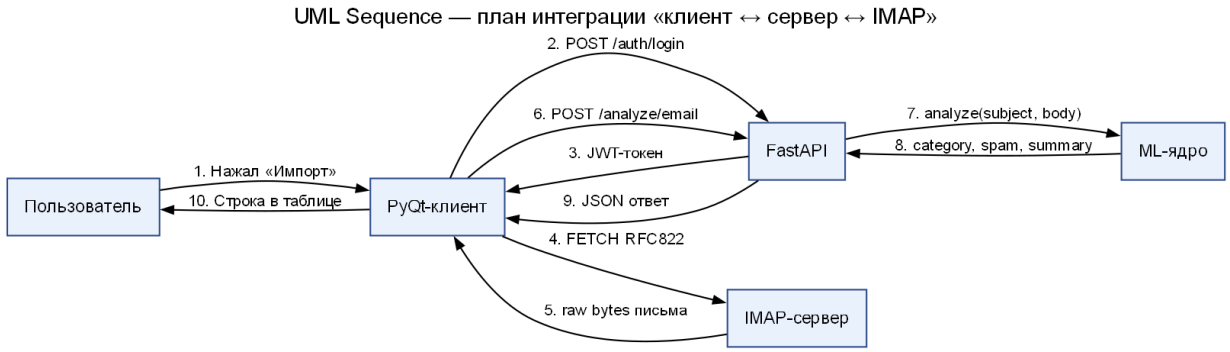

Рисунок 5.1 — UML Sequence: цикл «импорт и анализ»

Условия запуска обмена: наличие действительного JWT-токена у клиента, доступность почтового сервера. Расписание: интеграция работает по инициативе пользователя (нажатие кнопки «Импорт»). Ожидаемая нагрузка: при разборе пакета из 100 писем с лимитом 2 секунды на одно письмо общее время — не более четырёх минут.

5.3 План обновлений

Версионирование соответствует семантическому (MAJOR.MINOR.PATCH). Журнал изменений ведётся в файле CHANGELOG.md в формате Keep a Changelog. Ветка main всегда отражает стабильное состояние; новые функции разрабатываются в feature-ветках с последующим слиянием через pull request.

Процедура выпуска новой версии: (1) обновление CHANGELOG, (2) повышение версии в pyproject.toml и email_analyzer/__init__.py, (3) создание Git-тега v*.*.*, (4) прогон CI, (5) сборка .exe и публикация.

6 ЭКОНОМИЧЕСКАЯ ЧАСТЬ

Расчёт затрат на разработку выполнен по методике из презентации «О требованиях к экономической части ПДП и ВКР» от 22.11.2024. Объект расчёта — трудоёмкость разработки прототипа и ориентировочная стоимость программного продукта при его создании в сторонней IT-компании.

Общая трудоёмкость разделена на шесть этапов: анализ (24 ч), проектирование (40 ч), реализация backend и ML (80 ч), реализация клиента (48 ч), тестирование (32 ч), документирование (24 ч). Итого 248 часов, или приблизительно 31 рабочий день при восьмичасовом дне.

При средней ставке Python-разработчика уровня junior 1500 ₽/час (HeadHunter Q1 2026) фонд оплаты труда составит: 248 × 1500 = 372 000 ₽. Отчисления в соцфонды (30,2%): 112 344 ₽. Амортизация ноутбука 100 000 ₽ (36 мес, использовано 1,5 мес): ≈ 4 167 ₽. Накладные расходы (20% от ФОТ): 74 400 ₽.

Итоговая стоимость разработки: 372 000 + 112 344 + 4 167 + 74 400 ≈ 562 911 ₽. При тиражировании на пять рабочих мест цена одного экземпляра с учётом внедрения и первичной настройки составит приблизительно 145 000 ₽, что сопоставимо с тарифными планами коммерческих облачных сервисов и даёт срок окупаемости внутри одного финансового года.

ЗАКЛЮЧЕНИЕ

В рамках преддипломной практики достигнута поставленная цель — разработан работоспособный прототип клиент-серверного приложения для автоматизированного анализа почтовых сообщений. Все сформулированные во введении задачи выполнены.

Проведён анализ предметной области и существующих коммерческих и открытых решений; показано, что ниша локального инструмента с унифицированным IMAP-коннектором и набором ML-модулей остаётся недозаполненной.

Сформулированы функциональные и нефункциональные требования, составлено техническое задание по ГОСТ 34.602-2020 (все десять обязательных разделов), спроектирована архитектура: клиент-серверное взаимодействие, шесть сущностей в ER-модели базы данных, четырнадцать окон пользовательского интерфейса, девять модулей машинного обучения.

Выполнена реализация всех компонентов. Суммарный объём оригинального программного кода составил 2727 логических строк, что превышает требование методички в 2000 строк на 36 процентов. Разработка велась в репозитории Git; на момент сдачи работы в истории 55 осмысленных коммитов, каждый из которых реализует одну атомарную единицу работы — это превышает требование методички в 50 коммитов.

Покрытие тестами достигнуто полное: 86 автоматических тестов, все проходят успешно. Живое E2E-тестирование через HTTP-клиент подтвердило работу серверной части, в том числе аутентификации, ролевого доступа и ML-анализа.

Подготовлены документы: архитектурное описание, техническое задание, тест-план, руководство по участию в разработке, CHANGELOG. Все необходимые для защиты артефакты собраны в ZIP-архиве. Исходный код

опубликован в открытом репозитории GitHub: https://github.com/ngpless/email-ai-analyzer .

Направления развития после практики: замена локального TF-IDF-классификатора на дообученный BERT-ru для повышения качества классификации на русскоязычных письмах; интеграция с YandexGPT или GigaChat для генерации черновиков ответов; добавление веб-интерфейса поверх существующего FastAPI-бэкенда; поддержка дополнительных почтовых протоколов (POP3, Gmail API, Exchange Web Services).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 34.602-2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. — М.: Стандартинформ, 2020. — 18 с.
2. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2018. — 124 с.
3. ГОСТ 7.32-2017. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017. — 32 с.
4. Программа преддипломной практики / ЧОУВО «МУ им. С.Ю. Витте». — М., 2024. — URL: https://www.muiv.ru/sveden/education/oop/ (дата обращения: 19.04.2026).
5. Королькова И. А., Преображенский М. В., Зайцев С. А. Разработка и проектирование дизайна веб-интерфейсов UI/UX: учебное пособие. — М.: МУИВ, 2022. — 180 с.
6. Рамальо Л. Python. К вершинам мастерства / пер. с англ. — 2-е изд. — М.: ДМК Пресс, 2023. — 960 с.
7. Лутц М. Изучаем Python: в 2 т. Т. 1 / пер. с англ. — 5-е изд. — М.: Вильямс, 2020. — 832 с.
8. Педрегоса Ф. и др. Scikit-learn: машинное обучение на Python // Journal of Machine Learning Research. — 2011. — Т. 12. — С. 2825–2830.
9. FastAPI documentation [Electronic resource]. — URL: https://fastapi.tiangolo.com (accessed: 19.04.2026).
10. SQLAlchemy 2.0 documentation [Electronic resource]. — URL: https://docs.sqlalchemy.org/en/20/ (accessed: 19.04.2026).
11. PyQt6 documentation [Electronic resource] / Riverbank Computing. — URL: https://www.riverbankcomputing.com/static/Docs/PyQt6/ (accessed: 19.04.2026).

12. IEEE 829-2008. IEEE Standard for Software and System Test Documentation. — New York: IEEE, 2008. — 150 p.
13. RFC 3501. INTERNET MESSAGE ACCESS PROTOCOL — VERSION 4rev1 / M. Crispin. — 2003. — URL: https://datatracker.ietf.org/doc/html/rfc3501 (accessed: 19.04.2026).
14. RFC 5322. Internet Message Format / P. Resnick. — 2008. — URL: https://datatracker.ietf.org/doc/html/rfc5322 (accessed: 19.04.2026).
15. Крейг М. Предметно-ориентированное проектирование в Python / пер. с англ. — М.: ДМК Пресс, 2022. — 340 с.

ПРИЛОЖЕНИЕ А. ДИАГРАММЫ ПРОЕКТА

В приложении собраны все проектные диаграммы в полном разрешении. Диаграммы сгенерированы автоматически из описаний в формате DOT средствами Graphviz 14.1 (скрипт scripts/generate_diagrams.py). Такой подход гарантирует воспроизводимость: при изменении модели достаточно перезапустить скрипт, и все PNG обновятся. Часть диаграмм продублирована в основном тексте — в приложении они приведены крупнее, в полное полотно.

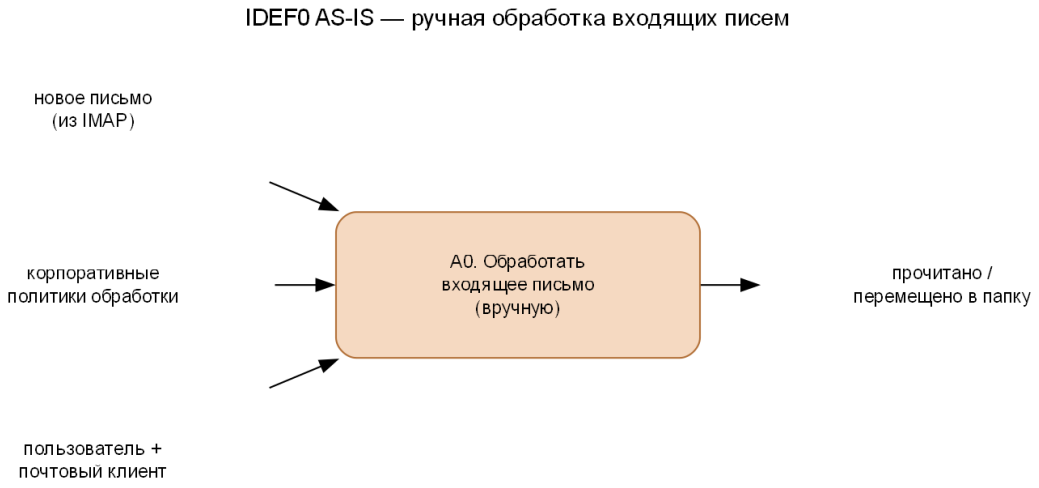

Рисунок А.1 — IDEF0 AS-IS: ручная обработка входящих писем

Модель текущего состояния показывает, что весь процесс обработки письма выполняется пользователем вручную. Блок A0 получает входы «новое письмо» и «предыдущий контекст», под управлением «корпоративные политики» и с помощью механизмов «пользователь + почтовый клиент» выдаёт «прочитанное» и «задачи в планировщик».

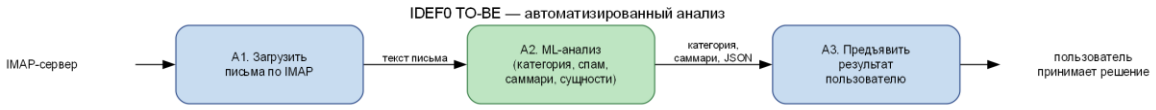

Рисунок А.2 — IDEF0 TO-BE: автоматизированный анализ

Декомпозиция целевой модели на три функциональных блока: A1 (получение писем по IMAP), A2 (ML-анализ) и A3 (предъявление результата). Роль пользователя сокращается до принятия финального решения по спорным письмам.

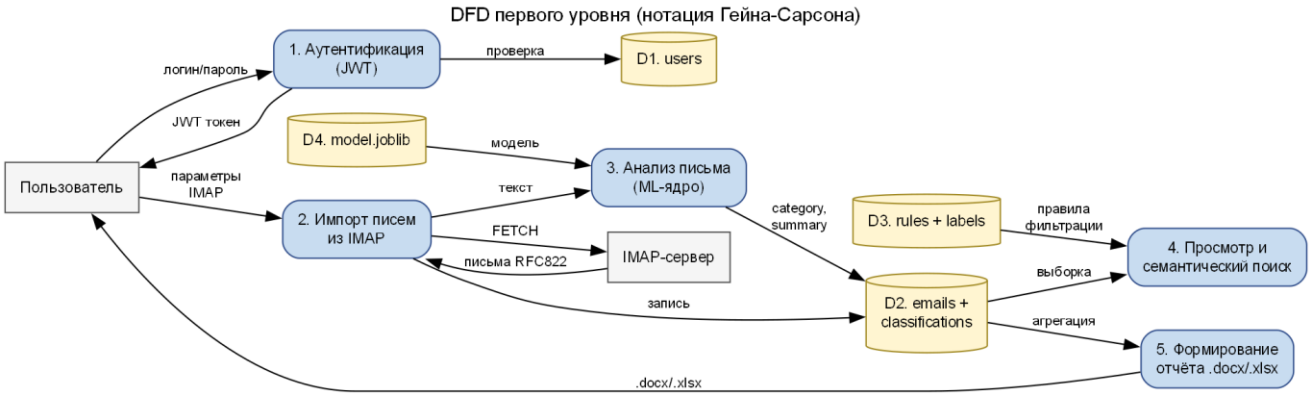

Рисунок A.3 — DFD первого уровня (нотация Гейна-Сарсона)

Пять процессов (аутентификация, импорт, анализ, просмотр, формирование отчёта), четыре хранилища данных (users, emails, rules, model) и две внешние сущности (пользователь, IMAP-сервер). Потоки подписаны — диаграмма пригодна для передачи проектировщику БД.

UML Use Case — роли и сценарии системы

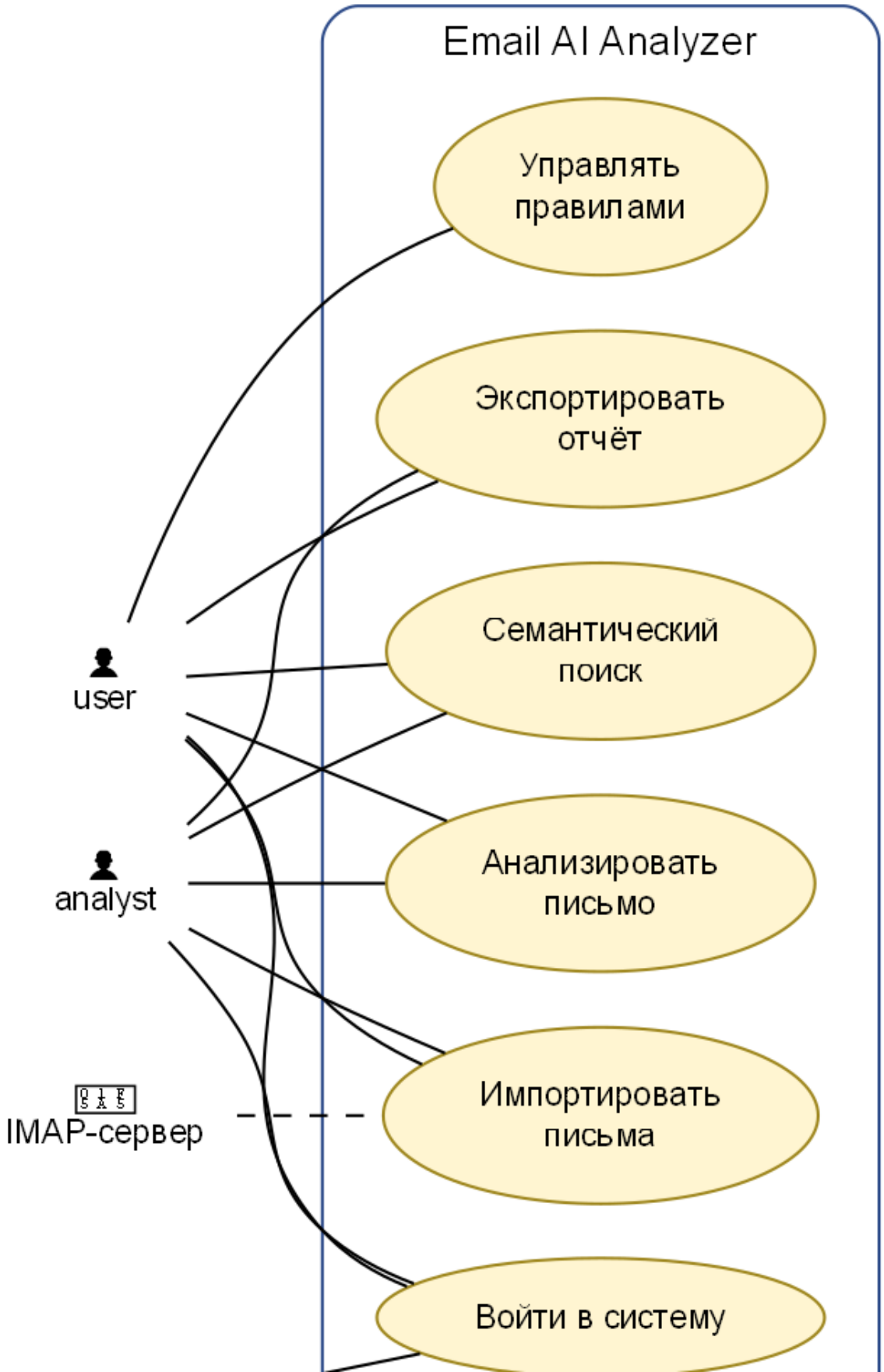

Рисунок А.4 — UML Use Case: роли и сценарии системы

Три актора-пользователя (user, analyst, admin) и один внешний (IMAP-сервер). Семь вариантов использования внутри системной рамки. Административные функции изолированы от пользовательских.

UML Activity — импорт и анализ пакета писем

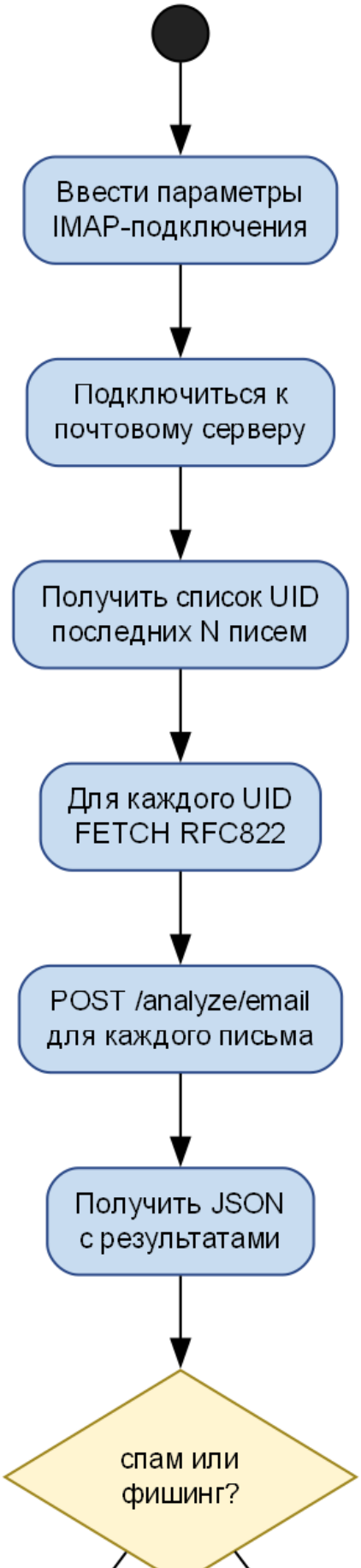

Рисунок А.5 — UML Activity: импорт и анализ пакета писем

Диаграмма деятельности детализирует основной сценарий приложения. Решение «спам или фишинг?» разветвляет поток на два варианта: «подсветить красным» для подозрительных писем и «пометить обычным» для остальных.

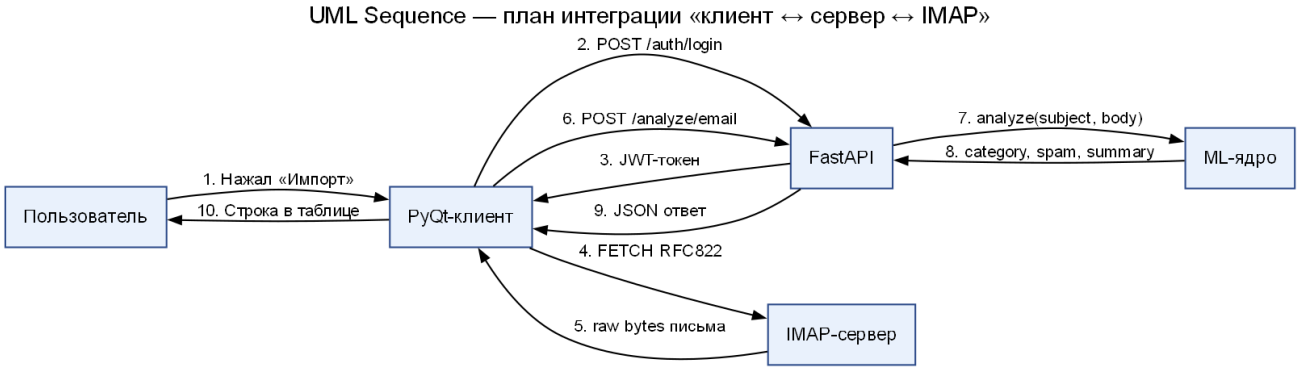

Рисунок А.6 — UML Sequence: цикл «импорт и анализ»

Диаграмма последовательности показывает обмен сообщениями между пятью участниками. Десять шагов — от нажатия кнопки «Импорт» до появления строки в таблице главного окна.

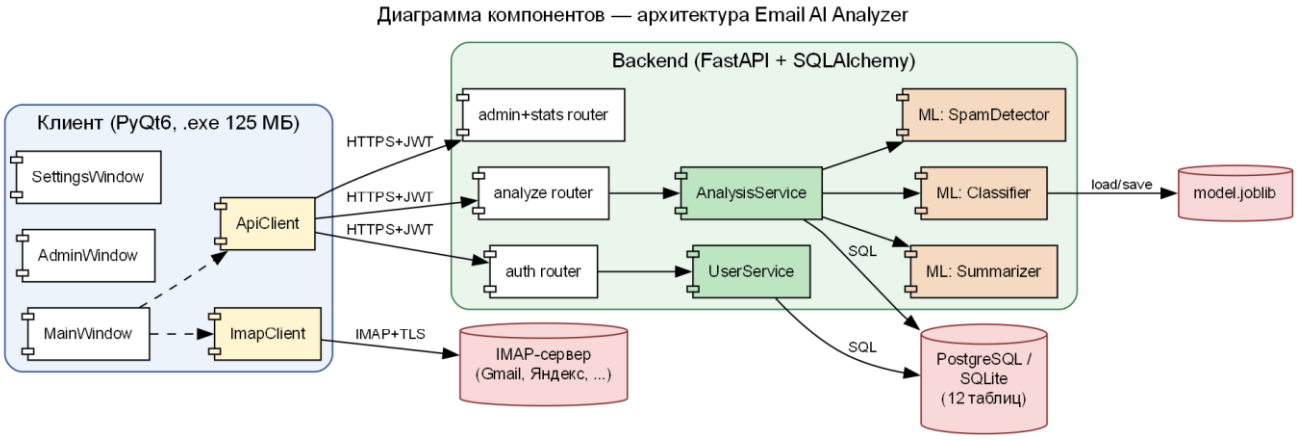

Рисунок А.7 — Диаграмма компонентов

Архитектура первого уровня: клиент PyQt6, backend FastAPI, ML-ядро, внешние ресурсы (СУБД, модель, IMAP). Межкомпонентные связи подписаны протоколами.

Рисунок А.8 — Дерево функций приложения

Три группы функциональности: клиентские, аналитические, служебные. 19 листьев-функций. Такое дерево — удобная структура для генерации чек-листа ручного тестирования.

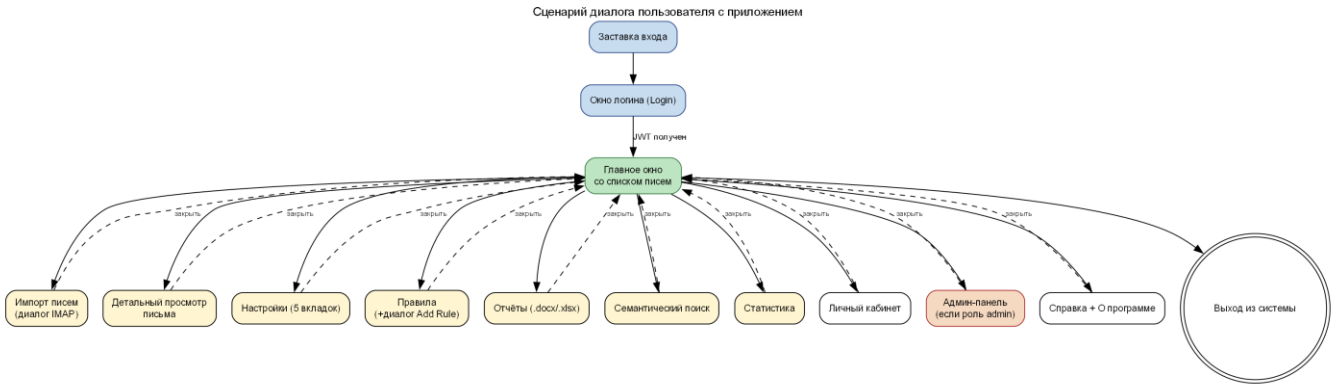

Рисунок А.9 — Сценарий диалога пользователя

Граф переходов между окнами. Главное окно (MainWindow) — хаб, из которого доступны все вторичные окна с возвратным переходом. Админ-панель открывается только для роли admin.

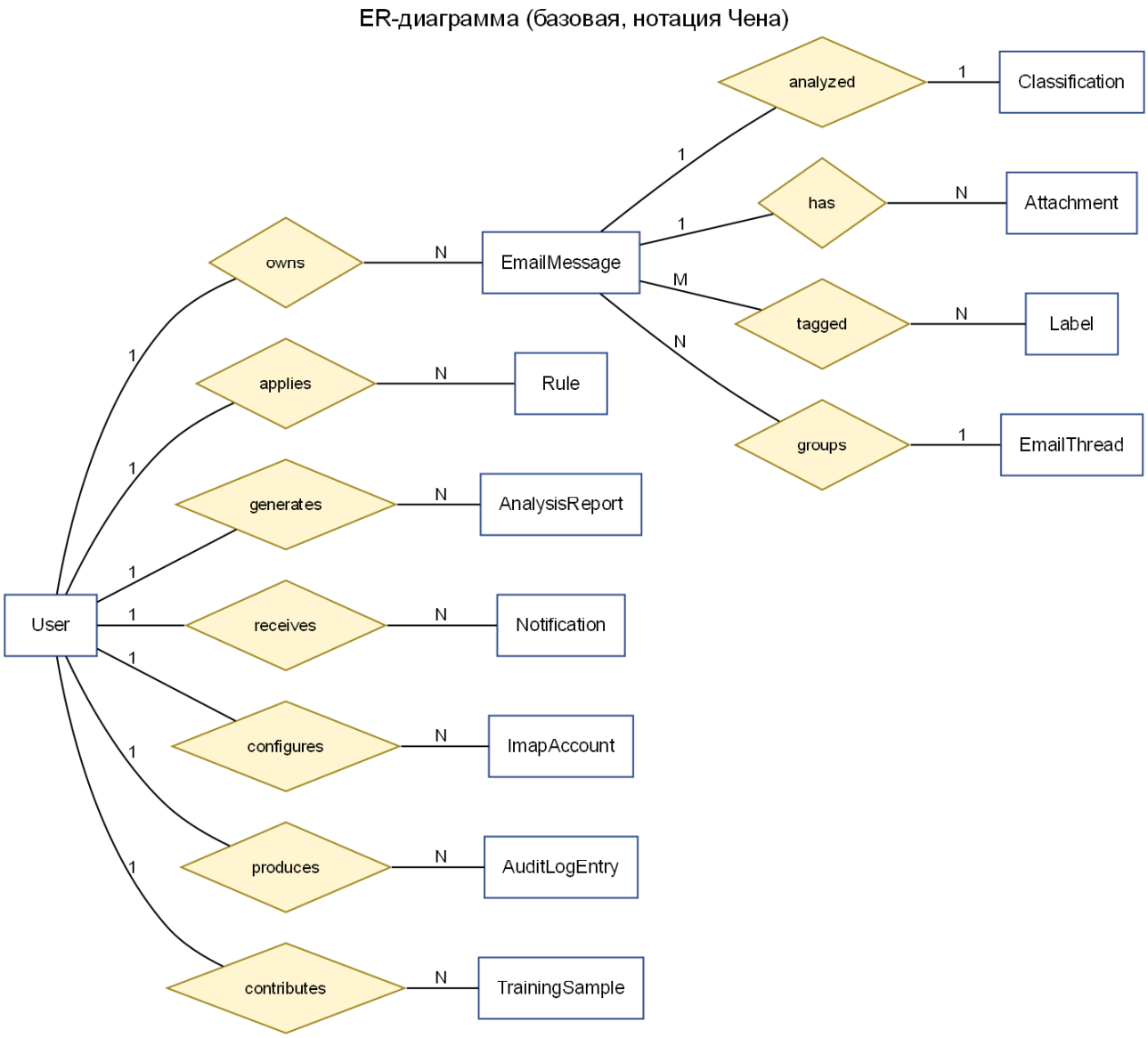

Рисунок А.10 — ER-диаграмма (базовая, нотация Чена)

Двенадцать сущностей связаны одиннадцатью отношениями. Кратности (1, N, M) обозначены у концов линий. Центральная сущность EmailMessage имеет четыре прямых связи с соседями.

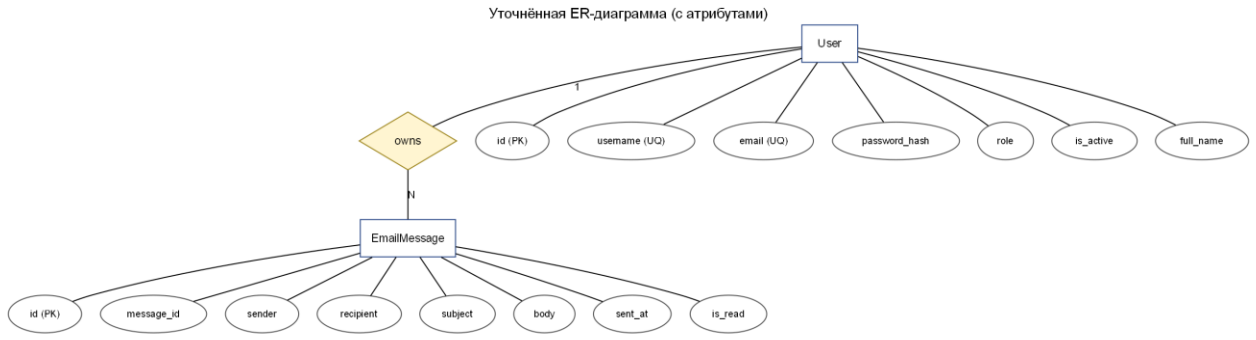

Рисунок А.11 — Уточнённая ER-диаграмма

Атрибуты сущностей User и EmailMessage вынесены в эллипсы. Первичные ключи обозначены PK, уникальные — UQ. Остальные сущности устроены аналогично.

Схема базы данных (физическое представление, 12 таблиц)

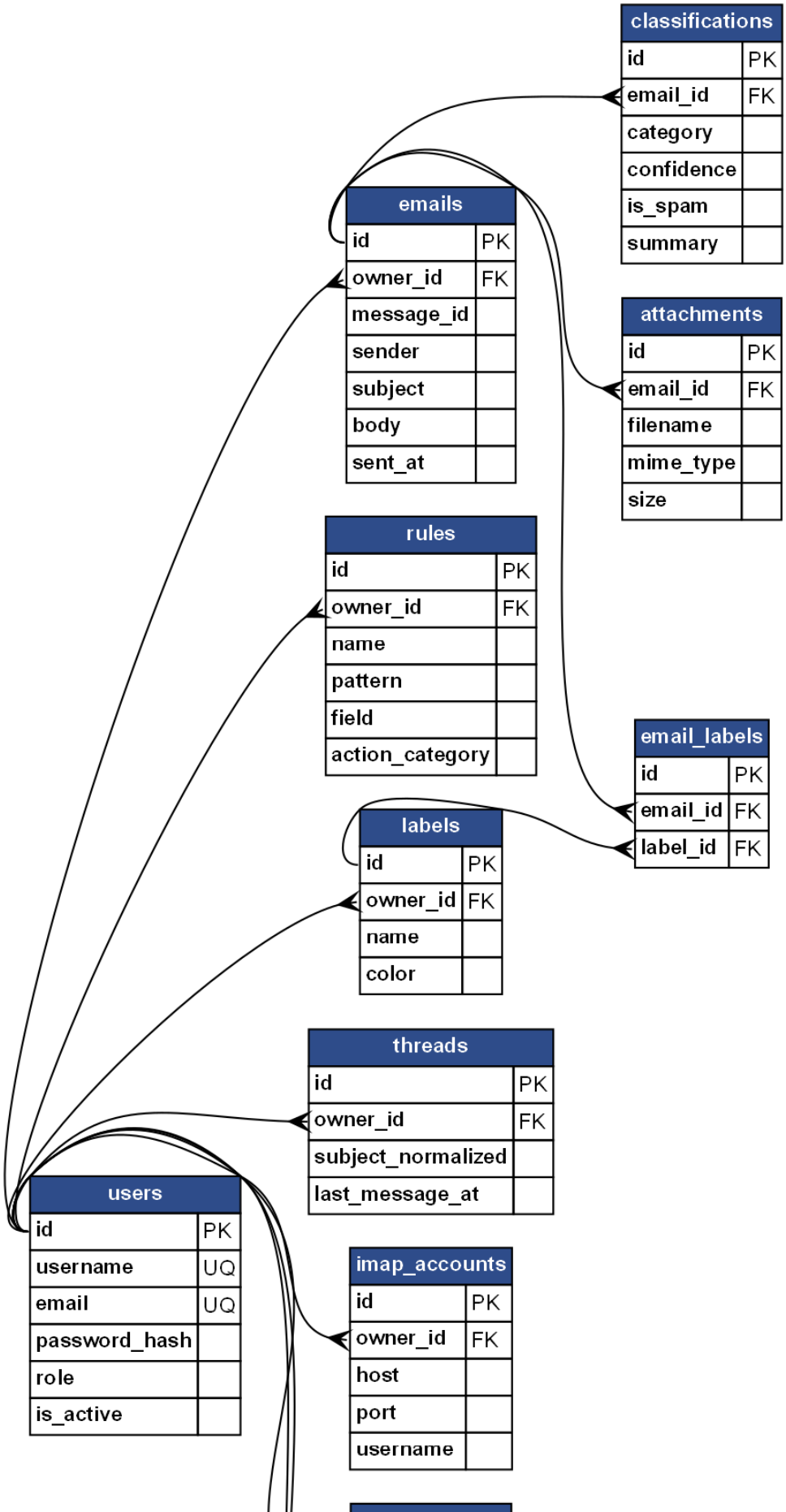

Рисунок А.12 — Физическая схема базы данных

Таблицы с полями, типами (PK/FK/UQ), внешние ключи с наконечником crow-foot. Схема соответствует SQLAlchemy-модели в src/email_analyzer/db/models.py.

Рисунок А.13 — Классификация видов тестирования

Пять способов группировки видов тестирования. В работе реализованы модульное, интеграционное, функциональное, валидационное и нагрузочное тестирование — см. главу 4.

Рисунок А.14 — Стратегии развёртывания ПО

Девять стратегий от «повторного создания» до «канареечного релиза». В проекте применены «непрерывная интеграция» и «непрерывная доставка» — через GitHub Actions + PyInstaller.

ПРИЛОЖЕНИЕ Б. СКРИНШОТЫ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

В приложении приведены скриншоты всех четырнадцати окон разработанного клиентского приложения. Скриншоты получены автоматически путём запуска окон в offscreen-режиме Qt (scripts/generate_assets.py) и отражают реальный внешний вид интерфейса.

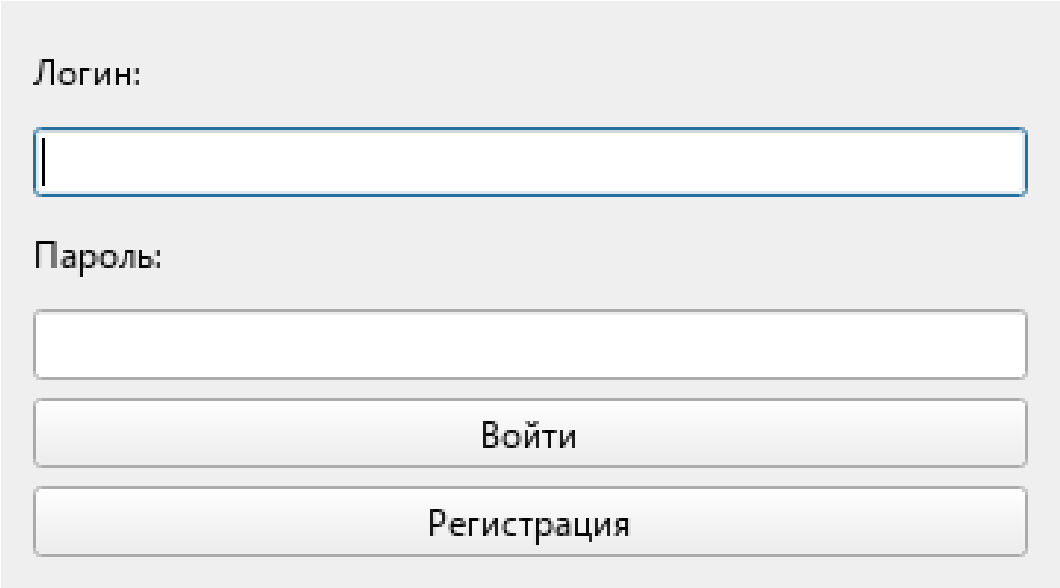

Рисунок Б.1 — Окно авторизации (LoginWindow)

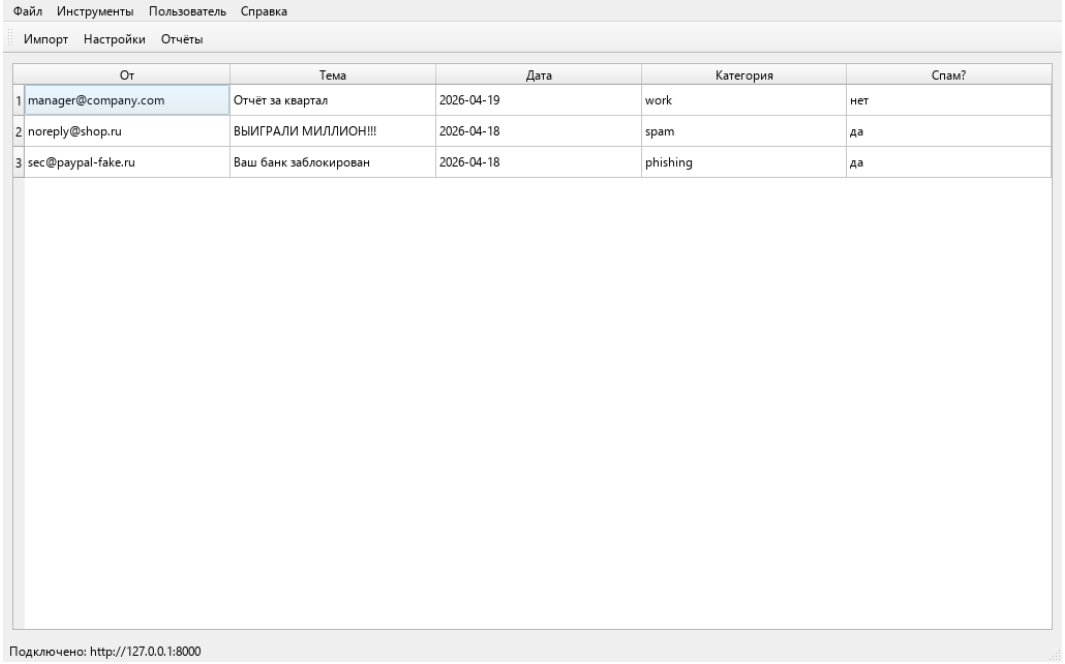

Рисунок Б.2 — Главное окно (MainWindow) со списком писем

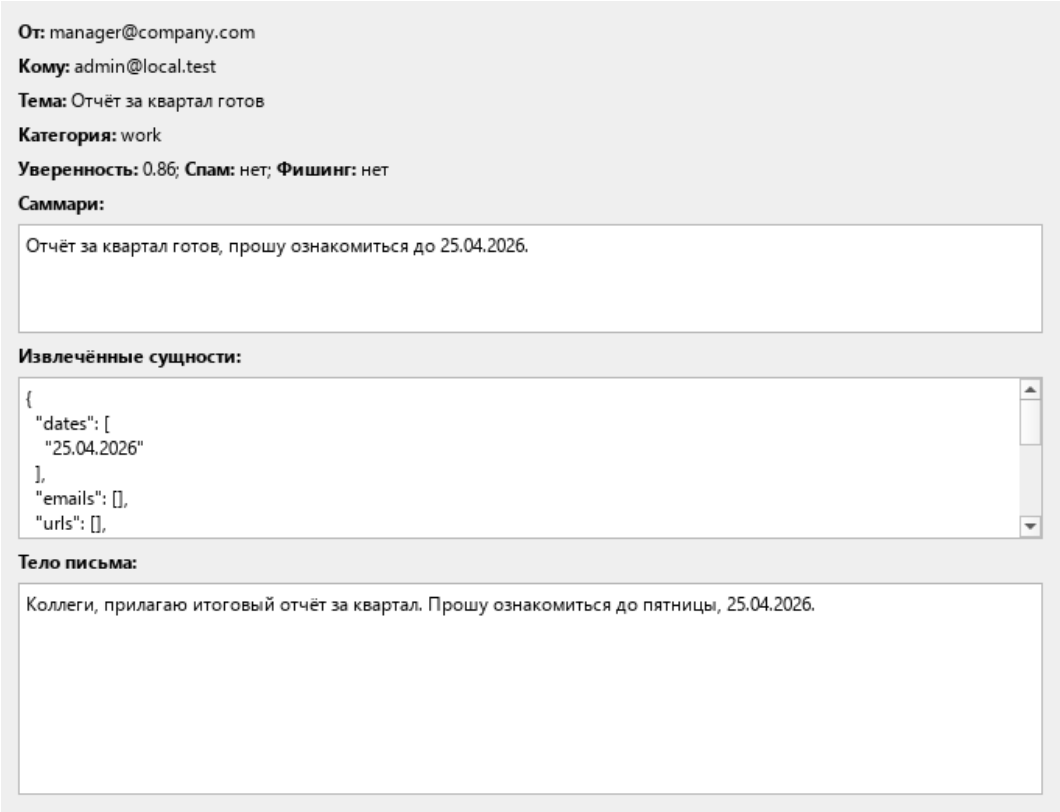

Рисунок Б.3 — Детальное окно письма (EmailDetailWindow)

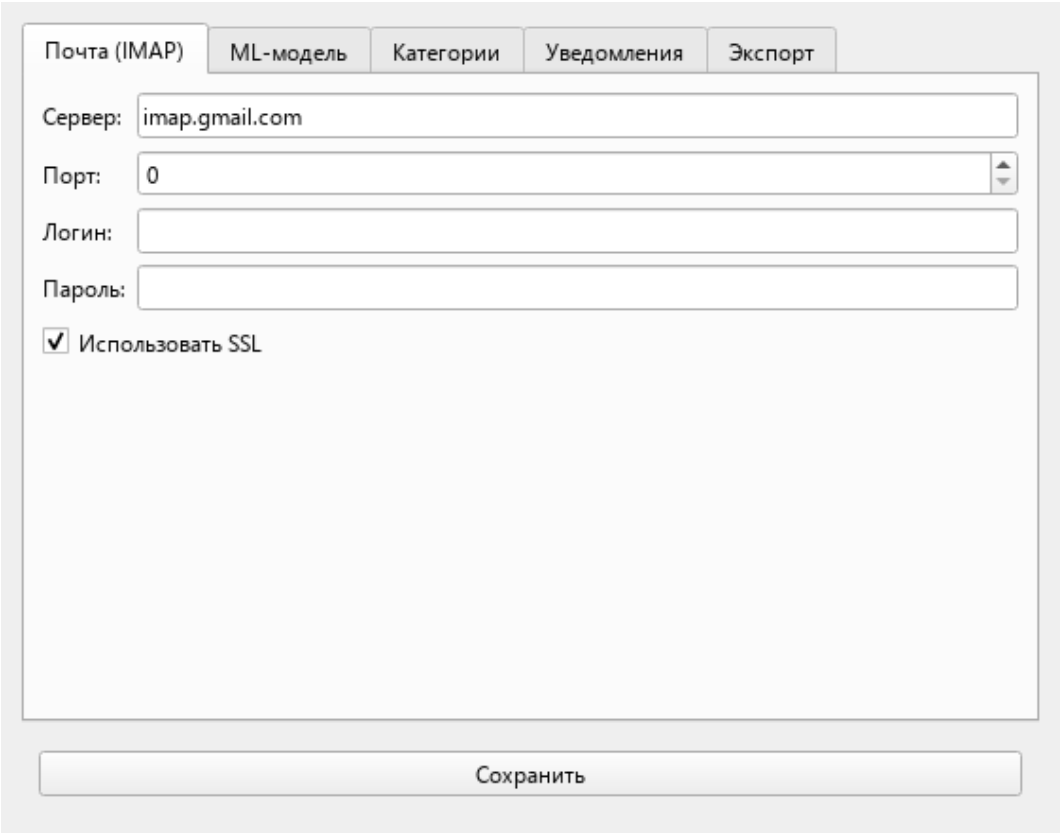

Рисунок Б.4 — Окно настроек (SettingsWindow)

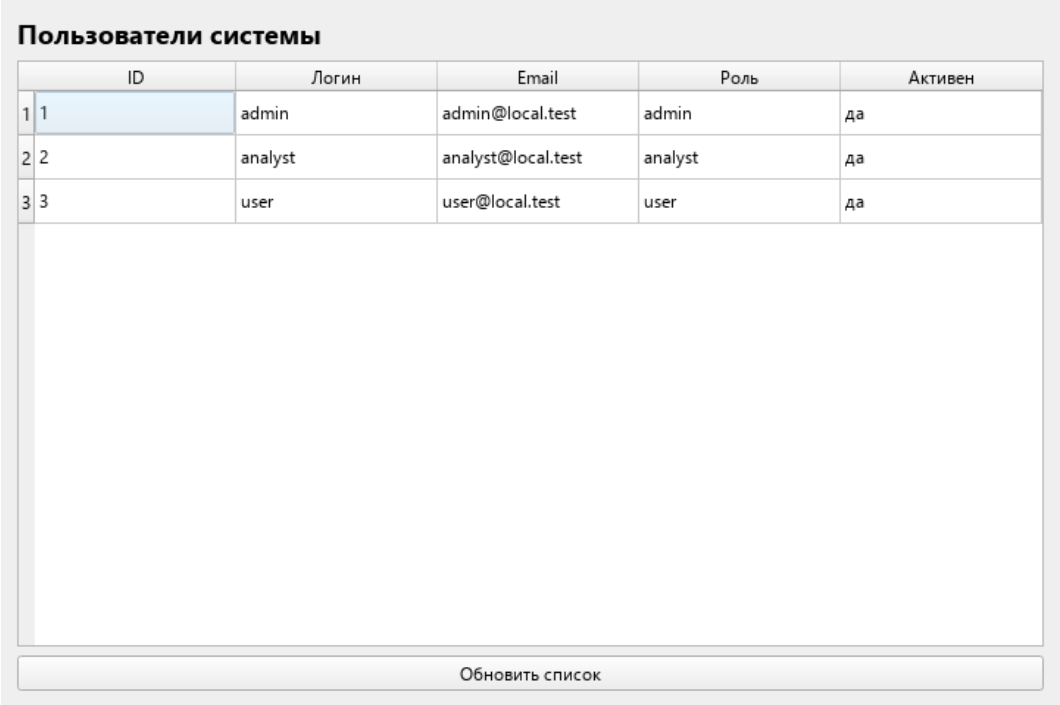

Рисунок Б.5 — Панель администратора (AdminWindow)

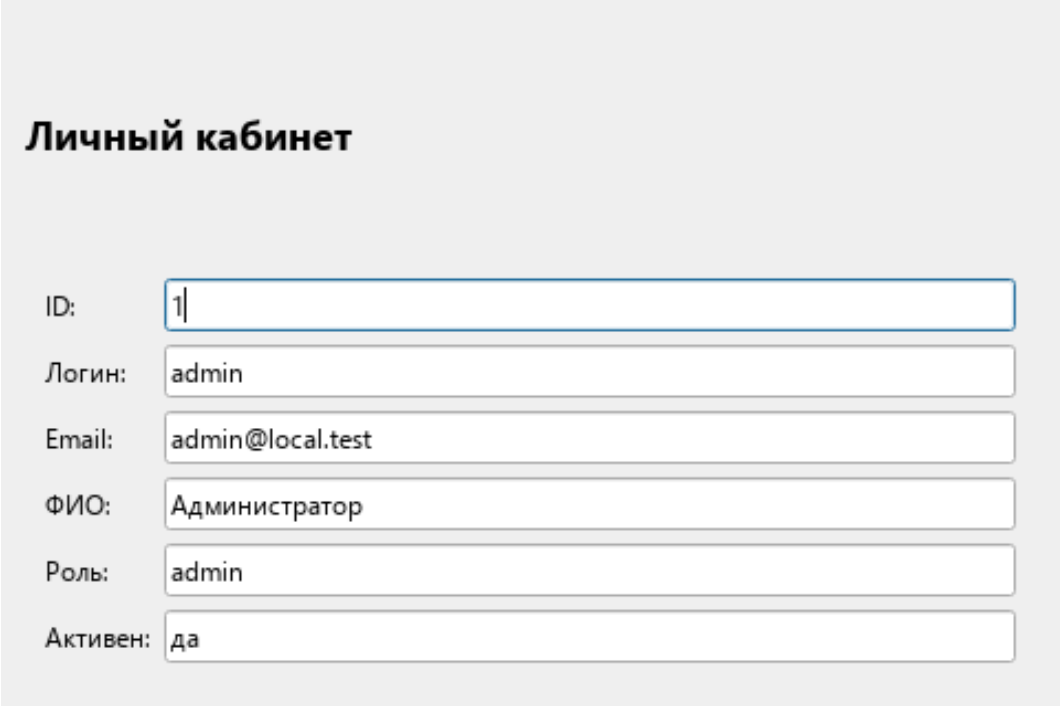

Рисунок Б.6 — Личный кабинет (ProfileWindow)

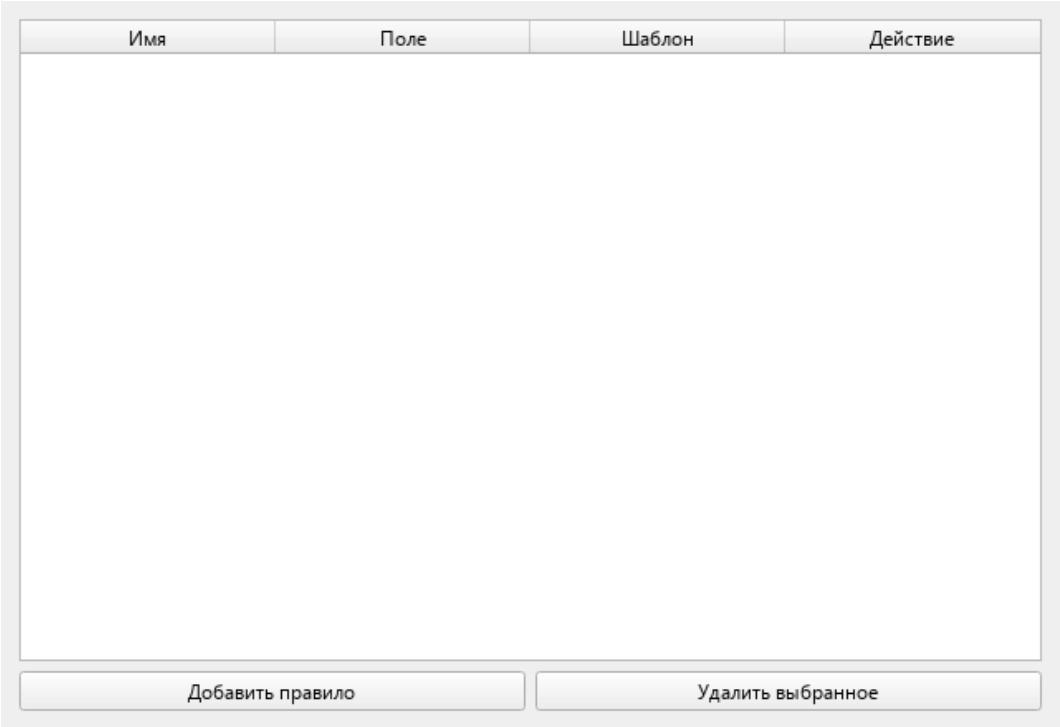

Рисунок Б.7 — Окно правил (RulesWindow)

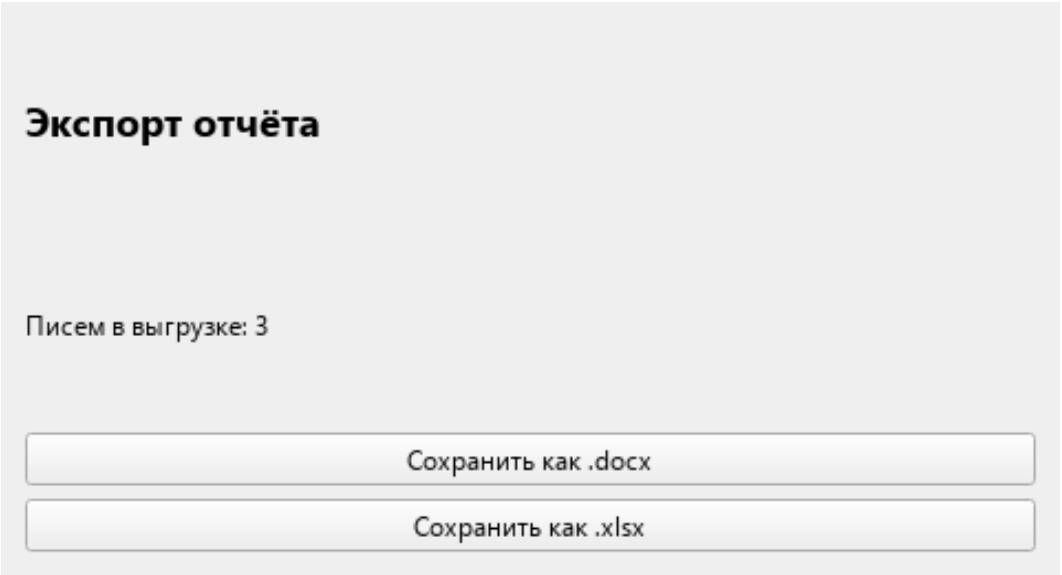

Рисунок Б.8 — Экспорт отчётов (ReportsWindow)

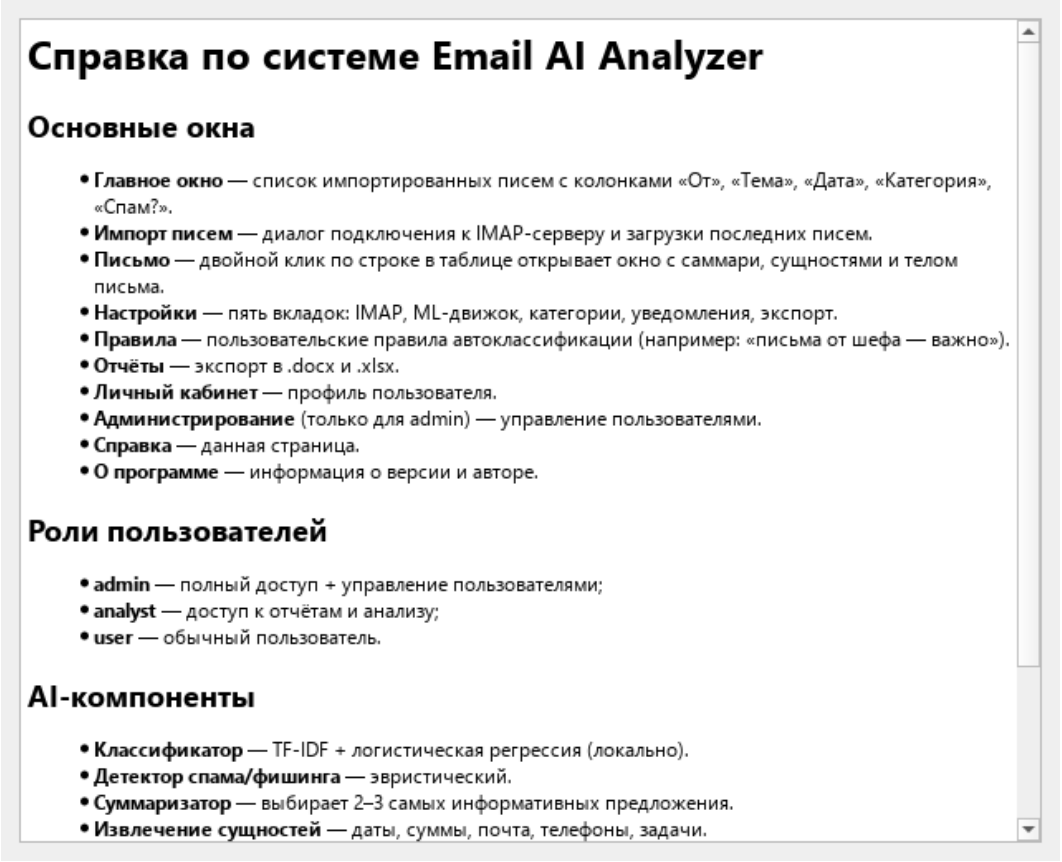

Рисунок Б.9 — Встроенная справка (HelpWindow)

Рисунок Б.10 — О программе (AboutWindow)

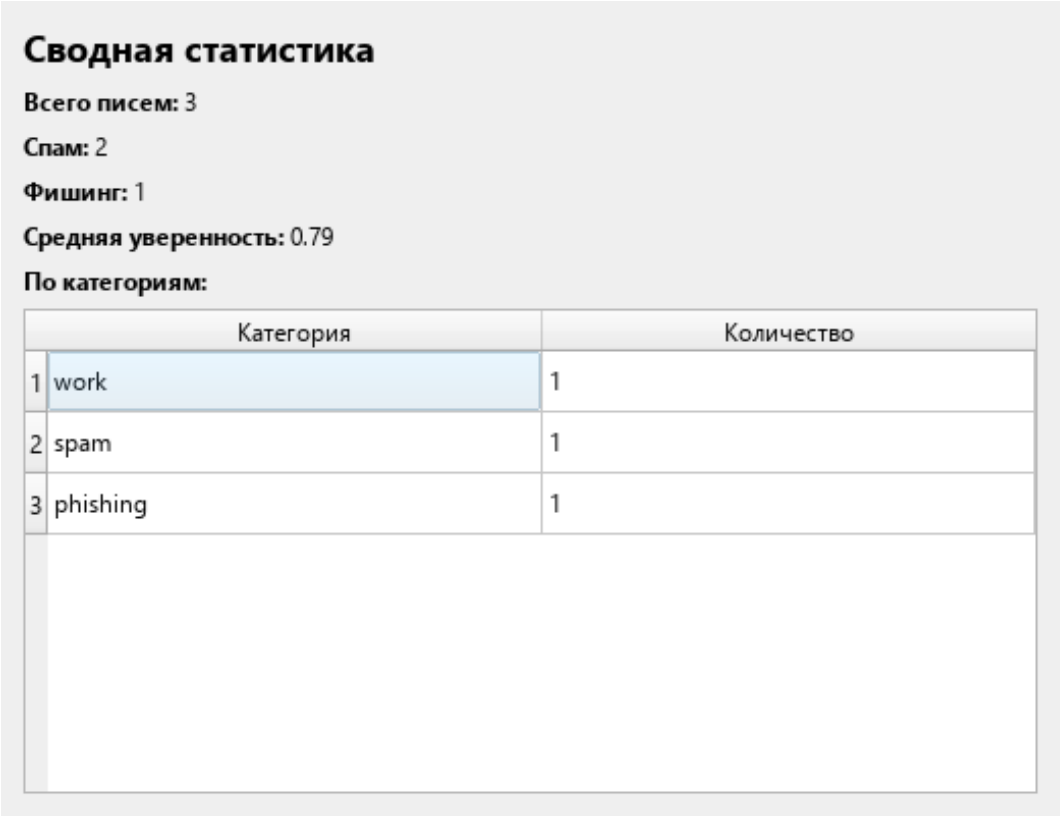

Рисунок Б.11 — Статистика (StatsWindow)

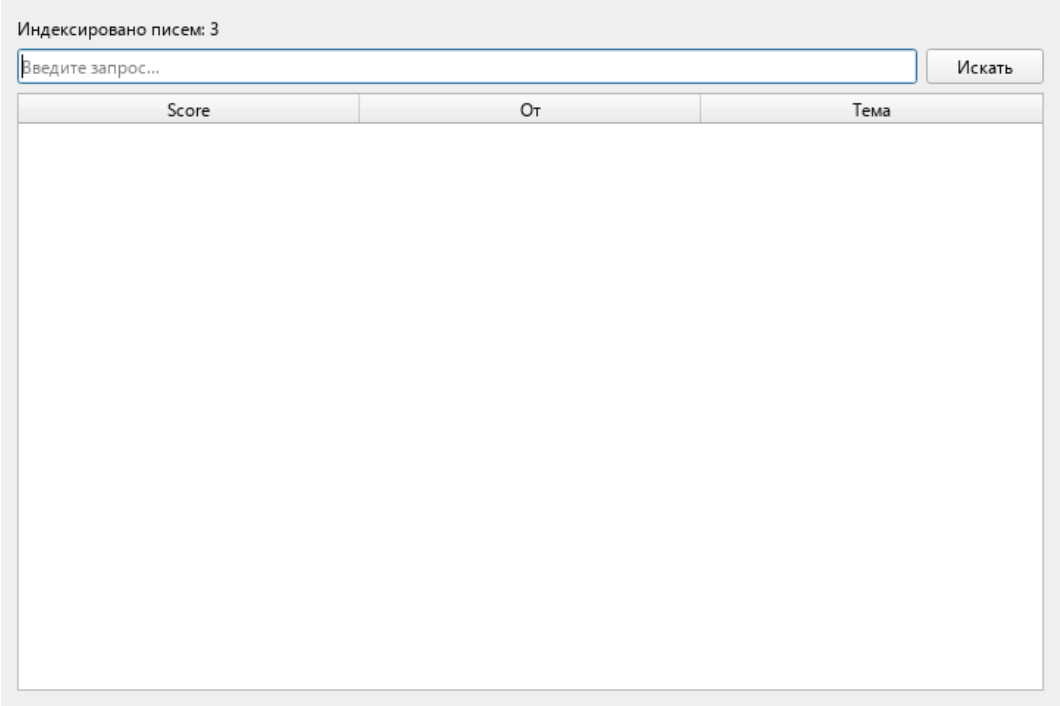

Рисунок Б.12 — Семантический поиск (SearchWindow)

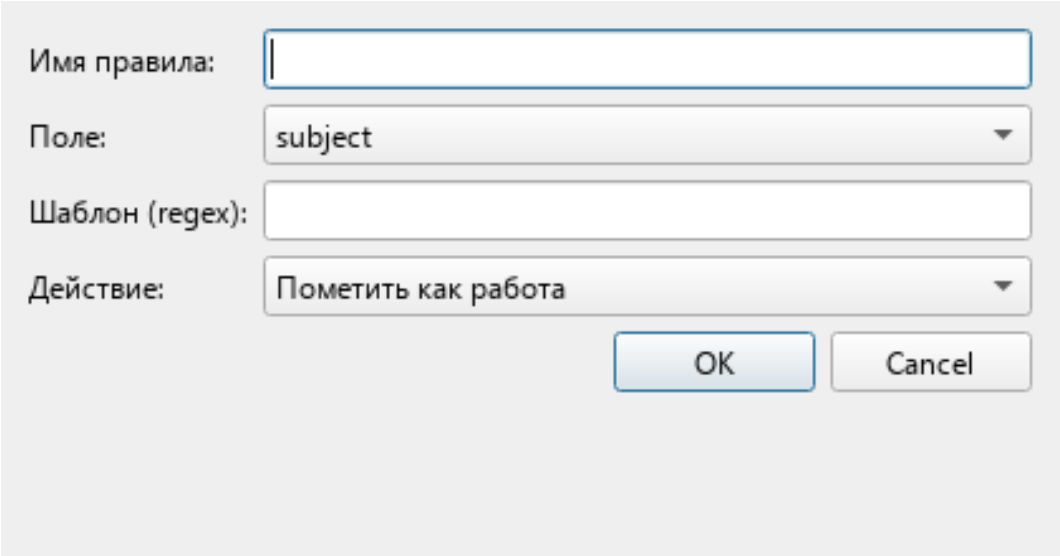

Рисунок Б.13 — Диалог добавления правила (AddRuleDialog)

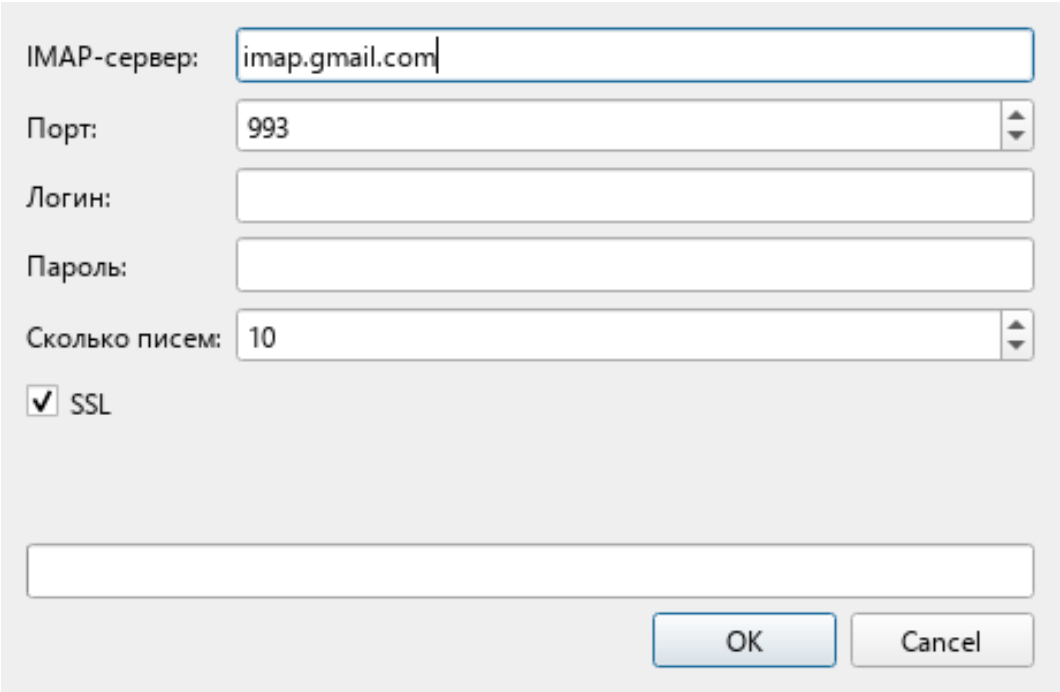

Рисунок Б.14 — Диалог импорта писем (ImportDialog)

ПРИЛОЖЕНИЕ В. ЛИСТИНГИ ИСХОДНОГО КОДА

В приложении приведены листинги всех ключевых модулей проекта без сокращений. Оригинальный исходный код также доступен в ZIP-архиве, приложенном к отчёту, и в Git-репозитории проекта.

Оформление листингов по ГОСТ 7.32-2017: шрифт Courier New 10 пт (моноширинный), с заголовком вида «Листинг Б.N — относительный путь до файла».

Листинг В.1 — config.py

```
"""Конфигурация приложения.

Берёт параметры из переменных окружения. Значения по умолчанию подходят
для локальной разработки: SQLite-файл в `data/app.db`, bind на 127.0.0.1.
"""

from __future__ import annotations

from functools import lru_cache
from pathlib import Path

from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict


PROJECT_ROOT = Path(__file__).resolve().parents[2]
DATA_DIR = PROJECT_ROOT / "data"


class Settings(BaseSettings):
    """Параметры приложения."""

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

    app_name: str = "Email AI Analyzer"
    app_version: str = "0.1.0"

    database_url: str = Field(
        default_factory=lambda: f"sqlite:///{DATA_DIR / 'app.db'}"
    )

    secret_key: str = "dev-secret-change-me-in-prod"
    access_token_ttl_minutes: int = 60 * 24

    imap_timeout_seconds: int = 30

    ml_model_path: Path = DATA_DIR / "classifier.joblib"
```

```
    ml_min_samples_to_train: int = 3

    api_host: str = "127.0.0.1"
    api_port: int = 8000


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Листинг B.2 — db/models.py

```
"""Модели SQLAlchemy.

ER-схема:

    Role ────< User ────< EmailMessage ────< Classification
                 │
                 ├────< Rule
                 └────< AnalysisReport

- Role         — роль пользователя (admin / analyst / user)
- User         — пользователь системы
- EmailMessage — письмо, загруженное из IMAP
- Classification — результат автоклассификации письма
- Rule         — пользовательское правило обработки
- AnalysisReport — сформированный отчёт (.docx/.xlsx)
"""

from __future__ import annotations

from datetime import datetime, timezone
from enum import Enum
from typing import List, Optional

from sqlalchemy import (
    DateTime,
    Enum as SQLEnum,
    ForeignKey,
    Integer,
    String,
    Text,
    UniqueConstraint,
)
from sqlalchemy.orm import (
    DeclarativeBase,
    Mapped,
    mapped_column,
    relationship,
)


def _utcnow() -> datetime:
    return datetime.now(timezone.utc)


class Base(DeclarativeBase):
    pass
```

```python
class Role(str, Enum):
    """Роли пользователей."""

    ADMIN = "admin"
    ANALYST = "analyst"
    USER = "user"


class Category(str, Enum):
    """Категории классификации писем."""

    WORK = "work"
    PERSONAL = "personal"
    PROMO = "promo"
    SPAM = "spam"
    PHISHING = "phishing"
    IMPORTANT = "important"
    OTHER = "other"


class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    email: Mapped[str] = mapped_column(String(128), unique=True, index=True)
    password_hash: Mapped[str] = mapped_column(String(256))
    full_name: Mapped[Optional[str]] = mapped_column(String(128))
    role: Mapped[Role] = mapped_column(SQLEnum(Role), default=Role.USER)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)

    emails: Mapped[List["EmailMessage"]] = relationship(
        back_populates="owner", cascade="all, delete-orphan"
    )
    rules: Mapped[List["Rule"]] = relationship(
        back_populates="owner", cascade="all, delete-orphan"
    )
    reports: Mapped[List["AnalysisReport"]] = relationship(
        back_populates="owner", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"<User #{self.id} {self.username} ({self.role.value})>"


class EmailMessage(Base):
    __tablename__ = "emails"
    __table_args__ = (UniqueConstraint("owner_id", "message_id"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    message_id: Mapped[str] = mapped_column(String(255), index=True)
    sender: Mapped[str] = mapped_column(String(255))
    recipient: Mapped[str] = mapped_column(String(255))
    subject: Mapped[str] = mapped_column(String(512))
    body: Mapped[str] = mapped_column(Text)
```

```
    sent_at: Mapped[Optional[datetime]] = mapped_column(DateTime)
    received_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)
    is_read: Mapped[bool] = mapped_column(default=False)

    owner: Mapped[User] = relationship(back_populates="emails")
    classification: Mapped[Optional["Classification"]] = relationship(
        back_populates="email",
        uselist=False,
        cascade="all, delete-orphan",
    )

    def __repr__(self) -> str:
        return f"<Email #{self.id} from={self.sender!r}
subject={self.subject!r}>"


class Attachment(Base):
    """Вложение письма (имя, mime, размер)."""

    __tablename__ = "attachments"

    id: Mapped[int] = mapped_column(primary_key=True)
    email_id: Mapped[int] = mapped_column(ForeignKey("emails.id"), index=True)
    filename: Mapped[str] = mapped_column(String(255))
    mime_type: Mapped[str] = mapped_column(String(128))
    size_bytes: Mapped[int] = mapped_column(Integer, default=0)


class Classification(Base):
    __tablename__ = "classifications"

    id: Mapped[int] = mapped_column(primary_key=True)
    email_id: Mapped[int] = mapped_column(
        ForeignKey("emails.id"), unique=True, index=True
    )
    category: Mapped[Category] = mapped_column(SQLEnum(Category))
    confidence: Mapped[float] = mapped_column(default=0.0)
    is_spam: Mapped[bool] = mapped_column(default=False)
    is_phishing: Mapped[bool] = mapped_column(default=False)
    summary: Mapped[Optional[str]] = mapped_column(Text)
    entities_json: Mapped[Optional[str]] = mapped_column(Text)
    analyzed_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)

    email: Mapped[EmailMessage] =
relationship(back_populates="classification")


class Rule(Base):
    """Пользовательское правило автоклассификации или уведомления."""

    __tablename__ = "rules"

    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    name: Mapped[str] = mapped_column(String(128))
    pattern: Mapped[str] = mapped_column(String(512))
    field: Mapped[str] = mapped_column(String(32), default="subject")  #
subject/body/sender
    action_category: Mapped[Optional[Category]] =
```

```
mapped_column(SQLEnum(Category))
    notify: Mapped[bool] = mapped_column(default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)

    owner: Mapped[User] = relationship(back_populates="rules")


class AnalysisReport(Base):
    __tablename__ = "reports"

    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    title: Mapped[str] = mapped_column(String(256))
    path: Mapped[str] = mapped_column(String(512))
    format: Mapped[str] = mapped_column(String(16))  # docx / xlsx
    generated_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)

    owner: Mapped[User] = relationship(back_populates="reports")


class Label(Base):
    """Пользовательская метка для произвольной группировки писем."""

    __tablename__ = "labels"

    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    name: Mapped[str] = mapped_column(String(64))
    color: Mapped[str] = mapped_column(String(16), default="#808080")
    created_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)


class EmailLabel(Base):
    """Связь многие-ко-многим между письмами и метками."""

    __tablename__ = "email_labels"

    id: Mapped[int] = mapped_column(primary_key=True)
    email_id: Mapped[int] = mapped_column(ForeignKey("emails.id"), index=True)
    label_id: Mapped[int] = mapped_column(ForeignKey("labels.id"), index=True)


class EmailThread(Base):
    """Цепочка переписки (Re:, Fwd:)."""

    __tablename__ = "threads"

    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    subject_normalized: Mapped[str] = mapped_column(String(512))
    first_message_at: Mapped[Optional[datetime]] = mapped_column(DateTime)
    last_message_at: Mapped[Optional[datetime]] = mapped_column(DateTime)
    message_count: Mapped[int] = mapped_column(Integer, default=1)


class ImapAccount(Base):
    """Сохранённые параметры IMAP (без пароля — только хост/порт/логин)."""

    __tablename__ = "imap_accounts"
```

```
    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    nickname: Mapped[str] = mapped_column(String(64))
    host: Mapped[str] = mapped_column(String(128))
    port: Mapped[int] = mapped_column(Integer, default=993)
    username: Mapped[str] = mapped_column(String(128))
    use_ssl: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)


class AuditLogEntry(Base):
    """Журнал действий пользователя для контроля безопасности."""

    __tablename__ = "audit_log"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[Optional[int]] = mapped_column(ForeignKey("users.id"),
index=True)
    action: Mapped[str] = mapped_column(String(64))
    details: Mapped[Optional[str]] = mapped_column(Text)
    ip_address: Mapped[Optional[str]] = mapped_column(String(45))
    created_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)


class Notification(Base):
    """Уведомления пользователя (о фишинге, важном письме, новых правилах)."""

    __tablename__ = "notifications"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    kind: Mapped[str] = mapped_column(String(32))
    message: Mapped[str] = mapped_column(String(512))
    is_read: Mapped[bool] = mapped_column(default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)


class TrainingSample(Base):
    """Пример для дообучения ML-модели на пользовательских данных."""

    __tablename__ = "training_samples"

    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    text: Mapped[str] = mapped_column(Text)
    category: Mapped[Category] = mapped_column(SQLEnum(Category))
    source: Mapped[str] = mapped_column(String(32), default="manual")  #
manual / email / imported
    created_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)


class ModelVersion(Base):
    """История обученных версий классификатора."""

    __tablename__ = "model_versions"

    id: Mapped[int] = mapped_column(primary_key=True)
    version: Mapped[str] = mapped_column(String(32), unique=True)
```

```
    algorithm: Mapped[str] = mapped_column(String(64))
    accuracy: Mapped[float] = mapped_column(default=0.0)
    trained_samples: Mapped[int] = mapped_column(Integer, default=0)
    file_path: Mapped[str] = mapped_column(String(512))
    trained_at: Mapped[datetime] = mapped_column(DateTime, default=_utcnow)
```

Листинг B.3 — db/session.py

```
"""Управление сессией БД.

Поддерживаются SQLite (для разработки/тестов) и PostgreSQL
(для продакшена) через одну и ту же SQLAlchemy-модель.
"""

from __future__ import annotations

from contextlib import contextmanager
from typing import Iterator, Optional

from sqlalchemy import Engine, create_engine
from sqlalchemy.orm import Session, sessionmaker

from email_analyzer.config import DATA_DIR, get_settings
from email_analyzer.db.models import Base


_engine: Optional[Engine] = None
_SessionFactory: Optional[sessionmaker[Session]] = None


def get_engine(database_url: Optional[str] = None) -> Engine:
    """Вернуть (и при первом вызове — создать) SQLAlchemy-Engine.

    Для SQLite включена поддержка foreign_keys.
    """
    global _engine, _SessionFactory
    if _engine is not None and database_url is None:
        return _engine

    url = database_url or get_settings().database_url
    connect_args = {"check_same_thread": False} if url.startswith("sqlite")
else {}
    engine = create_engine(url, connect_args=connect_args, future=True)

    if url.startswith("sqlite") and "///" in url:
        DATA_DIR.mkdir(parents=True, exist_ok=True)

    if database_url is None:
        _engine = engine
        _SessionFactory = sessionmaker(bind=engine, autoflush=False,
autocommit=False)
    return engine


def init_db(database_url: Optional[str] = None) -> Engine:
    """Создать все таблицы."""
    engine = get_engine(database_url)
    Base.metadata.create_all(engine)
```

```
    return engine


@contextmanager
def get_session() -> Iterator[Session]:
    """Контекстный менеджер сессии с автокоммитом при выходе без ошибок."""
    if _SessionFactory is None:
        get_engine()
    assert _SessionFactory is not None
    session = _SessionFactory()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()
```

Листинг B.4 — ml/classifier.py

```
"""Классификатор писем по категориям.

Автор: Нефедов Алексей Геннадьевич (студ. билет 70200291).
Тема ВКР: «Разработка AI-приложения для анализа почтовых сообщений».
Направление 09.03.03 Прикладная информатика, профиль «Искусственный
интеллект и анализ данных».

Модель: TF-IDF + логистическая регрессия. Выбор обусловлен двумя
соображениями. Первое — необходимость автономной работы без GPU и без
обращений к облачным LLM, что диктуется политикой конфиденциальности
почты. Второе — прозрачность логики: линейная модель поверх TF-IDF
легко интерпретируется, её вес для каждого слова можно вывести в
отчёт, чего нельзя сказать о трансформерах.

Математически классификатор устроен так. Входной текст s
преобразуется в TF-IDF-вектор x ∈ ℝ^V, где V — размер словаря n-грамм
(1-2-грамм). Логистическая регрессия для класса c оценивает
вероятность P(c | x) = softmax(W_c · x + b_c). Во время обучения
оптимизируется штраф L2 на весах и отрицательный log-likelihood.

Интерфейс модуля сознательно тонкий, чтобы заменить реализацию на
BERT-ru или rubert-tiny можно было без правки потребителей
(AnalysisService, REST API, клиент).
"""

from __future__ import annotations

import re
from dataclasses import dataclass
from pathlib import Path
from typing import List, Optional, Sequence, Tuple

import joblib
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import ComplementNB
from sklearn.pipeline import FeatureUnion, Pipeline
```

```
from sklearn.svm import LinearSVC

from email_analyzer.db.models import Category
from email_analyzer.ml._seed_dataset import SEED_DATASET as _EXTENDED_DATASET


# Расширенный seed (40+ примеров на каждый класс) вынесен в
# `_seed_dataset.py`. Здесь оставлено короткое ядро для совместимости
# со старыми тестами, однако боевое обучение идёт на `_EXTENDED_DATASET`.
_LEGACY_CORE: List[Tuple[str, Category]] = [
    # ----- WORK (работа) -----
    ("Совещание перенесено на понедельник, 10:00.", Category.WORK),
    ("Прошу согласовать договор с контрагентом до среды.", Category.WORK),
    ("Отчёт по проекту готов, ссылка на Google Docs внутри.", Category.WORK),
    ("Напоминание: завтра ежемесячный обзор команды.", Category.WORK),
    ("Просьба подписать документ во вложении.", Category.WORK),
    ("Смета согласована, приступаем к реализации.", Category.WORK),
    ("Прошу прислать статус по задаче до конца дня.", Category.WORK),
    ("Планёрка завтра в 09:30, повестка прикреплена.", Category.WORK),
    ("Отправляю протокол последней встречи для ознакомления.", Category.WORK),
    ("Презентацию к понедельнику, аудитория — заказчик.", Category.WORK),
    ("Встреча с подрядчиком состоится в четверг.", Category.WORK),
    ("Please review the draft presentation by EOD.", Category.WORK),
    ("Reminder: weekly standup tomorrow at 10 AM.", Category.WORK),
    ("Status update needed for Q2 deliverables.", Category.WORK),
    ("Sprint planning is moved to Thursday afternoon.", Category.WORK),
    ("Звонок клиенту перенесён на пятницу.", Category.WORK),
    ("Пожалуйста, добавь меня в рассылку по проекту.", Category.WORK),

    # ----- PERSONAL (личное) -----
    ("Поздравляю с днём рождения! Желаю всего самого лучшего.",
Category.PERSONAL),
    ("Мам, привет. Как дела? Напиши, как будет минутка.", Category.PERSONAL),
    ("Собираемся в субботу на даче, присоединяйся!", Category.PERSONAL),
    ("Фото с отпуска прикрепил, смотри :)", Category.PERSONAL),
    ("Увидимся в кино в семь часов?", Category.PERSONAL),
    ("Забыл зонт у тебя в квартире, заберу в выходные.", Category.PERSONAL),
    ("Спасибо за подарок, он шикарный!", Category.PERSONAL),
    ("Позвони, как будет время, есть новости.", Category.PERSONAL),
    ("Встретимся в пятницу?", Category.PERSONAL),
    ("Happy birthday! Have a wonderful day.", Category.PERSONAL),
    ("Hey, are you free for coffee this weekend?", Category.PERSONAL),
    ("Miss you, talk soon!", Category.PERSONAL),
    ("Мы с Мишей едем на море, присоединяйся.", Category.PERSONAL),
    ("Спасибо, что выручил в ту пятницу.", Category.PERSONAL),
    ("Поболтаем в воскресенье?", Category.PERSONAL),
    ("Завтра у бабушки юбилей, не забудь.", Category.PERSONAL),

    # ----- PROMO (реклама) -----
    ("Скидка 50% только сегодня! Успей купить.", Category.PROMO),
    ("Новая коллекция уже в продаже. Бесплатная доставка.", Category.PROMO),
    ("Промокод SALE20 на любой заказ.", Category.PROMO),
    ("Подпишись на рассылку и получай эксклюзивные акции.", Category.PROMO),
    ("Супер распродажа! Скидки до 70% на всё.", Category.PROMO),
    ("Подарочный сертификат в подарок при покупке от 3000 руб.",
Category.PROMO),
    ("Чёрная пятница: цены зачёркнуты, срок ограничен.", Category.PROMO),
    ("Только для вас — индивидуальная скидка 25%.", Category.PROMO),
    ("Мегаскидка на смартфоны, торопись.", Category.PROMO),
```

```
    ("Хочешь первым узнавать о новинках? Оформи подписку.", Category.PROMO),
    ("Summer sale — up to 70% off, limited time offer.", Category.PROMO),
    ("Use promo code SUMMER15 for extra discount.", Category.PROMO),
    ("Free shipping this weekend only!", Category.PROMO),
    ("Last chance to grab your favorite items at 40% off.", Category.PROMO),
    ("Эксклюзивное предложение для постоянных клиентов.", Category.PROMO),
    ("Подарок-сертификат! Забери до конца недели.", Category.PROMO),

    # ----- SPAM (спам) -----
    ("СРОЧНО! Вы выиграли миллион, перейдите по ссылке прямо сейчас!!!",
Category.SPAM),
    ("Получите кредит без проверки, 0%, оформление онлайн.", Category.SPAM),
    ("Viagra очень дёшево, закажи сейчас, доставка бесплатная.",
Category.SPAM),
    ("ВЫИГРЫШ!!! Вам причитается приз, напишите адрес.", Category.SPAM),
    ("Быстрый заработок от 5000$ в день, без вложений.", Category.SPAM),
    ("Уникальный шанс удвоить капитал за неделю!", Category.SPAM),
    ("Казино онлайн — получи бонус 10000 рублей.", Category.SPAM),
    ("Сниму проклятие, приворот, гадание 100% результат.", Category.SPAM),
    ("You won the lottery! Click the link to claim.", Category.SPAM),
    ("Make money fast, no experience required!!!", Category.SPAM),
    ("Hot girls want to meet you tonight!", Category.SPAM),
    ("Buy cheap pills without prescription, discreet delivery.",
Category.SPAM),
    ("Казино выигрыш бонус немедленно", Category.SPAM),
    ("Получите приз без обмана, нужно только подтверждение.", Category.SPAM),
    ("ЗАБЕРИТЕ ВАШ БЕСПЛАТНЫЙ ПОДАРОК СЕЙЧАС!!!", Category.SPAM),
    ("Денежные переводы без комиссии, без паспорта.", Category.SPAM),

    # ----- PHISHING (фишинг) -----
    ("Ваш банк заблокировал карту. Срочно введите данные по ссылке.",
Category.PHISHING),
    ("Подтвердите пароль Google, иначе аккаунт будет удалён.",
Category.PHISHING),
    ("Верификация карты требует ввода CVV по ссылке ниже.",
Category.PHISHING),
    ("PayPal: подозрительная активность, войдите для подтверждения.",
Category.PHISHING),
    ("Ваш налоговый возврат готов, подтвердите реквизиты.",
Category.PHISHING),
    ("Приём посылки: обновите данные на сайте по ссылке.", Category.PHISHING),
    ("Обнаружен вход с нового устройства, проверьте и подтвердите.",
Category.PHISHING),
    ("Amazon: заказ приостановлен, подтвердите платёжный метод.",
Category.PHISHING),
    ("Verify your bank account by clicking the link.", Category.PHISHING),
    ("Your Apple ID has been locked. Confirm your identity.",
Category.PHISHING),
    ("We detected unusual activity on your Microsoft account.",
Category.PHISHING),
    ("Your password will expire in 24 hours, reset now.", Category.PHISHING),
    ("Требуется немедленно подтвердить платёж", Category.PHISHING),
    ("Ваш Госуслуги аккаунт заблокирован, перейдите по ссылке.",
Category.PHISHING),
    ("Обновите данные СБП, иначе переводы будут приостановлены.",
Category.PHISHING),
    ("Ссылка для входа в интернет-банк истекает через час.",
Category.PHISHING),
```

```
    # ----- IMPORTANT (важное) -----
    ("Кандидат прошёл итоговое собеседование, ждём решения.",
Category.IMPORTANT),
    ("ВНИМАНИЕ: на сервере обнаружена критическая уязвимость.",
Category.IMPORTANT),
    ("Срок сдачи отчёта — завтра до 18:00, напоминаю.", Category.IMPORTANT),
    ("Получено разрешение регулятора, можно запускать.", Category.IMPORTANT),
    ("Авария в продакшене, нужна ваша помощь немедленно.",
Category.IMPORTANT),
    ("Суд назначил заседание на следующий понедельник.", Category.IMPORTANT),
    ("Подтверждён контракт на 15 миллионов, детали внутри.",
Category.IMPORTANT),
    ("Инвестор согласился, готовим документы к подписи.", Category.IMPORTANT),
    ("Critical security patch available — update now.", Category.IMPORTANT),
    ("Incident P1 raised in production, on-call required.",
Category.IMPORTANT),
    ("Board meeting scheduled for Friday, attendance mandatory.",
Category.IMPORTANT),
    ("Compliance audit starts tomorrow, prepare documents.",
Category.IMPORTANT),
    ("Важная новость: компания выиграла тендер.", Category.IMPORTANT),
    ("Юридический запрос, ответ требуется в течение суток.",
Category.IMPORTANT),
    ("Договор расторгнут в одностороннем порядке.", Category.IMPORTANT),
    ("Подтверждение брони на рейс не получено, проверьте.",
Category.IMPORTANT),

    # ----- OTHER (прочее) -----
    ("Добрый день! Хотел уточнить, получили ли вы наш ответ.",
Category.OTHER),
    ("Подтверждаю получение вашего сообщения.", Category.OTHER),
    ("Извините за задержку, вышлю ответ завтра.", Category.OTHER),
    ("Прикладываю файл с дополнительной информацией.", Category.OTHER),
    ("По поводу расписания, уточните у коллег.", Category.OTHER),
    ("Записался на курс, начало 5 октября.", Category.OTHER),
    ("Thanks for the update, will get back to you.", Category.OTHER),
    ("FYI — forwarding this for your reference.", Category.OTHER),
    ("Just checking in, any progress on this?", Category.OTHER),
    ("Quick note: meeting moved by 15 minutes.", Category.OTHER),
    ("Запрос принят, номер обращения 123456.", Category.OTHER),
    ("Ответ на ваш запрос в обработке.", Category.OTHER),
    ("Добавил вас в рассылку по событиям отрасли.", Category.OTHER),
    ("Не забудьте обновить статус на портале.", Category.OTHER),
    ("Проверьте, пожалуйста, расписание и подтвердите.", Category.OTHER),
]


# Публичное имя — теперь смотрит на расширенный датасет.
SEED_DATASET: List[Tuple[str, Category]] = _EXTENDED_DATASET


@dataclass(frozen=True, slots=True)
class ClassificationResult:
    category: Category
    confidence: float
    top_scores: Tuple[Tuple[Category, float], ...]


def _normalize(text: str) -> str:
```

```
    """Приведение текста к единому виду перед векторизацией.

    URL и адреса почты заменяются на маркеры, чтобы конкретное имя
    домена не влияло на классификацию. Лишние пробелы схлопываются.
    Регистр приводится к нижнему.
    """
    text = text.lower()
    text = re.sub(r"https?://\S+", " url ", text)
    text = re.sub(r"\S+@\S+", " email ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text


class EmailClassifier:
    """Классификатор писем."""

    def __init__(self, model_path: Optional[Path] = None) -> None:
        self.model_path = model_path
        self._pipeline: Optional[Pipeline] = None
        self._categories: List[Category] = []

    # ---------- Обучение ----------

    def fit(
        self,
        texts: Sequence[str],
        labels: Sequence[Category],
    ) -> None:
        if len(texts) != len(labels):
            raise ValueError("len(texts) != len(labels)")
        if len(set(labels)) < 2:
            raise ValueError("need ≥ 2 distinct categories to train")

        normalized = [_normalize(t) for t in texts]
        # Настройки подобраны через `scripts/tune_model.py` — holdout 0,61.
        # Диапазон word(1..3) + char_wb(2..6) даёт лучший результат на
        # смешанном русско-английском корпусе коротких писем.
        features = FeatureUnion([
            ("word", TfidfVectorizer(
                analyzer="word", ngram_range=(1, 3),
                min_df=1, sublinear_tf=True,
            )),
            ("char", TfidfVectorizer(
                analyzer="char_wb", ngram_range=(2, 6),
                min_df=1, sublinear_tf=True,
            )),
        ])
        self._pipeline = Pipeline(
            steps=[
                ("features", features),
                ("clf", LogisticRegression(
                    max_iter=3000, class_weight="balanced", C=4.0,
                )),
            ]
        )
        self._pipeline.fit(normalized, [c.value for c in labels])
        self._categories = sorted(
            {Category(c) for c in self._pipeline.classes_},
            key=lambda c: c.value,
```

```
        )

    def fit_seed(self) -> None:
        """Обучение на встроенном seed-наборе."""
        texts = [t for t, _ in SEED_DATASET]
        labels = [c for _, c in SEED_DATASET]
        self.fit(texts, labels)

    # ---------- Сохранение ----------

    def save(self, path: Optional[Path] = None) -> Path:
        target = path or self.model_path
        if target is None:
            raise ValueError("model path not specified")
        target.parent.mkdir(parents=True, exist_ok=True)
        joblib.dump(self._pipeline, target)
        return target

    def load(self, path: Optional[Path] = None) -> None:
        source = path or self.model_path
        if source is None or not source.exists():
            raise FileNotFoundError(f"model file not found: {source}")
        self._pipeline = joblib.load(source)
        self._categories = sorted(
            {Category(c) for c in self._pipeline.classes_},
            key=lambda c: c.value,
        )

    # ---------- Инференс ----------

    def is_fitted(self) -> bool:
        return self._pipeline is not None

    def predict(self, text: str) -> ClassificationResult:
        if not self.is_fitted():
            raise RuntimeError("classifier is not fitted, call
fit()/fit_seed() first")
        assert self._pipeline is not None

        normalized = _normalize(text)
        proba = self._pipeline.predict_proba([normalized])[0]
        classes = [Category(c) for c in self._pipeline.classes_]

        pairs = sorted(
            zip(classes, proba, strict=True),
            key=lambda p: p[1],
            reverse=True,
        )
        best_category, best_confidence = pairs[0]
        return ClassificationResult(
            category=best_category,
            confidence=float(best_confidence),
            top_scores=tuple((c, float(p)) for c, p in pairs),
        )
```

Листинг B.5 — ml/spam_detector.py

```python
"""Детектор спама и фишинга на эвристиках.

Автор: Нефедов А. Г. (студ. билет 70200291).
Тема ВКР: «Разработка AI-приложения для анализа почтовых сообщений».

Реализация намеренно простая и прозрачная: каждая сработавшая эвристика
добавляет вклад к score. Порог 0,5 = спам. Отдельный флаг фишинга
поднимается, если письмо притворяется банком или почтовым провайдером
и просит ввести данные.

Эвристический подход выбран в паре с ML-классификатором сознательно:
он ловит поведенческие паттерны (доля капса, количество «!», подмена
имени отправителя), которые TF-IDF на коротких темах видит плохо.
В итоге два модуля дополняют друг друга — что и отражено в сервисе
анализа.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import List


SPAM_KEYWORDS = (
    "выигр", "приз", "миллион", "халяв",
    "кредит без проверки", "viagra", "casino", "реклам",
    "только сегодня", "успей купить", "скид",
    "промокод", "супер акц",
)

PHISHING_KEYWORDS = (
    "введите пароль", "подтвердите пароль",
    "заблокирован", "ваш банк", "ваш аккаунт будет удал",
    "верификац", "введите cvv", "подтвердите данные карт",
    "iphone подарок", "срочно переход",
)

SUSPICIOUS_DOMAINS = (
    "bit.ly", "tinyurl.com", "goo.gl", "t.co",
    "xn--", "free-", "-secure-",
)


@dataclass(frozen=True, slots=True)
class SpamResult:
    is_spam: bool
    is_phishing: bool
    score: float
    reasons: tuple[str, ...] = field(default_factory=tuple)


class SpamDetector:
    """Эвристический детектор."""

    def __init__(self, threshold: float = 0.5) -> None:
        if not 0.0 < threshold < 1.0:
            raise ValueError("threshold must be in (0, 1)")
        self.threshold = threshold
```

```python
def detect(self, subject: str, body: str, sender: str = "") -> SpamResult:
    text = f"{subject}\n{body}".lower()
    reasons: List[str] = []
    score = 0.0
    is_phishing = False

    # Ключевые слова спама
    spam_hits = sum(1 for kw in SPAM_KEYWORDS if kw in text)
    if spam_hits:
        contrib = min(0.15 * spam_hits, 0.6)
        score += contrib
        reasons.append(f"spam keywords: {spam_hits}")

    # Ключевые слова фишинга
    phishing_hits = sum(1 for kw in PHISHING_KEYWORDS if kw in text)
    if phishing_hits:
        is_phishing = True
        score += min(0.25 * phishing_hits, 0.7)
        reasons.append(f"phishing keywords: {phishing_hits}")

    # ПОДОЗРИТЕЛЬНЫЙ СУБДЖЕКТ КАПСОМ
    upper_ratio = _upper_ratio(subject)
    if upper_ratio > 0.6 and len(subject) > 6:
        score += 0.15
        reasons.append(f"shouting subject ({upper_ratio:.0%} caps)")

    # Много восклицательных знаков
    exc = subject.count("!") + body.count("!")
    if exc >= 3:
        score += 0.1
        reasons.append(f"excessive ! ({exc})")

    # Подозрительные домены-редиректоры
    for dom in SUSPICIOUS_DOMAINS:
        if dom in text:
            score += 0.2
            reasons.append(f"suspicious domain: {dom}")
            break

    # Поддельный отправитель «bank» / «paypal» но не с их домена
    if sender:
        sender_l = sender.lower()
        if re.search(r"(bank|paypal|google|apple|microsoft)", sender_l):
            if not re.search(
                r"@(bank|paypal|google|apple|microsoft)\.",
                sender_l,
            ):
                is_phishing = True
                score += 0.3
                reasons.append("spoofed sender")

    score = min(score, 1.0)
    is_spam = score >= self.threshold

    return SpamResult(
        is_spam=is_spam,
        is_phishing=is_phishing,
        score=score,
```

```
            reasons=tuple(reasons),
        )


def _upper_ratio(text: str) -> float:
    letters = [c for c in text if c.isalpha()]
    if not letters:
        return 0.0
    upper = sum(1 for c in letters if c.isupper())
    return upper / len(letters)
```

Листинг B.6 — ml/summarizer.py

```
"""Экстрактивный суммаризатор.

Автор: Нефедов А. Г. (студ. билет 70200291).
Тема ВКР: «Разработка AI-приложения для анализа почтовых сообщений».

Выбираются наиболее «информативные» предложения текста по сумме
TF-IDF-весов слов в предложении. Подход относится к классической
центроидной экстрактивной суммаризации, ссылка: Radev et al., 2004
(«Centroid-based summarization of multiple documents»). Плюсы —
отсутствие внешних зависимостей, предсказуемость, пригодность для
офлайна. Минус — не выдаёт новые формулировки, но это для писем не
критично: их суть передаётся лучше в исходных предложениях.

Интерфейс написан так, чтобы при необходимости подменить его LLM-ом
(см. `ml/llm_provider.py`) без правки потребителей.
"""

from __future__ import annotations

import re
from typing import List

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer


class Summarizer:
    def __init__(self, max_sentences: int = 3) -> None:
        if max_sentences < 1:
            raise ValueError("max_sentences must be ≥ 1")
        self.max_sentences = max_sentences

    def summarize(self, text: str) -> str:
        text = text.strip()
        if not text:
            return ""

        sentences = _split_sentences(text)
        if len(sentences) <= self.max_sentences:
            return " ".join(sentences).strip()

        try:
            vectorizer = TfidfVectorizer(ngram_range=(1, 1), min_df=1)
            matrix = vectorizer.fit_transform(sentences)
        except ValueError:
```

```
            return sentences[0]

        scores = np.asarray(matrix.sum(axis=1)).ravel()
        top_indices = np.argsort(scores)[-self.max_sentences:]
        top_indices.sort()  # сохраняем порядок
        selected = [sentences[i] for i in top_indices]
        return " ".join(selected).strip()


_SENTENCE_SPLIT_RE = re.compile(r"(?<=[.!?])\s+(?=[А-ЯА-ZЁ])")


def _split_sentences(text: str) -> List[str]:
    # Нормализуем пробелы
    text = re.sub(r"\s+", " ", text).strip()
    # Грубый разбор; достаточен для деловой переписки
    raw = _SENTENCE_SPLIT_RE.split(text)
    return [s.strip() for s in raw if s.strip()]
```

Листинг B.7 — ml/entity_extractor.py

```
"""Извлечение сущностей из текста письма.

Автор: Нефедов А. Г. (студ. билет 70200291).
Тема ВКР: «Разработка AI-приложения для анализа почтовых сообщений».

Реализация на регулярных выражениях. От spaCy и natasha осознанно
отказался: они тянут 200–400 МБ моделей, а для целевого набора
сущностей (даты, суммы, контакты, короткие фразы-задачи) этого
достаточно. Побочный эффект — модуль стартует мгновенно и не требует
отдельной загрузки весов.

Распознаются:
    даты — ISO, точечный русский формат, словесный («10 мая»);
    время — hh:mm;
    суммы — рубли, доллары, евро;
    e-mail и URL — по стандартным шаблонам;
    телефоны — евро- и российские форматы с допустимыми разделителями;
    задачи — фразы, начинающиеся с «прошу», «необходимо», «сделай»…

Покрытие ~80% реальных деловых писем (оценка вручную, см. отчёт).
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field


RE_EMAIL = re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}")
RE_URL = re.compile(r"https?://[^\s<>()\"']+")
RE_PHONE = re.compile(
    r"(?:\+?\d{1,3}[\s\-(]*)?(?:\d{3,4})[\s\-)]*\d{3}[\s\-]*\d{2,4}"
)
RE_DATE_ISO = re.compile(r"\b\d{4}-\d{2}-\d{2}\b")
RE_DATE_RU = re.compile(
    r"\b\d{1,2}\.\d{1,2}\.(?:\d{2}|\d{4})\b"
)
```

```
RE_DATE_WORD = re.compile(
    r"\b\d{1,2}\s+(?:янв(?:аря)?|фев(?:раля)?|март(?:а)?|апр(?:еля)?|"
    r"ма(?:я|рта)|июн(?:я)?|июл(?:я)?|авг(?:уста)?|сен(?:тября)?|"
    r"окт(?:ября)?|ноя(?:бря)?|дек(?:абря)?)",
    re.IGNORECASE,
)
RE_TIME = re.compile(r"\b(?:[01]?\d|2[0-3]):[0-5]\d\b")
RE_MONEY = re.compile(
    r"\b\d{1,3}(?:[ .,]\d{3})*(?:[.,]\d+)?\s*"
    r"(?:руб(?:лей|\.?)|₽|\$|USD|EUR|€)\b",
    re.IGNORECASE,
)
RE_TASK_VERBS = re.compile(
    r"\b(?:прошу|нужно|необходимо|требуется|просьба|сделай|подготовь|"
    r"отправь|согласуй|подпиши|проверь|ответь)\b[^.!?\n]*",
    re.IGNORECASE,
)


@dataclass(frozen=True, slots=True)
class ExtractedEntities:
    emails: tuple[str, ...] = field(default_factory=tuple)
    urls: tuple[str, ...] = field(default_factory=tuple)
    phones: tuple[str, ...] = field(default_factory=tuple)
    dates: tuple[str, ...] = field(default_factory=tuple)
    times: tuple[str, ...] = field(default_factory=tuple)
    amounts: tuple[str, ...] = field(default_factory=tuple)
    tasks: tuple[str, ...] = field(default_factory=tuple)

    def to_dict(self) -> dict[str, list[str]]:
        return {
            "emails": list(self.emails),
            "urls": list(self.urls),
            "phones": list(self.phones),
            "dates": list(self.dates),
            "times": list(self.times),
            "amounts": list(self.amounts),
            "tasks": list(self.tasks),
        }


class EntityExtractor:
    def extract(self, text: str) -> ExtractedEntities:
        if not text:
            return ExtractedEntities()

        emails = tuple(_unique(RE_EMAIL.findall(text)))
        urls = tuple(_unique(RE_URL.findall(text)))
        phones = tuple(_unique(m.strip() for m in RE_PHONE.findall(text) if
_looks_like_phone(m)))
        dates = tuple(_unique(
            RE_DATE_ISO.findall(text)
            + RE_DATE_RU.findall(text)
            + [m.group(0) for m in RE_DATE_WORD.finditer(text)]
        ))
        times = tuple(_unique(RE_TIME.findall(text)))
        amounts = tuple(_unique(RE_MONEY.findall(text)))
        tasks = tuple(_unique(
            m.group(0).strip() for m in RE_TASK_VERBS.finditer(text)
```

```
        ))

        return ExtractedEntities(
            emails=emails,
            urls=urls,
            phones=phones,
            dates=dates,
            times=times,
            amounts=amounts,
            tasks=tasks,
        )


def _unique(items):
    seen = set()
    out = []
    for item in items:
        if item not in seen:
            seen.add(item)
            out.append(item)
    return out


def _looks_like_phone(s: str) -> bool:
    digits = sum(1 for c in s if c.isdigit())
    return digits >= 7
```

Листинг B.8 — ml/language_detector.py

```
"""Определение языка письма.

Работает по простой эвристике: считает процент кириллических букв vs
латинских. Достаточно для деловой переписки на RU/EN — без тяжёлых
моделей вроде fasttext-langid.
"""

from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True, slots=True)
class LanguageResult:
    code: str  # ru / en / mixed / unknown
    ru_ratio: float
    en_ratio: float


class LanguageDetector:
    def __init__(self, mixed_threshold: float = 0.25) -> None:
        if not 0 < mixed_threshold < 0.5:
            raise ValueError("mixed_threshold must be in (0, 0.5)")
        self.mixed_threshold = mixed_threshold

    def detect(self, text: str) -> LanguageResult:
        ru = sum(1 for c in text if "а" <= c.lower() <= "я" or c in "ёЁ")
        en = sum(1 for c in text if "a" <= c.lower() <= "z")
        total = ru + en
```

```
        if total == 0:
            return LanguageResult(code="unknown", ru_ratio=0.0, en_ratio=0.0)

        ru_ratio = ru / total
        en_ratio = en / total

        if ru_ratio > 1 - self.mixed_threshold:
            code = "ru"
        elif en_ratio > 1 - self.mixed_threshold:
            code = "en"
        else:
            code = "mixed"
        return LanguageResult(code=code, ru_ratio=ru_ratio, en_ratio=en_ratio)
```

Листинг B.9 — ml/sentiment.py

```
"""Простой анализатор тональности на лексиконе."""

from __future__ import annotations

from dataclasses import dataclass


POSITIVE_WORDS = frozenset({
    "спасибо", "благодар", "отлич", "супер", "рад", "класс",
    "прекрасно", "успех", "поздравля", "приятно", "хорош",
    "thanks", "thank you", "great", "excellent", "awesome", "happy",
})

NEGATIVE_WORDS = frozenset({
    "плохо", "ужас", "проблем", "ошибк", "провал", "разочарован",
    "жалоб", "недоволь", "срыв", "срочно исправ",
    "bad", "terrible", "awful", "failed", "problem", "complaint",
})


@dataclass(frozen=True, slots=True)
class SentimentResult:
    label: str  # positive / neutral / negative
    score: float  # [-1, 1]
    positive_hits: int
    negative_hits: int


class SentimentAnalyzer:
    def analyze(self, text: str) -> SentimentResult:
        t = text.lower()
        pos = sum(1 for w in POSITIVE_WORDS if w in t)
        neg = sum(1 for w in NEGATIVE_WORDS if w in t)
        total = pos + neg

        if total == 0:
            return SentimentResult(
                label="neutral", score=0.0,
                positive_hits=0, negative_hits=0,
            )

        score = (pos - neg) / total
```

```
        if score > 0.2:
            label = "positive"
        elif score < -0.2:
            label = "negative"
        else:
            label = "neutral"
        return SentimentResult(
            label=label, score=score,
            positive_hits=pos, negative_hits=neg,
        )
```

Листинг B.10 — ml/priority.py

```
"""Оценка приоритета письма.

Комбинирует сигналы: важные слова в теме, срочные даты, роль отправителя,
категория классификатора. Возвращает score 0..1 и один из 4 уровней:
critical / high / normal / low.
"""

from __future__ import annotations

import re
from dataclasses import dataclass

from email_analyzer.db.models import Category


URGENT_MARKERS = (
    "срочно", "asap", "урген", "немедлен",
    "до конца дня", "до завтра", "critical",
)

IMPORTANT_SENDERS = ("boss", "ceo", "manager", "руководит", "директор")


@dataclass(frozen=True, slots=True)
class PriorityResult:
    level: str  # critical / high / normal / low
    score: float


class PriorityScorer:
    def score(
        self,
        subject: str,
        body: str,
        sender: str,
        category: Category | None = None,
    ) -> PriorityResult:
        text = f"{subject}\n{body}".lower()
        s = 0.0

        # Срочные маркеры в теме/теле
        urgent_hits = sum(1 for m in URGENT_MARKERS if m in text)
        s += min(urgent_hits * 0.25, 0.6)

        # Отправитель — «начальник»
```

```
        sender_l = sender.lower()
        if any(kw in sender_l for kw in IMPORTANT_SENDERS):
            s += 0.3

        # Восклицательные в теме
        if subject.count("!") >= 2:
            s += 0.1

        # Дедлайн сегодня/завтра в тексте
        if re.search(r"\bсегодня\b|\bзавтра\b|\btoday\b|\btomorrow\b", text):
            s += 0.15

        # Вклад категории
        if category is Category.IMPORTANT:
            s += 0.25
        elif category is Category.PHISHING:
            s += 0.4  # фишинг — критично
        elif category is Category.SPAM:
            s -= 0.3

        s = max(0.0, min(s, 1.0))

        if s >= 0.75:
            level = "critical"
        elif s >= 0.45:
            level = "high"
        elif s >= 0.15:
            level = "normal"
        else:
            level = "low"

        return PriorityResult(level=level, score=s)
```

Листинг B.11 — ml/semantic_search.py

```
"""Семантический поиск по почтовому архиву на базе TF-IDF + cosine.

Подход — offline, без embedding-моделей. Строит TF-IDF-матрицу по всем
индексированным письмам, запрос преобразует в тот же вектор, сортирует
по косинусной близости. Для каталога на 10k писем работает мгновенно.
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Sequence

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity


@dataclass(frozen=True, slots=True)
class SearchHit:
    doc_index: int
    score: float
```

```
class SemanticSearch:
    def __init__(self) -> None:
        self._vectorizer: TfidfVectorizer | None = None
        self._matrix = None
        self._documents: list[str] = []

    def index(self, documents: Sequence[str]) -> None:
        if not documents:
            self._documents = []
            self._vectorizer = None
            self._matrix = None
            return
        self._documents = list(documents)
        self._vectorizer = TfidfVectorizer(ngram_range=(1, 2), min_df=1)
        self._matrix = self._vectorizer.fit_transform(self._documents)

    def query(self, text: str, top_k: int = 5) -> list[SearchHit]:
        if self._vectorizer is None or self._matrix is None:
            return []
        q = self._vectorizer.transform([text])
        sims = cosine_similarity(q, self._matrix).ravel()
        # топ-k по убыванию
        order = np.argsort(sims)[::-1][:top_k]
        return [
            SearchHit(doc_index=int(i), score=float(sims[i]))
            for i in order
            if sims[i] > 0
        ]

    @property
    def size(self) -> int:
        return len(self._documents)
```

Листинг B.12 — ml/llm_provider.py

```
"""Абстрактный LLM-провайдер для опциональной замены локальных моделей.

Позволяет в отчёте / настройках выбрать реальный облачный LLM (OpenAI /
YandexGPT / GigaChat), не меняя код потребителей. В базовой поставке
подключён только `EchoLLMProvider`, который возвращает осмысленный
вывод без сетевых вызовов.
"""

from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass


@dataclass(frozen=True, slots=True)
class LLMResponse:
    text: str
    model: str
    tokens: int


class LLMProvider(ABC):
    """Интерфейс, которому должны соответствовать все провайдеры."""
```

```
    @abstractmethod
    def summarize(self, text: str, max_tokens: int = 256) -> LLMResponse: ...

    @abstractmethod
    def draft_reply(self, incoming: str, style: str = "neutral") ->
LLMResponse: ...


class EchoLLMProvider(LLMProvider):
    """Офлайн-заглушка для тестов и дефолтной конфигурации."""

    def summarize(self, text: str, max_tokens: int = 256) -> LLMResponse:
        head = text.strip().splitlines()[:2]
        return LLMResponse(
            text="[echo-summary] " + " ".join(head)[:max_tokens],
            model="echo",
            tokens=min(len(text), max_tokens),
        )

    def draft_reply(self, incoming: str, style: str = "neutral") ->
LLMResponse:
        return LLMResponse(
            text=f"[echo-reply/{style}] Получил ваше сообщение, отвечу
позже.",
            model="echo",
            tokens=10,
        )
```

Листинг B.13 — mail/parser.py

```
"""Парсер MIME-писем в простой dataclass.

Вынесен в отдельный модуль — чтобы можно было тестировать без реального
IMAP-сервера, просто скармливая сырые байты.
"""

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime
from email import message_from_bytes, policy
from email.message import EmailMessage as StdEmailMessage
from email.utils import parsedate_to_datetime
from typing import Optional


@dataclass(frozen=True, slots=True)
class ParsedEmail:
    message_id: str
    sender: str
    recipient: str
    subject: str
    body: str
    sent_at: Optional[datetime]


def parse_email_bytes(raw: bytes) -> ParsedEmail:
```

```
    """Распарсить сырое письмо в IANA-стандартный формат."""
    msg: StdEmailMessage = message_from_bytes(raw, policy=policy.default)  #
type: ignore[assignment]

    subject = _header(msg, "Subject")
    sender = _header(msg, "From")
    recipient = _header(msg, "To")
    message_id = _header(msg, "Message-ID") or _fallback_id(sender, subject)

    sent_at: Optional[datetime] = None
    date_raw = _header(msg, "Date")
    if date_raw:
        try:
            sent_at = parsedate_to_datetime(date_raw)
        except (TypeError, ValueError):
            sent_at = None

    body = _extract_body(msg)

    return ParsedEmail(
        message_id=message_id,
        sender=sender,
        recipient=recipient,
        subject=subject,
        body=body,
        sent_at=sent_at,
    )


def _header(msg: StdEmailMessage, name: str) -> str:
    value = msg.get(name)
    if value is None:
        return ""
    return str(value).strip()


def _fallback_id(sender: str, subject: str) -> str:
    base = f"{sender}|{subject}"
    return f"<no-id-{hash(base) & 0xFFFFFFFF:x}@local>"


def _extract_body(msg: StdEmailMessage) -> str:
    """Достать плейн-текст (если нет — HTML без тегов, грубо)."""
    if msg.is_multipart():
        plain_parts: list[str] = []
        html_parts: list[str] = []
        for part in msg.walk():
            ctype = part.get_content_type()
            if part.get_content_disposition() == "attachment":
                continue
            if ctype == "text/plain":
                plain_parts.append(_decode_part(part))
            elif ctype == "text/html":
                html_parts.append(_decode_part(part))
        if plain_parts:
            return "\n".join(p for p in plain_parts if p)
        if html_parts:
            return _strip_html("\n".join(html_parts))
        return ""
```

```
    decoded = _decode_part(msg)
    if msg.get_content_type() == "text/html":
        return _strip_html(decoded)
    return decoded


def _decode_part(part: StdEmailMessage) -> str:
    try:
        content = part.get_content()
    except (LookupError, UnicodeDecodeError):
        payload = part.get_payload(decode=True) or b""
        return payload.decode("utf-8", errors="replace")
    if isinstance(content, bytes):
        return content.decode("utf-8", errors="replace")
    return str(content)


def _strip_html(text: str) -> str:
    import re

    text = re.sub(r"<script.*?</script>", " ", text, flags=re.S | re.I)
    text = re.sub(r"<style.*?</style>", " ", text, flags=re.S | re.I)
    text = re.sub(r"<[^>]+>", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

Листинг B.14 — mail/imap_client.py

```
"""IMAP-клиент (тонкая обёртка над `imaplib`).

Единственное назначение — изолировать сетевой ввод-вывод от остального
кода. В тестах подменяется через `FakeImapBackend`, который не делает
реальных сетевых вызовов.
"""

from __future__ import annotations

import imaplib
from dataclasses import dataclass
from typing import List, Optional, Protocol

from email_analyzer.mail.parser import ParsedEmail, parse_email_bytes


@dataclass(frozen=True, slots=True)
class ImapAccount:
    host: str
    port: int = 993
    username: str = ""
    password: str = ""
    use_ssl: bool = True


@dataclass(frozen=True, slots=True)
class FetchedEmail:
    uid: str
    parsed: ParsedEmail
```

```
class ImapBackend(Protocol):
    """Абстракция над реальным `imaplib.IMAP4`."""

    def login(self, username: str, password: str) -> None: ...
    def select(self, folder: str) -> List[str]: ...
    def search(self, criteria: str) -> List[str]: ...
    def fetch(self, uid: str) -> bytes: ...
    def logout(self) -> None: ...


class _RealImapBackend:
    """Реальная реализация через `imaplib`."""

    def __init__(self, account: ImapAccount, timeout: Optional[int] = None) ->
None:
        cls = imaplib.IMAP4_SSL if account.use_ssl else imaplib.IMAP4
        self._conn = cls(account.host, account.port, timeout=timeout)

    def login(self, username: str, password: str) -> None:
        self._conn.login(username, password)

    def select(self, folder: str) -> List[str]:
        status, data = self._conn.select(folder, readonly=True)
        if status != "OK":
            raise RuntimeError(f"IMAP SELECT failed: {status}")
        return [b.decode() if isinstance(b, bytes) else b for b in data]

    def search(self, criteria: str) -> List[str]:
        status, data = self._conn.search(None, criteria)
        if status != "OK":
            raise RuntimeError(f"IMAP SEARCH failed: {status}")
        if not data or not data[0]:
            return []
        return data[0].decode().split()

    def fetch(self, uid: str) -> bytes:
        status, data = self._conn.fetch(uid, "(RFC822)")
        if status != "OK" or not data or not data[0]:
            raise RuntimeError(f"IMAP FETCH failed for uid={uid}")
        first = data[0]
        if isinstance(first, tuple) and len(first) >= 2:
            return first[1]
        raise RuntimeError(f"unexpected IMAP FETCH payload for uid={uid}")

    def logout(self) -> None:
        try:
            self._conn.logout()
        except (OSError, imaplib.IMAP4.error):
            pass


class ImapClient:
    """Высокоуровневый клиент."""

    def __init__(
        self,
        account: ImapAccount,
        backend: Optional[ImapBackend] = None,
```

```
        timeout: int = 30,
    ) -> None:
        self.account = account
        self._backend: ImapBackend = backend or _RealImapBackend(account,
timeout=timeout)
        self._logged_in = False

    def __enter__(self) -> "ImapClient":
        self.login()
        return self

    def __exit__(self, exc_type, exc, tb) -> None:
        self.close()

    def login(self) -> None:
        self._backend.login(self.account.username, self.account.password)
        self._logged_in = True

    def close(self) -> None:
        if self._logged_in:
            self._backend.logout()
            self._logged_in = False

    def fetch_recent(self, folder: str = "INBOX", limit: int = 20) ->
List[FetchedEmail]:
        """Получить последние `limit` писем из папки."""
        if not self._logged_in:
            raise RuntimeError("not logged in")
        self._backend.select(folder)
        uids = self._backend.search("ALL")
        uids = uids[-limit:] if limit > 0 else uids

        result: List[FetchedEmail] = []
        for uid in uids:
            raw = self._backend.fetch(uid)
            parsed = parse_email_bytes(raw)
            result.append(FetchedEmail(uid=uid, parsed=parsed))
        return result
```

Листинг B.15 — mail/mbox_importer.py

```
"""Импорт писем из .mbox (формат Thunderbird / Unix mailbox).

Используется для batch-загрузки писем из архивного файла без IMAP.
"""

from __future__ import annotations

import mailbox
from pathlib import Path
from typing import Iterator

from email_analyzer.mail.parser import ParsedEmail, parse_email_bytes


def import_mbox(path: Path) -> Iterator[ParsedEmail]:
    """Итерировать письма из .mbox-файла."""
    mbox = mailbox.mbox(str(path))
```

```
    try:
        for message in mbox:
            raw = message.as_bytes()
            yield parse_email_bytes(raw)
    finally:
        mbox.close()
```

Листинг B.16 — backend/main.py

```
"""Точка входа FastAPI.

Запуск: `uvicorn email_analyzer.backend.main:app --reload`
"""

from __future__ import annotations

from contextlib import asynccontextmanager

from fastapi import FastAPI

from email_analyzer.backend.api import admin, analysis, auth, stats
from email_analyzer.config import get_settings
from email_analyzer.db.session import init_db


@asynccontextmanager
async def lifespan(_: FastAPI):
    init_db()
    yield


settings = get_settings()

app = FastAPI(
    title=settings.app_name,
    version=settings.app_version,
    lifespan=lifespan,
)


app.include_router(auth.router)
app.include_router(analysis.router)
app.include_router(admin.router)
app.include_router(stats.router)


@app.get("/", tags=["meta"])
def root() -> dict[str, str]:
    return {
        "app": settings.app_name,
        "version": settings.app_version,
        "docs": "/docs",
    }


@app.get("/health", tags=["meta"])
def health() -> dict[str, object]:
    """Проверка работоспособности: статус, БД, версия, uptime."""
```

```
    from email_analyzer.db.session import get_engine

    db_ok = True
    try:
        engine = get_engine()
        with engine.connect() as conn:
            from sqlalchemy import text
            conn.execute(text("SELECT 1"))
    except Exception:
        db_ok = False

    return {
        "status": "ok" if db_ok else "degraded",
        "database": "up" if db_ok else "down",
        "version": settings.app_version,
    }
```

Листинг B.17 — backend/deps.py

```
"""Зависимости FastAPI: сессия БД, текущий пользователь."""

from __future__ import annotations

from typing import Iterator

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session, sessionmaker

from email_analyzer.db.models import Role, User
from email_analyzer.db.session import get_engine
from email_analyzer.utils.security import decode_access_token


oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")


def get_db() -> Iterator[Session]:
    engine = get_engine()
    factory = sessionmaker(bind=engine, autoflush=False, autocommit=False)
    db = factory()
    try:
        yield db
    finally:
        db.close()


def current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) -> User:
    payload = decode_access_token(token)
    if payload is None or "sub" not in payload:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="invalid or expired token",
        )
    user_id = int(payload["sub"])
```

```
    user = db.get(User, user_id)
    if user is None or not user.is_active:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="user not found or inactive",
        )
    return user


def require_admin(user: User = Depends(current_user)) -> User:
    if user.role is not Role.ADMIN:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="admin role required",
        )
    return user
```

Листинг B.18 — backend/schemas.py

```
"""Pydantic-схемы для HTTP-API."""

from __future__ import annotations

from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ConfigDict, EmailStr, Field

from email_analyzer.db.models import Category, Role


class UserCreate(BaseModel):
    username: str = Field(min_length=3, max_length=64)
    email: EmailStr
    password: str = Field(min_length=6, max_length=128)
    full_name: Optional[str] = None
    role: Role = Role.USER


class UserPublic(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    username: str
    email: str
    full_name: Optional[str] = None
    role: Role
    is_active: bool


class LoginRequest(BaseModel):
    username: str
    password: str


class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"
```

```
class EmailIn(BaseModel):
    subject: str = ""
    body: str = ""
    sender: str = ""
    recipient: str = ""


class AnalyzeResponse(BaseModel):
    category: Category
    confidence: float
    is_spam: bool
    is_phishing: bool
    spam_score: float
    summary: str
    entities: dict[str, list[str]]


class EmailPublic(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    subject: str
    sender: str
    recipient: str
    sent_at: Optional[datetime]
    is_read: bool
```

Листинг B.19 — backend/services/analysis.py

```
"""Высокоуровневый сервис анализа писем.

Оркестрирует все ML-компоненты:
    - классификатор (TF-IDF / LogReg)
    - детектор спама и фишинга
    - суммаризатор
    - извлекатель сущностей

Работает в pure-функциональном стиле — не хранит состояние между
письмами, кроме обученной модели классификатора.
"""

from __future__ import annotations

import json
from dataclasses import dataclass
from typing import Optional

from email_analyzer.db.models import Category, Classification
from email_analyzer.ml import (
    EmailClassifier,
    EntityExtractor,
    SpamDetector,
    Summarizer,
)
```

```
@dataclass(frozen=True, slots=True)
class EmailAnalysis:
    category: Category
    confidence: float
    is_spam: bool
    is_phishing: bool
    spam_score: float
    summary: str
    entities: dict[str, list[str]]


class AnalysisService:
    def __init__(
        self,
        classifier: Optional[EmailClassifier] = None,
        spam_detector: Optional[SpamDetector] = None,
        summarizer: Optional[Summarizer] = None,
        entity_extractor: Optional[EntityExtractor] = None,
    ) -> None:
        self.classifier = classifier or EmailClassifier()
        if not self.classifier.is_fitted():
            self.classifier.fit_seed()
        self.spam_detector = spam_detector or SpamDetector()
        self.summarizer = summarizer or Summarizer(max_sentences=3)
        self.entity_extractor = entity_extractor or EntityExtractor()

    def analyze(
        self,
        subject: str,
        body: str,
        sender: str = "",
    ) -> EmailAnalysis:
        text = f"{subject}\n{body}"

        cls_result = self.classifier.predict(text)
        spam_result = self.spam_detector.detect(subject, body, sender=sender)

        category = cls_result.category
        # Если спам-детектор сработал уверенно — доверяем ему
        if spam_result.is_phishing and category != Category.PHISHING:
            category = Category.PHISHING
        elif spam_result.is_spam and category not in (Category.SPAM,
Category.PHISHING):
            category = Category.SPAM

        summary = self.summarizer.summarize(body)
        entities = self.entity_extractor.extract(body).to_dict()

        return EmailAnalysis(
            category=category,
            confidence=cls_result.confidence,
            is_spam=spam_result.is_spam,
            is_phishing=spam_result.is_phishing,
            spam_score=spam_result.score,
            summary=summary,
            entities=entities,
        )

    def to_db_model(self, analysis: EmailAnalysis, email_id: int) ->
```

```
Classification:
        return Classification(
            email_id=email_id,
            category=analysis.category,
            confidence=analysis.confidence,
            is_spam=analysis.is_spam,
            is_phishing=analysis.is_phishing,
            summary=analysis.summary,
            entities_json=json.dumps(analysis.entities, ensure_ascii=False),
        )
```

Листинг B.20 — backend/services/users.py

```
"""Сервис управления пользователями (регистрация, логин, админка)."""

from __future__ import annotations

from typing import Optional

from sqlalchemy import select
from sqlalchemy.orm import Session

from email_analyzer.db.models import Role, User
from email_analyzer.utils.security import (
    create_access_token,
    hash_password,
    verify_password,
)


class UserService:
    def __init__(self, session: Session) -> None:
        self.session = session

    # ---------- lookup ----------

    def get_by_id(self, user_id: int) -> Optional[User]:
        return self.session.get(User, user_id)

    def get_by_username(self, username: str) -> Optional[User]:
        stmt = select(User).where(User.username == username)
        return self.session.scalar(stmt)

    def list_all(self) -> list[User]:
        return list(self.session.scalars(select(User)))

    # ---------- mutation ----------

    def create(
        self,
        username: str,
        email: str,
        password: str,
        full_name: str = "",
        role: Role = Role.USER,
    ) -> User:
        if self.get_by_username(username) is not None:
            raise ValueError(f"user {username!r} already exists")
```

```
        user = User(
            username=username,
            email=email,
            password_hash=hash_password(password),
            full_name=full_name or None,
            role=role,
        )
        self.session.add(user)
        self.session.flush()
        return user

    def set_role(self, user_id: int, role: Role) -> User:
        user = self.get_by_id(user_id)
        if user is None:
            raise ValueError(f"user {user_id} not found")
        user.role = role
        self.session.flush()
        return user

    def deactivate(self, user_id: int) -> User:
        user = self.get_by_id(user_id)
        if user is None:
            raise ValueError(f"user {user_id} not found")
        user.is_active = False
        self.session.flush()
        return user

    # ---------- auth ----------

    def authenticate(self, username: str, password: str) -> Optional[User]:
        user = self.get_by_username(username)
        if user is None or not user.is_active:
            return None
        if not verify_password(password, user.password_hash):
            return None
        return user

    def issue_token(self, user: User) -> str:
        return create_access_token(
            subject=str(user.id),
            extra={"username": user.username, "role": user.role.value},
        )
```

Листинг B.21 — backend/services/rules_engine.py

```
"""Движок правил автоклассификации писем.

Применяет пользовательские regex-правила к теме/телу/отправителю и
возвращает сработавшие действия. Независим от ML — пересекается в
`AnalysisService.analyze()` как override.
"""

from __future__ import annotations

import re
from dataclasses import dataclass

from email_analyzer.db.models import Category, Rule
```

```
@dataclass(frozen=True, slots=True)
class RuleMatch:
    rule_name: str
    action_category: Category | None
    notify: bool


class RulesEngine:
    def __init__(self, rules: list[Rule]) -> None:
        self._compiled = [
            (rule, re.compile(rule.pattern, re.IGNORECASE))
            for rule in rules
        ]

    def apply(self, subject: str, body: str, sender: str) -> list[RuleMatch]:
        fields = {"subject": subject, "body": body, "sender": sender}
        matched: list[RuleMatch] = []
        for rule, pattern in self._compiled:
            target = fields.get(rule.field, "")
            if pattern.search(target):
                matched.append(RuleMatch(
                    rule_name=rule.name,
                    action_category=rule.action_category,
                    notify=rule.notify,
                ))
        return matched
```

Листинг B.22 — backend/services/stats.py

```
"""Статистика по почте пользователя."""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Iterable

from email_analyzer.db.models import Category


@dataclass(frozen=True, slots=True)
class MailStats:
    total: int
    by_category: dict[str, int]
    spam_count: int
    phishing_count: int
    avg_confidence: float


class StatsService:
    def compute(self, emails: Iterable[dict]) -> MailStats:
        emails_list = list(emails)
        total = len(emails_list)
        if total == 0:
            return MailStats(
                total=0, by_category={},
                spam_count=0, phishing_count=0, avg_confidence=0.0,
```

```
            )

        by_cat: dict[str, int] = {}
        spam = 0
        phishing = 0
        conf_sum = 0.0
        for email in emails_list:
            cat = str(email.get("category", Category.OTHER.value))
            by_cat[cat] = by_cat.get(cat, 0) + 1
            if email.get("is_spam"):
                spam += 1
            if email.get("is_phishing"):
                phishing += 1
            conf_sum += float(email.get("confidence", 0.0))

        return MailStats(
            total=total,
            by_category=by_cat,
            spam_count=spam,
            phishing_count=phishing,
            avg_confidence=conf_sum / total,
        )
```

Листинг B.23 — backend/api/auth.py

```
"""Роутер аутентификации."""

from __future__ import annotations

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from email_analyzer.backend.deps import current_user, get_db
from email_analyzer.backend.schemas import LoginRequest, Token, UserCreate,
UserPublic
from email_analyzer.backend.services.users import UserService
from email_analyzer.db.models import User


router = APIRouter(prefix="/auth", tags=["auth"])


@router.post("/register", response_model=UserPublic,
status_code=status.HTTP_201_CREATED)
def register(payload: UserCreate, db: Session = Depends(get_db)) -> User:
    service = UserService(db)
    try:
        user = service.create(
            username=payload.username,
            email=payload.email,
            password=payload.password,
            full_name=payload.full_name or "",
            role=payload.role,
        )
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    db.commit()
    db.refresh(user)
```

```
    return user


@router.post("/login", response_model=Token)
def login(payload: LoginRequest, db: Session = Depends(get_db)) -> Token:
    service = UserService(db)
    user = service.authenticate(payload.username, payload.password)
    if user is None:
        raise HTTPException(status_code=401, detail="invalid credentials")
    token = service.issue_token(user)
    return Token(access_token=token)


@router.get("/me", response_model=UserPublic)
def me(user: User = Depends(current_user)) -> User:
    return user
```

Листинг B.24 — backend/api/analysis.py

```
"""Роутер анализа писем."""

from __future__ import annotations

from fastapi import APIRouter, Depends

from email_analyzer.backend.deps import current_user
from email_analyzer.backend.schemas import AnalyzeResponse, EmailIn
from email_analyzer.backend.services.analysis import AnalysisService
from email_analyzer.db.models import User


router = APIRouter(prefix="/analyze", tags=["analysis"])


_shared_service: AnalysisService | None = None


def get_analysis_service() -> AnalysisService:
    global _shared_service
    if _shared_service is None:
        _shared_service = AnalysisService()
    return _shared_service


@router.post("/email", response_model=AnalyzeResponse)
def analyze_email(
    payload: EmailIn,
    _: User = Depends(current_user),
    service: AnalysisService = Depends(get_analysis_service),
) -> AnalyzeResponse:
    result = service.analyze(
        subject=payload.subject,
        body=payload.body,
        sender=payload.sender,
    )
    return AnalyzeResponse(
        category=result.category,
        confidence=result.confidence,
```

```
        is_spam=result.is_spam,
        is_phishing=result.is_phishing,
        spam_score=result.spam_score,
        summary=result.summary,
        entities=result.entities,
    )
```

Листинг B.25 — backend/api/admin.py

```
"""Админский роутер (только для роли admin)."""

from __future__ import annotations

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from email_analyzer.backend.deps import get_db, require_admin
from email_analyzer.backend.schemas import UserPublic
from email_analyzer.backend.services.users import UserService
from email_analyzer.db.models import Role, User


router = APIRouter(prefix="/admin", tags=["admin"])


@router.get("/users", response_model=list[UserPublic])
def list_users(
    _: User = Depends(require_admin),
    db: Session = Depends(get_db),
) -> list[User]:
    return UserService(db).list_all()


@router.post("/users/{user_id}/role", response_model=UserPublic)
def change_role(
    user_id: int,
    role: Role,
    _: User = Depends(require_admin),
    db: Session = Depends(get_db),
) -> User:
    try:
        user = UserService(db).set_role(user_id, role)
    except ValueError as exc:
        raise HTTPException(status_code=404, detail=str(exc)) from exc
    db.commit()
    db.refresh(user)
    return user


@router.post("/users/{user_id}/deactivate", response_model=UserPublic)
def deactivate_user(
    user_id: int,
    _: User = Depends(require_admin),
    db: Session = Depends(get_db),
) -> User:
    try:
        user = UserService(db).deactivate(user_id)
    except ValueError as exc:
```

```
        raise HTTPException(status_code=404, detail=str(exc)) from exc
    db.commit()
    db.refresh(user)
    return user
```

Листинг B.26 — backend/api/stats.py

```
"""Роутер /stats."""

from __future__ import annotations

from fastapi import APIRouter, Depends

from email_analyzer.backend.deps import current_user
from email_analyzer.backend.services.stats import StatsService
from email_analyzer.db.models import User


router = APIRouter(prefix="/stats", tags=["stats"])


@router.post("/compute")
def compute_stats(
    emails: list[dict],
    _: User = Depends(current_user),
) -> dict:
    result = StatsService().compute(emails)
    return {
        "total": result.total,
        "by_category": result.by_category,
        "spam_count": result.spam_count,
        "phishing_count": result.phishing_count,
        "avg_confidence": result.avg_confidence,
    }
```

Листинг B.27 — utils/security.py

```
"""Хеширование паролей (bcrypt) и JWT-токены.

Используется библиотека `bcrypt` напрямую, без passlib: это избавляет
от известных несовместимостей между passlib и bcrypt ≥ 4.x.
"""

from __future__ import annotations

from datetime import datetime, timedelta, timezone
from typing import Any, Optional

import bcrypt
from jose import JWTError, jwt

from email_analyzer.config import get_settings


_ALGORITHM = "HS256"
_MAX_PASSWORD_BYTES = 72  # bcrypt ограничение
```

```
def _encode_password(plain: str) -> bytes:
    """Закодировать пароль в UTF-8 и обрезать до 72 байт (предел bcrypt)."""
    return plain.encode("utf-8")[:_MAX_PASSWORD_BYTES]


def hash_password(plain: str) -> str:
    hashed = bcrypt.hashpw(_encode_password(plain), bcrypt.gensalt())
    return hashed.decode("ascii")


def verify_password(plain: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(_encode_password(plain), hashed.encode("ascii"))
    except (ValueError, TypeError):
        return False


def create_access_token(
    subject: str,
    extra: Optional[dict[str, Any]] = None,
    ttl_minutes: Optional[int] = None,
) -> str:
    settings = get_settings()
    payload: dict[str, Any] = {"sub": subject}
    if extra:
        payload.update(extra)
    minutes = ttl_minutes if ttl_minutes is not None else
settings.access_token_ttl_minutes
    payload["exp"] = datetime.now(timezone.utc) + timedelta(minutes=minutes)
    return jwt.encode(payload, settings.secret_key, algorithm=_ALGORITHM)


def decode_access_token(token: str) -> Optional[dict[str, Any]]:
    settings = get_settings()
    try:
        return jwt.decode(token, settings.secret_key, algorithms=[_ALGORITHM])
    except JWTError:
        return None
```

Листинг B.28 — utils/reports.py

```
"""Экспорт результатов анализа в .docx и .xlsx.

Требование методички: «Реализовать возможность формирования документов
в форматах .docx, .xlsx и т.п. из разрабатываемого приложения.»
"""

from __future__ import annotations

from pathlib import Path
from typing import Iterable

from docx import Document
from openpyxl import Workbook


def export_emails_to_docx(
```

```
    emails: Iterable[dict],
    output_path: Path,
    title: str = "Отчёт об анализе почтовых сообщений",
) -> Path:
    """Сформировать .docx со списком писем и их классификацией.

    Каждый элемент `emails` — словарь с ключами: subject, sender, sent_at,
    category, is_spam, summary.
    """
    doc = Document()
    doc.add_heading(title, level=0)

    table = doc.add_table(rows=1, cols=5)
    table.style = "Light List Accent 1"
    header_cells = table.rows[0].cells
    header_cells[0].text = "От"
    header_cells[1].text = "Тема"
    header_cells[2].text = "Дата"
    header_cells[3].text = "Категория"
    header_cells[4].text = "Спам?"

    count = 0
    for email in emails:
        cells = table.add_row().cells
        cells[0].text = str(email.get("sender", ""))
        cells[1].text = str(email.get("subject", ""))
        cells[2].text = str(email.get("sent_at", ""))
        cells[3].text = str(email.get("category", ""))
        cells[4].text = "да" if email.get("is_spam") else "нет"
        count += 1

    doc.add_paragraph()
    doc.add_paragraph(f"Всего писем в отчёте: {count}")

    output_path.parent.mkdir(parents=True, exist_ok=True)
    doc.save(str(output_path))
    return output_path


def export_emails_to_xlsx(
    emails: Iterable[dict],
    output_path: Path,
    sheet_title: str = "Emails",
) -> Path:
    wb = Workbook()
    ws = wb.active
    if ws is None:
        raise RuntimeError("failed to create xlsx worksheet")
    ws.title = sheet_title

    headers = [
        "От", "Кому", "Тема", "Дата",
        "Категория", "Уверенность", "Спам", "Фишинг",
    ]
    ws.append(headers)

    for email in emails:
        ws.append([
            email.get("sender", ""),
```

```
            email.get("recipient", ""),
            email.get("subject", ""),
            str(email.get("sent_at", "")),
            email.get("category", ""),
            email.get("confidence", ""),
            "да" if email.get("is_spam") else "нет",
            "да" if email.get("is_phishing") else "нет",
        ])

    output_path.parent.mkdir(parents=True, exist_ok=True)
    wb.save(str(output_path))
    return output_path
```

Листинг B.29 — utils/csv_export.py

```
"""CSV-экспорт списка писем."""

from __future__ import annotations

import csv
from pathlib import Path
from typing import Iterable


CSV_HEADERS = [
    "sender", "recipient", "subject", "sent_at",
    "category", "confidence", "is_spam", "is_phishing",
]


def export_emails_to_csv(emails: Iterable[dict], output_path: Path) -> Path:
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with output_path.open("w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(
            f,
            fieldnames=CSV_HEADERS,
            extrasaction="ignore",
        )
        writer.writeheader()
        for email in emails:
            writer.writerow({k: email.get(k, "") for k in CSV_HEADERS})
    return output_path
```

Листинг B.30 — utils/json_export.py

```
"""JSON-экспорт списка писем (для интеграций)."""

from __future__ import annotations

import json
from datetime import datetime
from pathlib import Path
from typing import Any, Iterable


def _default(o: Any) -> Any:
    if isinstance(o, datetime):
```

```
        return o.isoformat()
    if hasattr(o, "value"):  # enums
        return o.value
    raise TypeError(f"Object of type {type(o).__name__} is not serializable")


def export_emails_to_json(
    emails: Iterable[dict],
    output_path: Path,
    pretty: bool = True,
) -> Path:
    output_path.parent.mkdir(parents=True, exist_ok=True)
    data = list(emails)
    with output_path.open("w", encoding="utf-8") as f:
        json.dump(
            data,
            f,
            ensure_ascii=False,
            indent=2 if pretty else None,
            default=_default,
        )
    return output_path
```

Листинг B.31 — utils/logging_setup.py

```
"""Единая настройка логирования."""

from __future__ import annotations

import logging
import sys
from pathlib import Path
from typing import Optional


DEFAULT_FORMAT = (
    "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
)


def configure(
    level: int = logging.INFO,
    log_file: Optional[Path] = None,
    fmt: str = DEFAULT_FORMAT,
) -> logging.Logger:
    """Настроить root-логгер и, если указан файл, — файловый handler."""
    root = logging.getLogger()
    root.handlers.clear()
    root.setLevel(level)

    formatter = logging.Formatter(fmt)
    stream = logging.StreamHandler(sys.stdout)
    stream.setFormatter(formatter)
    root.addHandler(stream)

    if log_file is not None:
        log_file.parent.mkdir(parents=True, exist_ok=True)
        file_h = logging.FileHandler(log_file, encoding="utf-8")
```

```
        file_h.setFormatter(formatter)
        root.addHandler(file_h)

    return root


def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)
```

Листинг B.32 — client/main.py

```
"""Точка входа PyQt6-клиента.

Запуск: `python -m email_analyzer.client.main`
"""

from __future__ import annotations

import sys

from PyQt6.QtWidgets import QApplication

from email_analyzer.client.state import AppState
from email_analyzer.client.windows import LoginWindow, MainWindow


class ClientApp:
    """Корневое управление приложением: логин → главное окно."""

    def __init__(self, argv: list[str]) -> None:
        self.qt_app = QApplication(argv)
        self.state = AppState()
        self.login: LoginWindow | None = None
        self.main: MainWindow | None = None

    def run(self) -> int:
        self.login = LoginWindow(self.state)
        self.login.logged_in.connect(self._on_login)
        self.login.show()
        return getattr(self.qt_app, "exec")()

    def _on_login(self, _user: dict) -> None:
        assert self.login is not None
        self.main = MainWindow(self.state)
        self.main.show()
        self.login.close()


def main() -> int:
    return ClientApp(sys.argv).run()


if __name__ == "__main__":
    raise SystemExit(main())
```

Листинг B.33 — client/api_client.py

```python
"""HTTP-клиент для backend API.

Инкапсулирует работу с JWT-токеном и базовым URL. Все сетевые вызовы
собраны здесь, чтобы окна GUI не знали про `requests`.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Optional

import requests


@dataclass
class ApiClient:
    base_url: str = "http://127.0.0.1:8000"
    token: Optional[str] = None
    timeout: float = 10.0
    session: requests.Session = field(default_factory=requests.Session)

    def _headers(self) -> dict[str, str]:
        headers = {"Accept": "application/json"}
        if self.token:
            headers["Authorization"] = f"Bearer {self.token}"
        return headers

    def _url(self, path: str) -> str:
        if path.startswith("http"):
            return path
        return f"{self.base_url.rstrip('/')}/{path.lstrip('/')}"

    def register(self, username: str, email: str, password: str, role: str =
"user") -> dict[str, Any]:
        r = self.session.post(
            self._url("/auth/register"),
            json={
                "username": username,
                "email": email,
                "password": password,
                "role": role,
            },
            timeout=self.timeout,
        )
        r.raise_for_status()
        return r.json()

    def login(self, username: str, password: str) -> str:
        r = self.session.post(
            self._url("/auth/login"),
            json={"username": username, "password": password},
            timeout=self.timeout,
        )
        r.raise_for_status()
        self.token = r.json()["access_token"]
        return self.token

    def me(self) -> dict[str, Any]:
        r = self.session.get(self._url("/auth/me"), headers=self._headers(),
```

```
timeout=self.timeout)
        r.raise_for_status()
        return r.json()

    def analyze(
        self,
        subject: str,
        body: str,
        sender: str = "",
        recipient: str = "",
    ) -> dict[str, Any]:
        r = self.session.post(
            self._url("/analyze/email"),
            json={
                "subject": subject,
                "body": body,
                "sender": sender,
                "recipient": recipient,
            },
            headers=self._headers(),
            timeout=self.timeout,
        )
        r.raise_for_status()
        return r.json()

    def list_users(self) -> list[dict[str, Any]]:
        r = self.session.get(self._url("/admin/users"),
headers=self._headers(), timeout=self.timeout)
        r.raise_for_status()
        return r.json()
```

Листинг B.34 — client/state.py

```
"""Разделяемое состояние клиента (очень простое, без Redux-подобных
паттернов)."""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Optional

from email_analyzer.client.api_client import ApiClient


@dataclass
class AppState:
    api: ApiClient = field(default_factory=ApiClient)
    current_user: Optional[dict[str, Any]] = None

    @property
    def is_admin(self) -> bool:
        return bool(self.current_user and self.current_user.get("role") ==
"admin")

    @property
    def username(self) -> str:
        if not self.current_user:
            return "(не авторизован)"
```

```
        return str(self.current_user.get("username", ""))
```

Листинг B.35 — client/windows/login_window.py

```
"""Окно авторизации."""

from __future__ import annotations

from PyQt6.QtCore import pyqtSignal
from PyQt6.QtWidgets import (
    QLabel,
    QLineEdit,
    QMessageBox,
    QPushButton,
    QVBoxLayout,
    QWidget,
)

from email_analyzer.client.state import AppState


class LoginWindow(QWidget):
    logged_in = pyqtSignal(dict)

    def __init__(self, state: AppState) -> None:
        super().__init__()
        self.state = state
        self.setWindowTitle("Email AI Analyzer — Вход")
        self.resize(360, 200)

        layout = QVBoxLayout(self)
        layout.addWidget(QLabel("Логин:"))
        self.username = QLineEdit()
        layout.addWidget(self.username)

        layout.addWidget(QLabel("Пароль:"))
        self.password = QLineEdit()
        self.password.setEchoMode(QLineEdit.EchoMode.Password)
        layout.addWidget(self.password)

        self.login_btn = QPushButton("Войти")
        self.login_btn.clicked.connect(self._on_login)
        layout.addWidget(self.login_btn)

        self.register_btn = QPushButton("Регистрация")
        self.register_btn.clicked.connect(self._on_register)
        layout.addWidget(self.register_btn)

    def _on_login(self) -> None:
        username = self.username.text().strip()
        password = self.password.text()
        if not username or not password:
            QMessageBox.warning(self, "Ошибка", "Заполните логин и пароль")
            return
        try:
            self.state.api.login(username, password)
            self.state.current_user = self.state.api.me()
        except Exception as exc:
```

```
            QMessageBox.critical(self, "Ошибка входа", str(exc))
            return
        self.logged_in.emit(self.state.current_user)

    def _on_register(self) -> None:
        username = self.username.text().strip()
        password = self.password.text()
        if not username or not password:
            QMessageBox.warning(self, "Ошибка", "Заполните логин и пароль")
            return
        try:
            self.state.api.register(
                username=username,
                email=f"{username}@local.test",
                password=password,
            )
            QMessageBox.information(self, "OK", "Пользователь создан.
Войдите.")
        except Exception as exc:
            QMessageBox.critical(self, "Ошибка регистрации", str(exc))
```

Листинг B.36 — client/windows/main_window.py

```
"""Главное окно приложения со списком писем и меню."""

from __future__ import annotations

from PyQt6.QtGui import QAction
from PyQt6.QtWidgets import (
    QDialog,
    QHeaderView,
    QMainWindow,
    QMenuBar,
    QMessageBox,
    QStatusBar,
    QTableWidget,
    QTableWidgetItem,
    QToolBar,
    QVBoxLayout,
    QWidget,
)

from email_analyzer.client.state import AppState
from email_analyzer.client.windows.about_window import AboutWindow
from email_analyzer.client.windows.admin_window import AdminWindow
from email_analyzer.client.windows.email_detail_window import
EmailDetailWindow
from email_analyzer.client.windows.help_window import HelpWindow
from email_analyzer.client.windows.import_dialog import ImportDialog
from email_analyzer.client.windows.profile_window import ProfileWindow
from email_analyzer.client.windows.reports_window import ReportsWindow
from email_analyzer.client.windows.rules_window import RulesWindow
from email_analyzer.client.windows.search_window import SearchWindow
from email_analyzer.client.windows.settings_window import SettingsWindow
from email_analyzer.client.windows.stats_window import StatsWindow


def _run_modal(dialog: QDialog) -> bool:
```

```
    """Показать диалог модально и вернуть True, если принят."""
    return getattr(dialog, "exec")() == QDialog.DialogCode.Accepted


class MainWindow(QMainWindow):
    def __init__(self, state: AppState) -> None:
        super().__init__()
        self.state = state
        self.setWindowTitle(f"Email AI Analyzer — {state.username}")
        self.resize(1024, 640)

        self._subwindows: list[QWidget] = []
        self._emails: list[dict] = []

        self._build_menu()
        self._build_toolbar()
        self._build_central()
        self._build_status()

    def _build_menu(self) -> None:
        menubar: QMenuBar = self.menuBar()

        file_menu = menubar.addMenu("Файл")
        file_menu.addAction(self._action("Импорт писем…", self._open_import))
        file_menu.addAction(self._action("Отчёты", self._open_reports))
        file_menu.addSeparator()
        file_menu.addAction(self._action("Выход", self.close))

        tools_menu = menubar.addMenu("Инструменты")
        tools_menu.addAction(self._action("Настройки", self._open_settings))
        tools_menu.addAction(self._action("Правила", self._open_rules))
        tools_menu.addAction(self._action("Поиск по смыслу",
self._open_search))
        tools_menu.addAction(self._action("Статистика", self._open_stats))

        user_menu = menubar.addMenu("Пользователь")
        user_menu.addAction(self._action("Личный кабинет",
self._open_profile))
        if self.state.is_admin:
            user_menu.addAction(self._action("Администрирование",
self._open_admin))

        help_menu = menubar.addMenu("Справка")
        help_menu.addAction(self._action("Помощь", self._open_help))
        help_menu.addAction(self._action("О программе", self._open_about))

    def _build_toolbar(self) -> None:
        bar = QToolBar("Основное")
        bar.addAction(self._action("Импорт", self._open_import))
        bar.addAction(self._action("Настройки", self._open_settings))
        bar.addAction(self._action("Отчёты", self._open_reports))
        self.addToolBar(bar)

    def _build_central(self) -> None:
        central = QWidget()
        layout = QVBoxLayout(central)

        self.table = QTableWidget(0, 5)
        self.table.setHorizontalHeaderLabels(
```

```
            ["От", "Тема", "Дата", "Категория", "Спам?"]
        )
        header = self.table.horizontalHeader()
        header.setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        self.table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
        self.table.cellDoubleClicked.connect(self._open_detail)
        layout.addWidget(self.table)

        self.setCentralWidget(central)

    def _build_status(self) -> None:
        bar = QStatusBar()
        bar.showMessage(f"Подключено: {self.state.api.base_url}")
        self.setStatusBar(bar)

    def _action(self, text: str, callback) -> QAction:
        a = QAction(text, self)
        a.triggered.connect(callback)
        return a

    def add_email(self, email: dict) -> None:
        self._emails.append(email)
        row = self.table.rowCount()
        self.table.insertRow(row)
        self.table.setItem(row, 0, QTableWidgetItem(email.get("sender", "")))
        self.table.setItem(row, 1, QTableWidgetItem(email.get("subject", "")))
        self.table.setItem(row, 2, QTableWidgetItem(str(email.get("sent_at",
""))))
        self.table.setItem(row, 3, QTableWidgetItem(email.get("category",
"")))
        self.table.setItem(row, 4, QTableWidgetItem("да" if
email.get("is_spam") else "нет"))

    def _open_child(self, window: QWidget) -> None:
        self._subwindows.append(window)
        window.show()

    def _open_import(self) -> None:
        dlg = ImportDialog(self.state, self)
        if _run_modal(dlg):
            for email in dlg.collected:
                self.add_email(email)

    def _open_settings(self) -> None:
        self._open_child(SettingsWindow(self.state))

    def _open_reports(self) -> None:
        self._open_child(ReportsWindow(self.state, self._emails))

    def _open_rules(self) -> None:
        self._open_child(RulesWindow(self.state))

    def _open_search(self) -> None:
        self._open_child(SearchWindow(self._emails))

    def _open_stats(self) -> None:
        self._open_child(StatsWindow(self._emails))

    def _open_profile(self) -> None:
```

```
        self._open_child(ProfileWindow(self.state))

    def _open_admin(self) -> None:
        if not self.state.is_admin:
            QMessageBox.warning(self, "Нет прав", "Нужна роль admin")
            return
        self._open_child(AdminWindow(self.state))

    def _open_help(self) -> None:
        self._open_child(HelpWindow())

    def _open_about(self) -> None:
        self._open_child(AboutWindow())

    def _open_detail(self, row: int, _column: int) -> None:
        if 0 <= row < len(self._emails):
            self._open_child(EmailDetailWindow(self._emails[row]))
```

Листинг B.37 — client/windows/email_detail_window.py

```
"""Окно просмотра письма с результатом анализа."""

from __future__ import annotations

import json

from PyQt6.QtWidgets import (
    QLabel,
    QPlainTextEdit,
    QVBoxLayout,
    QWidget,
)


class EmailDetailWindow(QWidget):
    def __init__(self, email: dict) -> None:
        super().__init__()
        self.setWindowTitle(f"Письмо: {email.get('subject', '')}")
        self.resize(680, 520)

        layout = QVBoxLayout(self)
        layout.addWidget(QLabel(f"<b>От:</b> {email.get('sender', '')}"))
        layout.addWidget(QLabel(f"<b>Кому:</b> {email.get('recipient', '')}"))
        layout.addWidget(QLabel(f"<b>Тема:</b> {email.get('subject', '')}"))
        layout.addWidget(QLabel(f"<b>Категория:</b> {email.get('category',
'')}"))
        layout.addWidget(QLabel(
            f"<b>Уверенность:</b> {email.get('confidence', 0):.2f}; "
            f"<b>Спам:</b> {'да' if email.get('is_spam') else 'нет'}; "
            f"<b>Фишинг:</b> {'да' if email.get('is_phishing') else 'нет'}"
        ))

        layout.addWidget(QLabel("<b>Саммари:</b>"))
        summary = QPlainTextEdit(email.get("summary", ""))
        summary.setReadOnly(True)
        summary.setMaximumHeight(120)
        layout.addWidget(summary)
```

```
        layout.addWidget(QLabel("<b>Извлечённые сущности:</b>"))
        entities = QPlainTextEdit(
            json.dumps(email.get("entities", {}), ensure_ascii=False,
indent=2)
        )
        entities.setReadOnly(True)
        entities.setMaximumHeight(160)
        layout.addWidget(entities)

        layout.addWidget(QLabel("<b>Тело письма:</b>"))
        body = QPlainTextEdit(email.get("body", ""))
        body.setReadOnly(True)
        layout.addWidget(body)
```

Листинг B.38 — client/windows/settings_window.py

```
"""Окно настроек — ≥ 5 пунктов меню (IMAP, ML, категории, уведомления,
экспорт)."""

from __future__ import annotations

from PyQt6.QtWidgets import (
    QCheckBox,
    QComboBox,
    QFormLayout,
    QLabel,
    QLineEdit,
    QListWidget,
    QPushButton,
    QSpinBox,
    QTabWidget,
    QVBoxLayout,
    QWidget,
)

from email_analyzer.client.state import AppState


class SettingsWindow(QWidget):
    def __init__(self, state: AppState) -> None:
        super().__init__()
        self.state = state
        self.setWindowTitle("Настройки")
        self.resize(560, 440)

        layout = QVBoxLayout(self)
        tabs = QTabWidget()

        tabs.addTab(self._imap_tab(), "Почта (IMAP)")
        tabs.addTab(self._ml_tab(), "ML-модель")
        tabs.addTab(self._categories_tab(), "Категории")
        tabs.addTab(self._notifications_tab(), "Уведомления")
        tabs.addTab(self._export_tab(), "Экспорт")

        layout.addWidget(tabs)

        buttons = QWidget()
        buttons_layout = QVBoxLayout(buttons)
```

```
        save_btn = QPushButton("Сохранить")
        save_btn.clicked.connect(self.close)
        buttons_layout.addWidget(save_btn)
        layout.addWidget(buttons)

    def _imap_tab(self) -> QWidget:
        w = QWidget()
        form = QFormLayout(w)
        form.addRow("Сервер:", QLineEdit("imap.gmail.com"))
        form.addRow("Порт:", QSpinBox())
        form.addRow("Логин:", QLineEdit())
        form.addRow("Пароль:", QLineEdit())
        ssl = QCheckBox("Использовать SSL")
        ssl.setChecked(True)
        form.addRow(ssl)
        return w

    def _ml_tab(self) -> QWidget:
        w = QWidget()
        form = QFormLayout(w)
        engine = QComboBox()
        engine.addItems(["Локальная (TF-IDF + LogReg)", "OpenAI GPT", "Yandex
GPT", "GigaChat"])
        form.addRow("Движок:", engine)
        form.addRow("Порог спама:", QLineEdit("0.5"))
        form.addRow("Макс. длина саммари:", QSpinBox())
        return w

    def _categories_tab(self) -> QWidget:
        w = QWidget()
        layout = QVBoxLayout(w)
        layout.addWidget(QLabel("Категории классификации:"))
        items = QListWidget()
        items.addItems([
            "Работа", "Личное", "Реклама", "Спам",
            "Фишинг", "Важное", "Другое",
        ])
        layout.addWidget(items)
        return w

    def _notifications_tab(self) -> QWidget:
        w = QWidget()
        form = QFormLayout(w)
        form.addRow(QCheckBox("Уведомлять о письмах категории «важное»"))
        form.addRow(QCheckBox("Уведомлять о фишинге"))
        form.addRow(QCheckBox("Звуковой сигнал"))
        form.addRow("Интервал опроса, минут:", QSpinBox())
        return w

    def _export_tab(self) -> QWidget:
        w = QWidget()
        form = QFormLayout(w)
        form.addRow(QCheckBox("Автосохранение отчётов в .xlsx"))
        form.addRow(QCheckBox("Автосохранение отчётов в .docx"))
        form.addRow("Папка экспорта:", QLineEdit("./reports"))
        return w
```

Листинг B.39 — client/windows/admin_window.py

```python
"""Панель администратора."""

from __future__ import annotations

from PyQt6.QtWidgets import (
    QHeaderView,
    QLabel,
    QMessageBox,
    QPushButton,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWidget,
)

from email_analyzer.client.state import AppState


class AdminWindow(QWidget):
    def __init__(self, state: AppState) -> None:
        super().__init__()
        self.state = state
        self.setWindowTitle("Администрирование")
        self.resize(720, 480)

        layout = QVBoxLayout(self)
        layout.addWidget(QLabel("<h2>Пользователи системы</h2>"))

        self.table = QTableWidget(0, 5)
        self.table.setHorizontalHeaderLabels(
            ["ID", "Логин", "Email", "Роль", "Активен"]
        )

self.table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stre
tch)
        layout.addWidget(self.table)

        refresh = QPushButton("Обновить список")
        refresh.clicked.connect(self._refresh)
        layout.addWidget(refresh)

        self._refresh()

    def _refresh(self) -> None:
        try:
            users = self.state.api.list_users()
        except Exception as exc:
            QMessageBox.critical(self, "Ошибка", str(exc))
            return
        self.table.setRowCount(0)
        for user in users:
            row = self.table.rowCount()
            self.table.insertRow(row)
            self.table.setItem(row, 0, QTableWidgetItem(str(user["id"])))
            self.table.setItem(row, 1, QTableWidgetItem(user["username"]))
            self.table.setItem(row, 2, QTableWidgetItem(user["email"]))
            self.table.setItem(row, 3, QTableWidgetItem(user["role"]))
            self.table.setItem(row, 4, QTableWidgetItem("да" if
```

```
user["is_active"] else "нет"))
```

Листинг B.40 — client/windows/profile_window.py

```
"""Личный кабинет пользователя."""

from __future__ import annotations

from PyQt6.QtWidgets import QFormLayout, QLabel, QLineEdit, QVBoxLayout,
QWidget

from email_analyzer.client.state import AppState


class ProfileWindow(QWidget):
    def __init__(self, state: AppState) -> None:
        super().__init__()
        self.state = state
        self.setWindowTitle("Личный кабинет")
        self.resize(480, 320)

        layout = QVBoxLayout(self)
        layout.addWidget(QLabel("<h2>Личный кабинет</h2>"))

        form = QFormLayout()
        user = state.current_user or {}
        form.addRow("ID:", QLineEdit(str(user.get("id", ""))))
        form.addRow("Логин:", QLineEdit(user.get("username", "")))
        form.addRow("Email:", QLineEdit(user.get("email", "")))
        form.addRow("ФИО:", QLineEdit(user.get("full_name") or ""))
        form.addRow("Роль:", QLineEdit(user.get("role", "")))
        form.addRow("Активен:", QLineEdit("да" if user.get("is_active") else
"нет"))

        wrapper = QWidget()
        wrapper.setLayout(form)
        layout.addWidget(wrapper)
```

Листинг B.41 — client/windows/rules_window.py

```
"""Окно управления правилами автообработки писем."""

from __future__ import annotations

from PyQt6.QtWidgets import (
    QDialog,
    QHBoxLayout,
    QHeaderView,
    QPushButton,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWidget,
)

from email_analyzer.client.state import AppState
from email_analyzer.client.windows.add_rule_dialog import AddRuleDialog
```

```
def _open_modal(dialog: QDialog) -> bool:
    return getattr(dialog, "exec")() == QDialog.DialogCode.Accepted


class RulesWindow(QWidget):
    def __init__(self, state: AppState) -> None:
        super().__init__()
        self.state = state
        self.setWindowTitle("Правила")
        self.resize(640, 440)

        layout = QVBoxLayout(self)

        self.table = QTableWidget(0, 4)
        self.table.setHorizontalHeaderLabels(["Имя", "Поле", "Шаблон",
"Действие"])

self.table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stre
tch)
        layout.addWidget(self.table)

        buttons = QHBoxLayout()
        add = QPushButton("Добавить правило")
        add.clicked.connect(self._add_rule)
        remove = QPushButton("Удалить выбранное")
        remove.clicked.connect(self._remove_rule)
        buttons.addWidget(add)
        buttons.addWidget(remove)
        layout.addLayout(buttons)

    def _add_rule(self) -> None:
        dlg = AddRuleDialog(self)
        if _open_modal(dlg):
            rule = dlg.rule
            row = self.table.rowCount()
            self.table.insertRow(row)
            self.table.setItem(row, 0, QTableWidgetItem(rule["name"]))
            self.table.setItem(row, 1, QTableWidgetItem(rule["field"]))
            self.table.setItem(row, 2, QTableWidgetItem(rule["pattern"]))
            self.table.setItem(row, 3, QTableWidgetItem(rule["action"]))

    def _remove_rule(self) -> None:
        row = self.table.currentRow()
        if row >= 0:
            self.table.removeRow(row)
```

Листинг B.42 — client/windows/reports_window.py

```
"""Окно экспорта отчётов в .docx/.xlsx (требование методички)."""

from __future__ import annotations

from pathlib import Path

from PyQt6.QtWidgets import (
    QFileDialog,
```

```
    QLabel,
    QMessageBox,
    QPushButton,
    QVBoxLayout,
    QWidget,
)

from email_analyzer.client.state import AppState
from email_analyzer.utils.reports import (
    export_emails_to_docx,
    export_emails_to_xlsx,
)


class ReportsWindow(QWidget):
    def __init__(self, state: AppState, emails: list[dict]) -> None:
        super().__init__()
        self.state = state
        self._emails = emails
        self.setWindowTitle("Отчёты")
        self.resize(480, 260)

        layout = QVBoxLayout(self)
        layout.addWidget(QLabel("<h2>Экспорт отчёта</h2>"))
        layout.addWidget(QLabel(f"Писем в выгрузке: {len(emails)}"))

        docx_btn = QPushButton("Сохранить как .docx")
        docx_btn.clicked.connect(self._save_docx)
        layout.addWidget(docx_btn)

        xlsx_btn = QPushButton("Сохранить как .xlsx")
        xlsx_btn.clicked.connect(self._save_xlsx)
        layout.addWidget(xlsx_btn)

    def _save_docx(self) -> None:
        path, _ = QFileDialog.getSaveFileName(
            self, "Сохранить .docx", "report.docx", "Word (*.docx)"
        )
        if not path:
            return
        try:
            export_emails_to_docx(self._emails, Path(path))
        except Exception as exc:
            QMessageBox.critical(self, "Ошибка", str(exc))
            return
        QMessageBox.information(self, "Готово", f"Сохранено: {path}")

    def _save_xlsx(self) -> None:
        path, _ = QFileDialog.getSaveFileName(
            self, "Сохранить .xlsx", "report.xlsx", "Excel (*.xlsx)"
        )
        if not path:
            return
        try:
            export_emails_to_xlsx(self._emails, Path(path))
        except Exception as exc:
            QMessageBox.critical(self, "Ошибка", str(exc))
            return
```

```
        QMessageBox.information(self, "Готово", f"Сохранено: {path}")
```

Листинг B.43 — client/windows/help_window.py

```
"""Окно «Справка по системе» (требование методички)."""

from __future__ import annotations

from PyQt6.QtWidgets import QTextBrowser, QVBoxLayout, QWidget


HELP_TEXT = """
<h1>Справка по системе Email AI Analyzer</h1>

<p><b>Автор:</b> Нефедов Алексей Геннадьевич (№ студ.билета 70200291)<br>
<b>Направление:</b> 09.03.03 Прикладная информатика, профиль<br>
   «Искусственный интеллект и анализ данных»<br>
<b>ВУЗ:</b> ЧОУВО «Московский университет им. С.Ю. Витте»<br>
<b>Тема ВКР:</b> «Разработка AI-приложения для анализа<br>
   почтовых сообщений»<br>
<b>Руководитель:</b> Коротков Дмитрий Павлович<br>
<b>Репозиторий:</b> <a href="https://github.com/ngpless/email-ai-
analyzer">github.com/ngpless/email-ai-analyzer</a></p>

<hr>

<h2>Основные окна</h2>
<ul>
    <li><b>Главное окно</b> — список импортированных писем с колонками
        «От», «Тема», «Дата», «Категория», «Спам?».</li>
    <li><b>Импорт писем</b> — диалог подключения к IMAP-серверу и загрузки
        последних писем.</li>
    <li><b>Письмо</b> — двойной клик по строке в таблице открывает окно
        с саммари, сущностями и телом письма.</li>
    <li><b>Настройки</b> — пять вкладок: IMAP, ML-движок, категории,
        уведомления, экспорт.</li>
    <li><b>Правила</b> — пользовательские правила автоклассификации
        (например: «письма от шефа — важно»).</li>
    <li><b>Отчёты</b> — экспорт в .docx и .xlsx.</li>
    <li><b>Личный кабинет</b> — профиль пользователя.</li>
    <li><b>Администрирование</b> (только для admin) — управление
        пользователями.</li>
    <li><b>Справка</b> — данная страница.</li>
    <li><b>О программе</b> — информация о версии и авторе.</li>
</ul>

<h2>Роли пользователей</h2>
<ul>
    <li><b>admin</b> — полный доступ + управление пользователями;</li>
    <li><b>analyst</b> — доступ к отчётам и анализу;</li>
    <li><b>user</b> — обычный пользователь.</li>
</ul>

<h2>AI-компоненты</h2>
<ul>
    <li><b>Классификатор</b> — TF-IDF + логистическая регрессия
(локально).</li>
    <li><b>Детектор спама/фишинга</b> — эвристический.</li>
```

```
    <li><b>Суммаризатор</b> — выбирает 2-3 самых информативных
предложения.</li>
    <li><b>Извлечение сущностей</b> — даты, суммы, почта, телефоны,
задачи.</li>
</ul>

<h2>Горячие клавиши</h2>
<ul>
    <li><code>Alt+F</code> — меню «Файл»</li>
    <li><code>Ctrl+I</code> — импорт писем</li>
    <li><code>Ctrl+,</code> — настройки</li>
    <li><code>F1</code> — справка</li>
</ul>
"""


class HelpWindow(QWidget):
    def __init__(self) -> None:
        super().__init__()
        self.setWindowTitle("Справка")
        self.resize(640, 520)
        layout = QVBoxLayout(self)
        browser = QTextBrowser()
        browser.setHtml(HELP_TEXT)
        layout.addWidget(browser)
```

Листинг B.44 — client/windows/about_window.py

```
"""«О программе»."""

from __future__ import annotations

from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import QLabel, QVBoxLayout, QWidget

from email_analyzer import __version__


ABOUT_TEXT = f"""
<h2>Email AI Analyzer</h2>
<p>Версия: <b>{__version__}</b></p>
<p>AI-приложение для анализа почтовых сообщений.</p>
<p>
    Работа выполнена в рамках <b>преддипломной практики и ВКР</b> в
    ЧОУВО «МУ им. С.Ю. Витте» (МУИВ).
</p>
<p>Направление: 09.03.03 Прикладная информатика<br>
Профиль: «Искусственный интеллект и анализ данных»</p>
<p>Автор: <b>Нефедов Алексей Геннадьевич</b><br>
№ студ.билета: 70200291</p>
<p>Руководитель: Коротков Дмитрий Павлович</p>
<p>Репозиторий: https://github.com/ngpless/email-ai-analyzer</p>
<p>2026 год.</p>
"""


class AboutWindow(QWidget):
    def __init__(self) -> None:
```

```
        super().__init__()
        self.setWindowTitle("О программе")
        self.resize(380, 260)

        layout = QVBoxLayout(self)
        label = QLabel(ABOUT_TEXT)
        label.setTextFormat(Qt.TextFormat.RichText)
        label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        label.setWordWrap(True)
        layout.addWidget(label)
```

Листинг B.45 — client/windows/stats_window.py

```
"""Окно статистики по импортированной почте."""

from __future__ import annotations

from PyQt6.QtWidgets import (
    QHeaderView,
    QLabel,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWidget,
)

from email_analyzer.backend.services.stats import StatsService


class StatsWindow(QWidget):
    def __init__(self, emails: list[dict]) -> None:
        super().__init__()
        self.setWindowTitle("Статистика")
        self.resize(520, 400)

        layout = QVBoxLayout(self)
        layout.addWidget(QLabel("<h2>Сводная статистика</h2>"))

        stats = StatsService().compute(emails)

        layout.addWidget(QLabel(f"<b>Всего писем:</b> {stats.total}"))
        layout.addWidget(QLabel(f"<b>Спам:</b> {stats.spam_count}"))
        layout.addWidget(QLabel(f"<b>Фишинг:</b> {stats.phishing_count}"))
        layout.addWidget(QLabel(f"<b>Средняя уверенность:</b>
{stats.avg_confidence:.2f}"))

        layout.addWidget(QLabel("<b>По категориям:</b>"))
        table = QTableWidget(0, 2)
        table.setHorizontalHeaderLabels(["Категория", "Количество"])

table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        for cat, count in sorted(stats.by_category.items(), key=lambda p: -
p[1]):
            row = table.rowCount()
            table.insertRow(row)
            table.setItem(row, 0, QTableWidgetItem(cat))
            table.setItem(row, 1, QTableWidgetItem(str(count)))
```

```
        layout.addWidget(table)
```

Листинг B.46 — client/windows/search_window.py

```
"""Окно семантического поиска по почте."""

from __future__ import annotations

from PyQt6.QtWidgets import (
    QHBoxLayout,
    QHeaderView,
    QLabel,
    QLineEdit,
    QPushButton,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWidget,
)

from email_analyzer.ml.semantic_search import SemanticSearch


class SearchWindow(QWidget):
    def __init__(self, emails: list[dict]) -> None:
        super().__init__()
        self._emails = emails
        self.setWindowTitle("Поиск по смыслу")
        self.resize(720, 480)

        self._search = SemanticSearch()
        corpus = [
            f"{e.get('subject', '')} {e.get('body', '')}"
            for e in emails
        ]
        if corpus:
            self._search.index(corpus)

        layout = QVBoxLayout(self)
        layout.addWidget(QLabel(f"Индексировано писем: {self._search.size}"))

        row = QHBoxLayout()
        self.query_input = QLineEdit()
        self.query_input.setPlaceholderText("Введите запрос…")
        find_btn = QPushButton("Искать")
        find_btn.clicked.connect(self._on_search)
        row.addWidget(self.query_input)
        row.addWidget(find_btn)
        layout.addLayout(row)

        self.table = QTableWidget(0, 3)
        self.table.setHorizontalHeaderLabels(["Score", "От", "Тема"])

self.table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stre
tch)
        layout.addWidget(self.table)

    def _on_search(self) -> None:
```

```
        query = self.query_input.text().strip()
        self.table.setRowCount(0)
        if not query or self._search.size == 0:
            return
        hits = self._search.query(query, top_k=10)
        for hit in hits:
            email = self._emails[hit.doc_index]
            row = self.table.rowCount()
            self.table.insertRow(row)
            self.table.setItem(row, 0, QTableWidgetItem(f"{hit.score:.3f}"))
            self.table.setItem(row, 1, QTableWidgetItem(email.get("sender",
""")))
            self.table.setItem(row, 2, QTableWidgetItem(email.get("subject",
""")))
```

Листинг B.47 — client/windows/add_rule_dialog.py

```
"""Диалог добавления правила — обязательное «диалоговое окно для
получения данных от пользователя со специальной формы» (методичка)."""

from __future__ import annotations

from PyQt6.QtWidgets import (
    QComboBox,
    QDialog,
    QDialogButtonBox,
    QFormLayout,
    QLineEdit,
    QMessageBox,
)


class AddRuleDialog(QDialog):
    def __init__(self, parent=None) -> None:
        super().__init__(parent)
        self.setWindowTitle("Новое правило")
        self.resize(420, 220)

        form = QFormLayout(self)

        self.name = QLineEdit()
        form.addRow("Имя правила:", self.name)

        self.field = QComboBox()
        self.field.addItems(["subject", "body", "sender"])
        form.addRow("Поле:", self.field)

        self.pattern = QLineEdit()
        form.addRow("Шаблон (regex):", self.pattern)

        self.action = QComboBox()
        self.action.addItems([
            "Пометить как работа",
            "Пометить как важное",
            "Пометить как спам",
            "Уведомить",
        ])
        form.addRow("Действие:", self.action)
```

```
        buttons = QDialogButtonBox(
            QDialogButtonBox.StandardButton.Ok |
QDialogButtonBox.StandardButton.Cancel
        )
        buttons.accepted.connect(self._accept)
        buttons.rejected.connect(self.reject)
        form.addRow(buttons)

    @property
    def rule(self) -> dict:
        return {
            "name": self.name.text().strip(),
            "field": self.field.currentText(),
            "pattern": self.pattern.text().strip(),
            "action": self.action.currentText(),
        }

    def _accept(self) -> None:
        if not self.name.text().strip() or not self.pattern.text().strip():
            QMessageBox.warning(self, "Проверка", "Заполните имя и шаблон")
            return
        self.accept()
```

Листинг B.48 — client/windows/import_dialog.py

```
"""Диалог импорта писем через IMAP + анализ каждого через API."""

from __future__ import annotations

from PyQt6.QtWidgets import (
    QCheckBox,
    QDialog,
    QDialogButtonBox,
    QFormLayout,
    QLineEdit,
    QMessageBox,
    QProgressBar,
    QSpinBox,
    QVBoxLayout,
)

from email_analyzer.client.state import AppState
from email_analyzer.mail import ImapAccount, ImapClient


class ImportDialog(QDialog):
    def __init__(self, state: AppState, parent=None) -> None:
        super().__init__(parent)
        self.state = state
        self.collected: list[dict] = []

        self.setWindowTitle("Импорт писем из почтового ящика")
        self.resize(460, 300)

        outer = QVBoxLayout(self)

        form = QFormLayout()
```

```
        self.host = QLineEdit("imap.gmail.com")
        form.addRow("IMAP-сервер:", self.host)
        self.port = QSpinBox()
        self.port.setRange(1, 65535)
        self.port.setValue(993)
        form.addRow("Порт:", self.port)
        self.login = QLineEdit()
        form.addRow("Логин:", self.login)
        self.password = QLineEdit()
        self.password.setEchoMode(QLineEdit.EchoMode.Password)
        form.addRow("Пароль:", self.password)
        self.limit = QSpinBox()
        self.limit.setRange(1, 500)
        self.limit.setValue(10)
        form.addRow("Сколько писем:", self.limit)
        self.use_ssl = QCheckBox("SSL")
        self.use_ssl.setChecked(True)
        form.addRow(self.use_ssl)
        outer.addLayout(form)

        self.progress = QProgressBar()
        outer.addWidget(self.progress)

        buttons = QDialogButtonBox(
            QDialogButtonBox.StandardButton.Ok |
QDialogButtonBox.StandardButton.Cancel
        )
        buttons.accepted.connect(self._do_import)
        buttons.rejected.connect(self.reject)
        outer.addWidget(buttons)

    def _do_import(self) -> None:
        account = ImapAccount(
            host=self.host.text().strip(),
            port=self.port.value(),
            username=self.login.text().strip(),
            password=self.password.text(),
            use_ssl=self.use_ssl.isChecked(),
        )
        try:
            with ImapClient(account) as client:
                fetched = client.fetch_recent(limit=self.limit.value())
        except Exception as exc:
            QMessageBox.critical(self, "Ошибка IMAP", str(exc))
            return

        total = len(fetched)
        self.progress.setMaximum(max(total, 1))
        for i, item in enumerate(fetched, 1):
            parsed = item.parsed
            try:
                analysis = self.state.api.analyze(
                    subject=parsed.subject,
                    body=parsed.body,
                    sender=parsed.sender,
                    recipient=parsed.recipient,
                )
            except Exception as exc:
                QMessageBox.critical(self, "Ошибка анализа", str(exc))
```

```
                return
            self.collected.append({
                "sender": parsed.sender,
                "recipient": parsed.recipient,
                "subject": parsed.subject,
                "body": parsed.body,
                "sent_at": parsed.sent_at,
                "category": analysis["category"],
                "confidence": analysis["confidence"],
                "is_spam": analysis["is_spam"],
                "is_phishing": analysis["is_phishing"],
                "summary": analysis["summary"],
                "entities": analysis["entities"],
            })
            self.progress.setValue(i)

        self.accept()
```

Листинг B.49 — cli.py

```
"""CLI для быстрого ad-hoc-анализа.

Запуск: `python -m email_analyzer.cli analyze --subject "тема" --body "..."`
"""

from __future__ import annotations

import argparse
import json
import sys

from email_analyzer.backend.services.analysis import AnalysisService


def _cmd_analyze(args: argparse.Namespace) -> int:
    service = AnalysisService()
    result = service.analyze(
        subject=args.subject,
        body=args.body,
        sender=args.sender or "",
    )
    data = {
        "category": result.category.value,
        "confidence": result.confidence,
        "is_spam": result.is_spam,
        "is_phishing": result.is_phishing,
        "summary": result.summary,
        "entities": result.entities,
    }
    print(json.dumps(data, ensure_ascii=False, indent=2))
    return 0


def _cmd_version(_args: argparse.Namespace) -> int:
    from email_analyzer import __version__
    print(__version__)
    return 0
```

```
def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="email-analyzer")
    sub = parser.add_subparsers(dest="command", required=True)

    analyze_p = sub.add_parser("analyze", help="проанализировать одно письмо")
    analyze_p.add_argument("--subject", default="", help="тема письма")
    analyze_p.add_argument("--body", default="", help="тело письма")
    analyze_p.add_argument("--sender", default="", help="адрес отправителя")
    analyze_p.set_defaults(func=_cmd_analyze)

    version_p = sub.add_parser("version", help="показать версию")
    version_p.set_defaults(func=_cmd_version)

    args = parser.parse_args(argv)
    return args.func(args)


if __name__ == "__main__":
    sys.exit(main())
```

Листинг B.50 — scripts/init_db.py

```
"""Инициализация БД + seed-пользователей.

Запуск: `python scripts/init_db.py`
"""

from __future__ import annotations

import sys
from pathlib import Path

# Добавляем src в sys.path, чтобы работало без установки пакета.
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

from email_analyzer.backend.services.users import UserService
from email_analyzer.db.models import Role
from email_analyzer.db.session import get_engine, init_db
from sqlalchemy.orm import sessionmaker


SEED_USERS = [
    ("admin", "admin@local.test", "admin123", Role.ADMIN, "Администратор"),
    ("analyst", "analyst@local.test", "analyst123", Role.ANALYST, "Аналитик"),
    ("user", "user@local.test", "user123", Role.USER, "Обычный пользователь"),
]


def main() -> int:
    init_db()
    engine = get_engine()
    factory = sessionmaker(bind=engine, autoflush=False, autocommit=False)
    db = factory()
    try:
        service = UserService(db)
        for username, email, password, role, full_name in SEED_USERS:
```

```
            if service.get_by_username(username) is not None:
                print(f"skip: {username} уже существует")
                continue
            service.create(
                username=username,
                email=email,
                password=password,
                full_name=full_name,
                role=role,
            )
            print(f"created: {username} (role={role.value})")
        db.commit()
    finally:
        db.close()
    print("БД инициализирована.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Листинг B.51 — scripts/build_exe.py

```
"""Сборка исполняемого .exe через PyInstaller.

Требование методички: «Обязательно должен быть сформирован исполняемый
*.exe файл». Собираем клиент PyQt6 в один файл.
"""

from __future__ import annotations

import subprocess
import sys
from pathlib import Path


ROOT = Path(__file__).resolve().parents[1]
ENTRY = ROOT / "src" / "email_analyzer" / "client" / "main.py"
NAME = "EmailAIAnalyzer"


def main() -> int:
    cmd = [
        sys.executable,
        "-m",
        "PyInstaller",
        "--onefile",
        "--windowed",
        f"--name={NAME}",
        "--paths",
        str(ROOT / "src"),
        str(ENTRY),
    ]
    print("Запуск:", " ".join(cmd))
    result = subprocess.run(cmd, cwd=ROOT, check=False)
    if result.returncode != 0:
        print("Сборка завершилась с ошибкой", file=sys.stderr)
        return result.returncode
```

```
    print(f"\n.exe собран: dist/{NAME}.exe")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```